

LLM：大语言模型结构

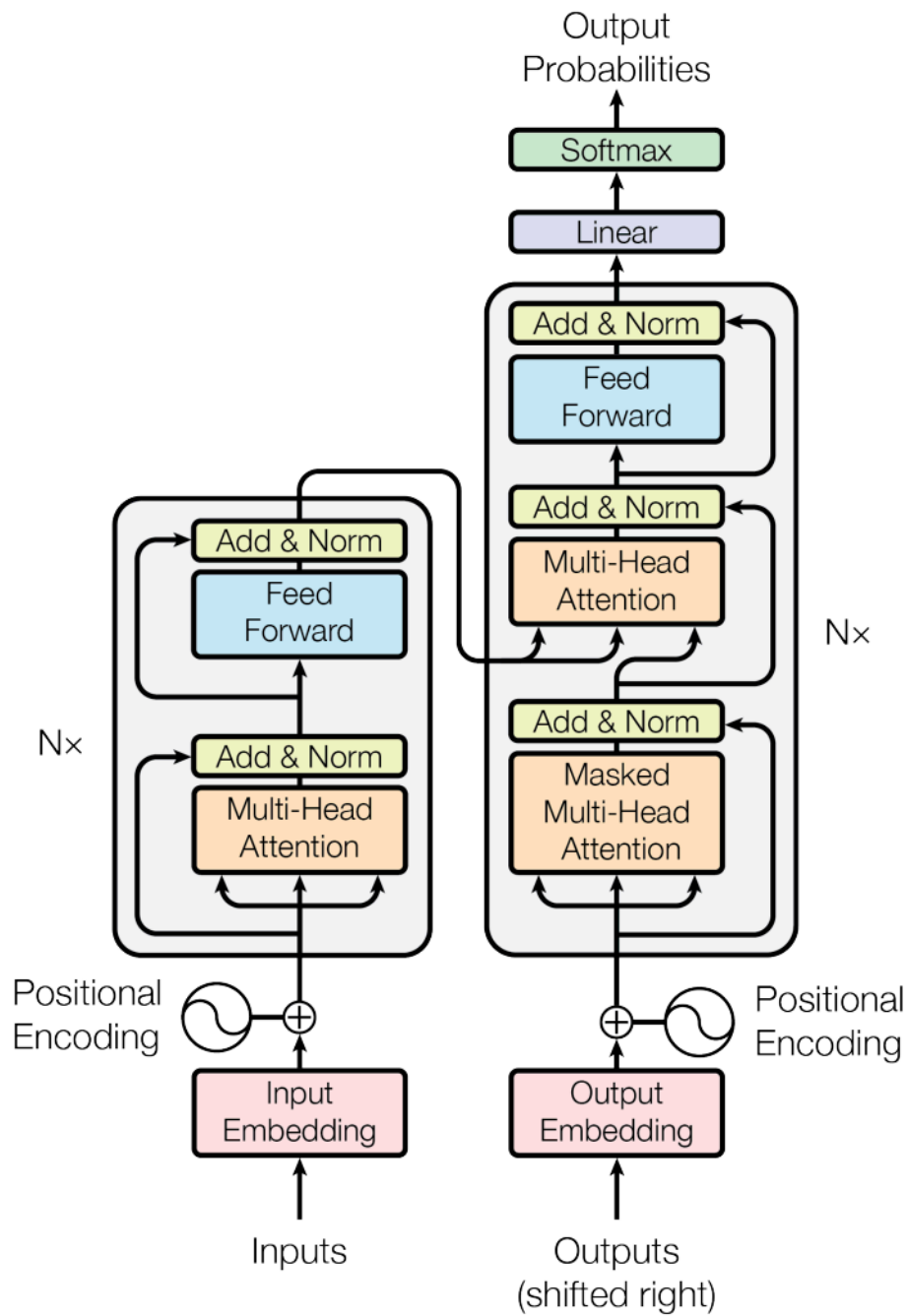
1、大语言模型

大语言模型是通过无监督学习训练的神经网络模型，特别是在大量的文本数据上进行预训练。其核心思想是让模型从上下文中预测词汇或子词的概率分布，进而捕捉语言的语法、语义、上下文信息等。

目前大语言模型的基础架构源于 Transformer，在 Transformer 出现之前，循环神经网络（RNN）及其变体如长短期记忆网络（LSTM）和门控循环单元（GRU）是处理自然语言序列的主要架构，但它们存在顺序处理导致的训练效率低和难以捕捉长距离依赖等问题。Transformer 的自注意力机制能够并行计算序列中各个位置的表示，大大提高了训练效率，使得训练大规模的 LLM 成为可能。

2、Transformer架构

Transformer 通过引入自注意力机制（Self-Attention），解决了传统 RNN（循环神经网络）和 LSTM（长短期记忆网络）在处理长序列时的效率瓶颈和梯度消失问题。Transformer 在并行计算和长距离依赖建模上具有显著的优势。



(/img/post/llm-model-1.png)

Transformer 由两个主要部分组成：编码器（Encoder）和解码器（Decoder）。这两个部分都由多个相同的层堆叠而成，每一层的结构相同。

- 编码器（Encoder）：接收输入序列，并将其编码为一个固定维度的表示
- 解码器（Decoder）：根据编码器输出的表示生成目标序列

2.1、编码器

Transformer 的编码器负责接收输入序列并将其转换为一个固定维度的表示，供解码器使用。每个编码器层包括以下几个子层：

1. **自注意力机制 (Self-Attention)**：计算输入序列中每个词与其他词的关系。通过“注意力”机制，模型能关注到序列中与当前词相关的重要信息
2. **前馈神经网络 (Feed-Forward Network)**：是一个全连接的神经网络，用于处理每个词的表示，增加模型的非线性表示能力
3. **残差连接和层归一化 (Residual Connection and Layer Normalization)**：每个子层都有一个残差连接，并且在每个子层之后执行层归一化，帮助训练时稳定梯度

编码器的每个层都会接收来自前一层的输出，并继续处理信息。编码器的输出是输入序列的表示，可以传递给解码器或用于其他任务。

2.2、解码器

解码器的任务是根据编码器的输出生成目标序列，解码器的结构与编码器类似但有部分区别：

1. **自注意力机制 (Self-Attention)**：不同于编码器，解码器的自注意力是掩蔽的 (Masked)，即当前时间步只能关注当前位置及之前的词，避免泄漏未来的信息
 2. **编码器-解码器注意力 (Encoder-Decoder Attention)**：解码器不仅要注意到目标序列中的信息，还要注意编码器的输出，即对源序列的表示。这部分机制计算解码器的查询（来自解码器的自注意力）与编码器的键和值之间的关系
 3. **前馈神经网络 (Feed-Forward Network)**：与编码器一样，解码器的每一层也包含一个前馈神经网络，该网络与编码器中的前馈神经网络相同
 4. **残差连接和层归一化 (Residual Connection and Layer Normalization)**：与编码器一样，解码器也使用残差连接和层归一化来稳定训练
-

2.3、自注意力机制

Self-Attention（自注意力）是 Transformer 模型的核心组件之一，它是用于捕捉输入序列中各个元素之间关系的一种机制。与传统的 RNN 或 CNN 不同，Self-Attention 并不依赖于顺序信息，而是通过计算输入序列中每个元素（Token）与其他所有元素的关系，来动态地决定它们之间的依赖关系，这是使模型能够处理长距离依赖的关键。

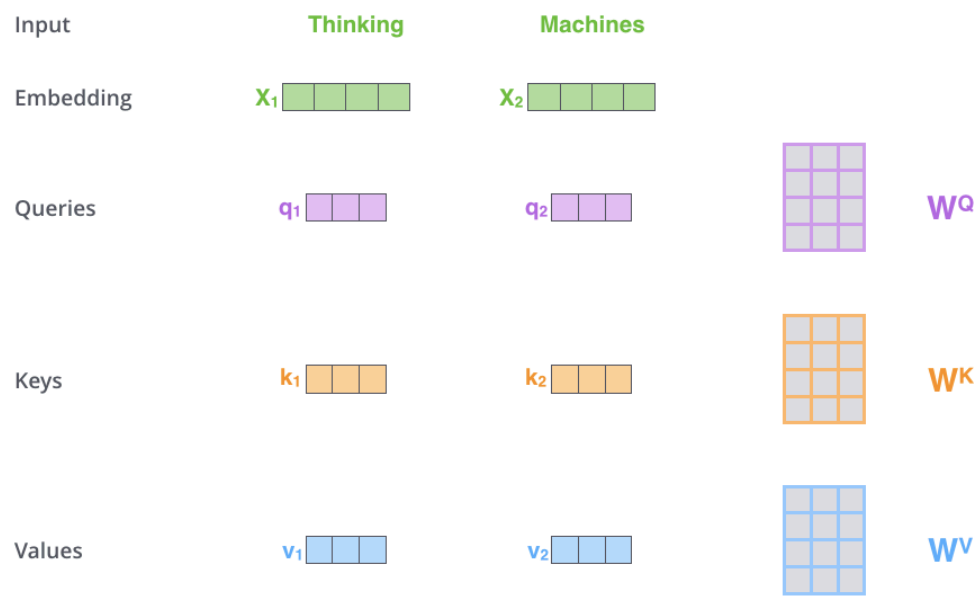
2.3.1、自注意力计算步骤

假设输入序列的长度为 n ，每个输入词的表示是一个 d 维向量，那么每个输入向量都需要通过和三个矩阵做点乘转换为查询（Query）、键（Key）、值（Value）向量：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

其中：

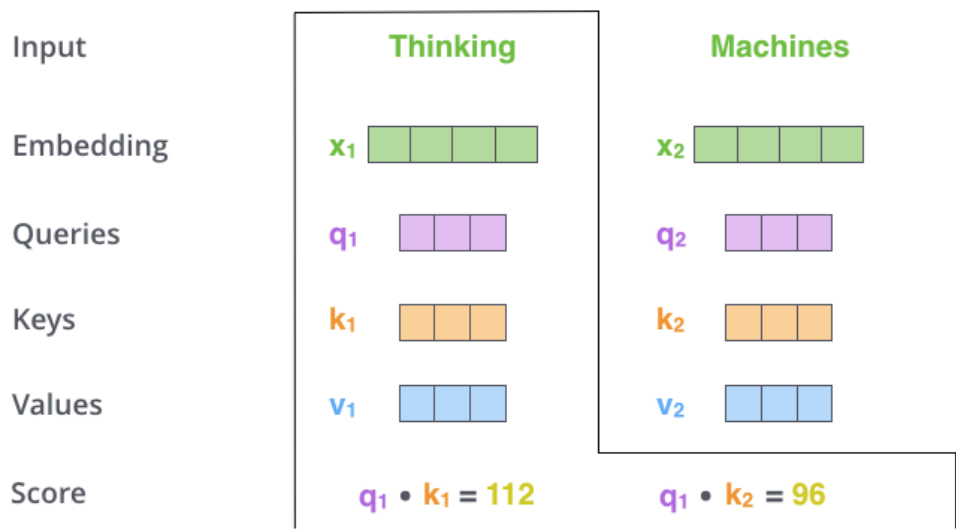
- $X \in \mathbb{R}^{n \times d}$ 是输入序列的嵌入表示
- $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ 是可训练的权重矩阵
- Q, K, V 的维度通常比词向量 X 的维度要小，如下图中词向量为512维， Q, K, V 为64维



步骤一：计算相似度（注意力权重）

通过计算当前词的 **Query** 与其他所有词的 **Key** 的点积，得到相似度得分。表示当前 Token 与其他 Token 的关系强弱。

$$\text{Attention Score} = Q \cdot K^T$$



(/img/post/llm-model-3.png)

步骤二：缩放点积

为了防止在高维情况下点积的值过大，通常会将点积结果除以一个缩放因子 $\sqrt{d_k}$ ，其中 d_k 是 Query 和 Key 的维度。

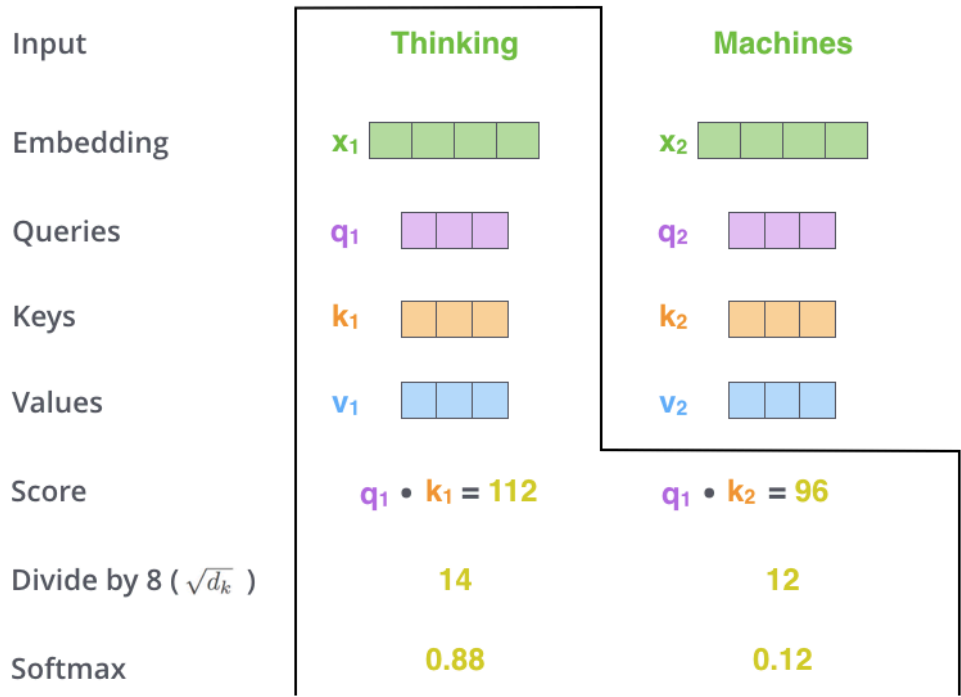
$$\text{Scaled Attention Score} = \frac{Q \cdot K^T}{\sqrt{d_k}}$$

缩放的作用： 缩放使得注意力权重更平滑，避免梯度爆炸并加速模型收敛，提高训练稳定性。具体的，Query 和 Key 的维度 d_k 的越大，两者点积的数值也会非常大。而 Softmax 是一个指数型函数，对输入值进行指数运算，放大输入值的差异。如果点积非常大，Softmax 函数的输出会变得非常偏向一个特定的 Token（即大部分的注意力权重集中在一个位置），从而丧失多样性，导致梯度不稳定（梯度爆炸）或者极端的注意力分布。将点积的值缩小，可以确保 Softmax 的输出不至于过于极端，保持稳定的注意力权重分布

步骤三： Softmax

对相似度得分应用 Softmax 函数，得到一个归一化的权重分布。这个分布表示每个 Token 对其他 Token 的注意力权重。

$$\text{Attention Weights} = \text{Softmax} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right)$$

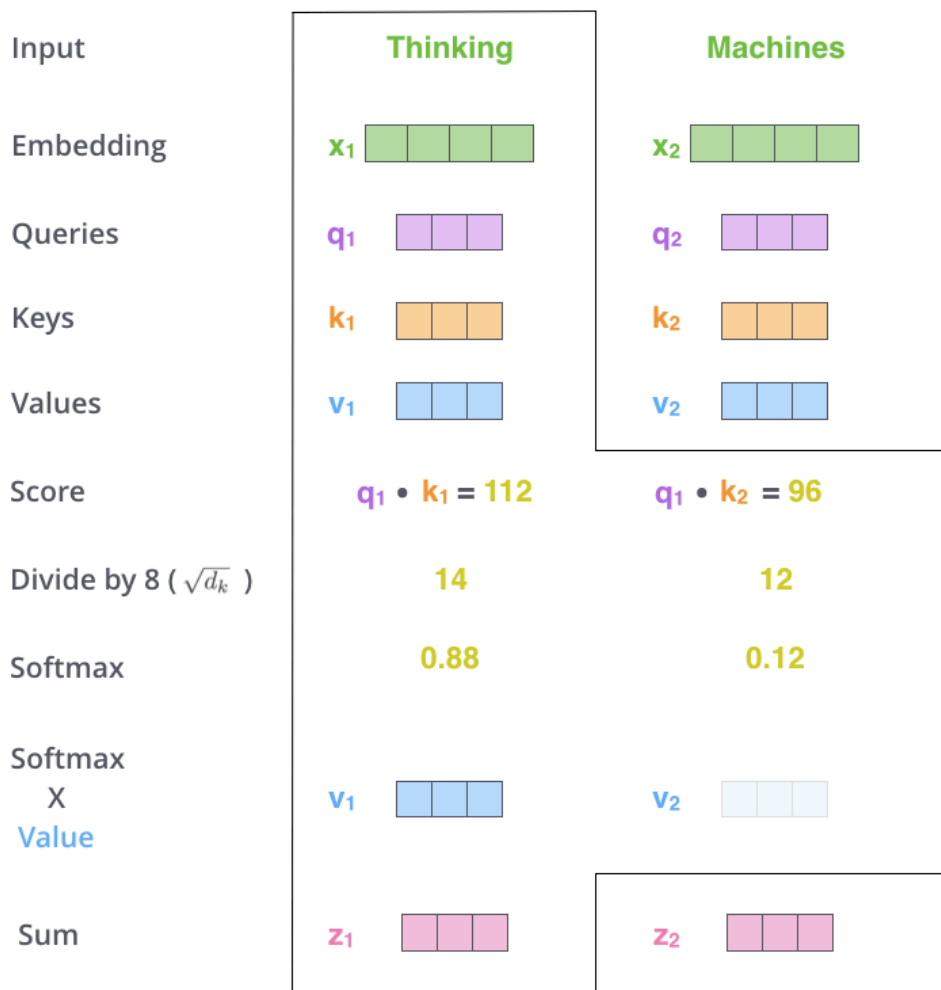


(/img/post/llm-model-4.png)

步骤四：加权求和

使用计算出的注意力权重对 **Value** 向量进行加权求和，得到当前 Token 的新表示。

$$\text{Output} = \text{Attention Weights} \cdot V$$



(/img/post/llm-model-5.png)

综上，自注意力机制可以用以下公式描述：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \text{3x3 grid} \end{matrix} \times \begin{matrix} \text{K}^T \\ \text{3x3 grid} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} \text{V} \\ \text{3x3 grid} \end{matrix}$$

$$= \begin{matrix} \text{Z} \\ \text{3x3 grid} \end{matrix}$$

(/img/post/llm-model-6.png)

2.3.2、多头注意力

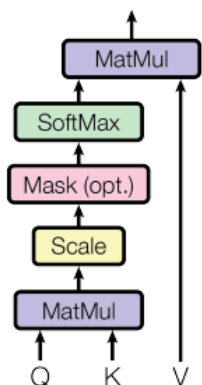
为了让模型能够从多个不同的角度理解信息，Transformer使用了多头注意力机制。即多个独立的注意力头会同时对输入序列进行计算，然后将它们的结果进行拼接，并通过一个线性变换得到最终的输出：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

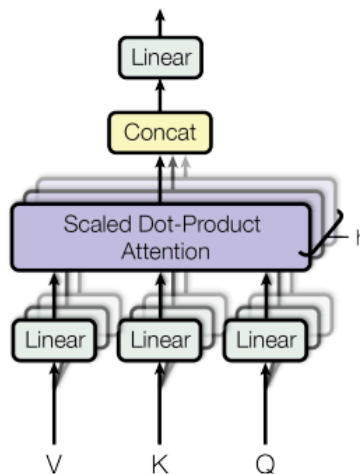
其中每个头的计算是：

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Scaled Dot-Product Attention



Multi-Head Attention



(/img/post/llm-model-7.png)

2.4、词嵌入和位置编码

词嵌入 (Word Embeddings) 是模型的输入之一，它将自然语言中的离散词汇转换为连续的向量。在 Transformer 模型中，词嵌入通常是由一个查找表 (Lookup Table) 来实现，该表存储了每个词的向量表示。每个词在 查找表/词汇表 中都有一个对应的向量，通常这些向量的维度较高 (如 $d_{\text{model}} = 512$ 或 1024)。

因为 Transformer 本身没有处理序列顺序的能力 (如 RNN 序列结构可以显式建模输入数据的顺序)，其自注意力机制是完全并行的。为了让模型能够感知 单词/Token 的顺序信息，通过给每个输入加上位置的 **位置编码** (Positional Encoding)，使模型能够捕捉到词语之间的顺序关系。

在 Transformer 中，位置编码的作用是通过一个固定的 (或可学习的) 向量，向每个词的表示中注入位置信息。具体地，位置编码与词嵌入向量逐元素相加，得到每个词的最终表示：

$$\text{Input}_i = \text{Embedding}(x_i) + \text{PE}_i$$

其中：

- $\text{Embedding}(x_i)$ 是词 x_i 的词嵌入向量
- PE_i 是词 x_i 的位置编码向量

2.4.1、位置编码

在原始的 Transformer 论文中 (《Attention Is All You Need》)，采用了正弦 (sin) 和余弦 (cos) 函数来生成位置编码。对于序列中的位置 pos 和维度索引 i ：

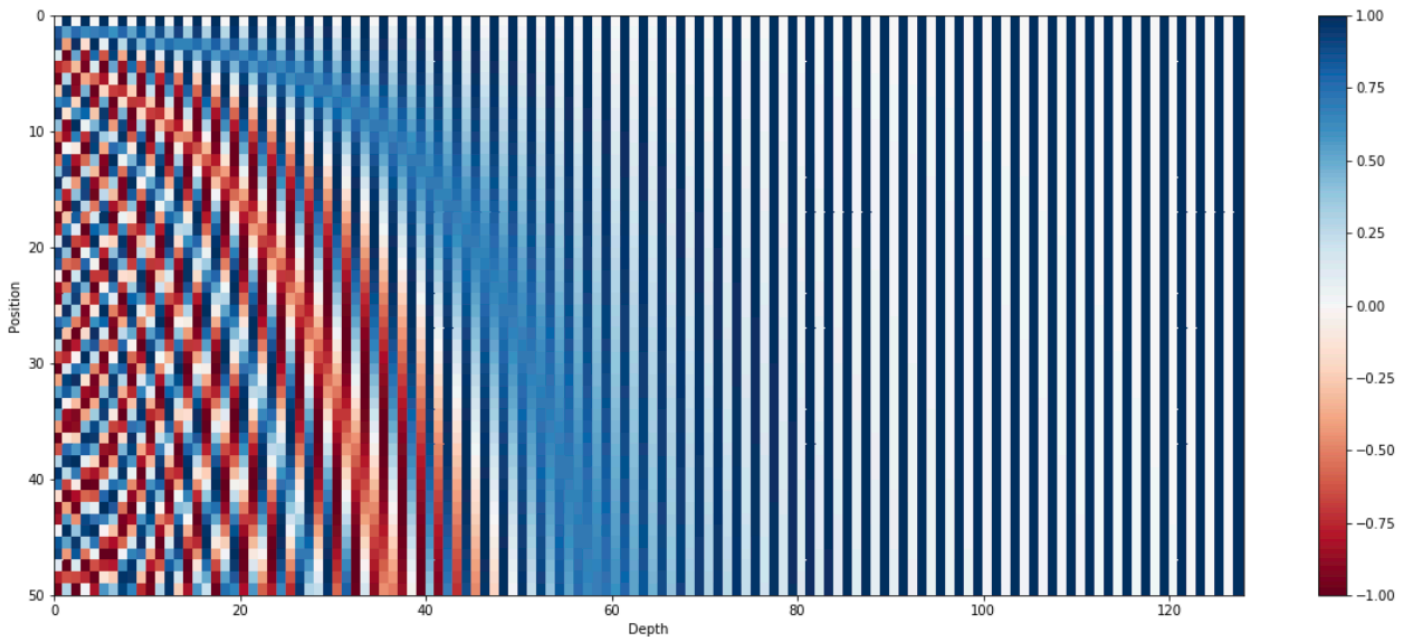
$$\begin{aligned} \text{PE}_{(pos, 2i)} &= \sin \left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}} \right) \\ \text{PE}_{(pos, 2i+1)} &= \cos \left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}} \right) \end{aligned}$$

其中 pos 为 Token 的位置， i 为向量的维度索引， d_{model} 为模型的隐藏层维度 (例如 512)。

由于正弦和余弦函数的值始终在 $[-1, 1]$ 之间，因此它们不会对位置编码的尺度造成极端影响，避免与位置编码相加时破坏原有单词语义信息。

正弦和余弦函数的周期与 $10000^{2i/d_{\text{model}}}$ 成反比，通过这个缩放因子，函数频率会随着向量维度 i 的增大而减小， i 决定了周期的大小，使得每一维的位置编码都对位置变化有不同的反应。较低的维度对较小的位置变化（局部位置）敏感，即高频的变化适合捕捉较短的依赖关系（如相邻词语之间的关系），而较远的词位置会有更低频率的变化，适合捕捉较长的依赖关系。

如图，最大长度为 50 的句子的 128 维位置编码，每行代表一个嵌入向量：



(/img/post/llm-model-8.png)

此外，正余弦函数使模型更容易学习根据相对位置进行注意力分配，因为对于任何固定的偏移量 k 而言， PE_{pos+k} 都可以表示为 PE_{pos} 的线性函数。

位置嵌入为什么与词嵌入相加而不是拼接？一旦信息到了高层之后，位置信息不就消失了嘛？为什么同时使用正弦和余弦？

(https://kazemnejad.com/blog/transformer_architecture_positional_encoding/)

2.5、前馈神经网络

前馈神经网络（Feed-Forward Neural Network, FFN）位于自注意力（Self-Attention）层之后，前馈神经网络由两个线性变换和一个非线性激活函数组成，对于每个位置共享相同的权重参数，通过非线性转换提高模型表达能力。

设输入为 $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ ，那么前馈神经网络的计算过程可以表示为：

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}W_1 + b_1)W_2 + b_2$$

其中：

- $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ 和 $b_1 \in \mathbb{R}^{d_{\text{ff}}}$ 是第一层的权重和偏置；
- $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ 和 $b_2 \in \mathbb{R}^{d_{\text{model}}}$ 是第二层的权重和偏置；
- $\max(0, \cdot)$ 是ReLU激活函数

1. **第一层线性变换** ($\mathbf{x}W_1 + b_1$) 将输入的维度从 d_{model} 映射到一个更大的维度 d_{ff} ，使得网络可以在更高维空间中进行更复杂的表示学习
2. **ReLU激活** ($\max(0, \cdot)$) 引入非线性，使得模型能够学习更加复杂的模式
3. **第二层线性变换** ($\cdot W_2 + b_2$) 将维度从 d_{ff} 映射回原始的 d_{model} ，以保持输出的维度与输入一致

2.6、残差连接和层归一化

2.6.1、残差连接

残差连接的主要目的是解决训练过程中因网络过深导致可能遇到的梯度消失问题，通过将输入直接加到输出上来进行“跳跃连接”，使得信息能够直接通过网络的每一层：

$$\mathbf{X}^{l+1} = \mathbf{X}^l + \mathcal{F}(\mathbf{X}^l)$$

其中， $\mathbf{X}^l$ 是第 l 层的输入， $\mathcal{F}(\mathbf{X}^l)$ 是该层的计算输出， $\mathbf{X}^{l+1}$ 是通过残差连接的最终输出。

2.6.2、层归一化

层归一化（Layer Normalization）对每一层的输出进行标准化，使得每层输入保持均值为0、方差为1。其目的是避免由于输入数据的分布不同而导致训练不稳定的问题。与批量归一化（Batch Normalization）不同，层归一化是在单个样本的层内部进行标准化，而不是在批次维度上进行标准化。

$$\mathbf{y} = \gamma \cdot \left(\frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \right) + \beta$$

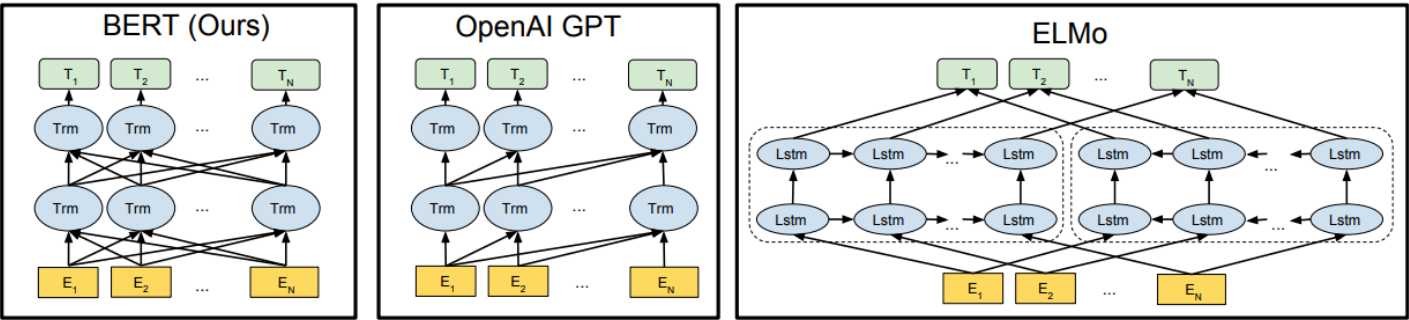
其中， $\mathbf{x}$ 是输入向量， μ 和 σ^2 分别是均值和方差， γ 和 β 是可训练的缩放和偏移参数， ϵ 是防止除以零的小常数。

3、BERT

BERT（**Bidirectional Encoder Representations from Transformers**）是一种基于 Transformer 架构的预训练语言模型，其在自然语言理解（NLU）任务中取得了革命性的进展，并成为了很多下游任务（如问答、文本分类、命名实体识别等）的基础模型。BERT 的核心创新是采用了 **双向编码器** 和 **预训练-微调范式**。

3.1、双向编码器

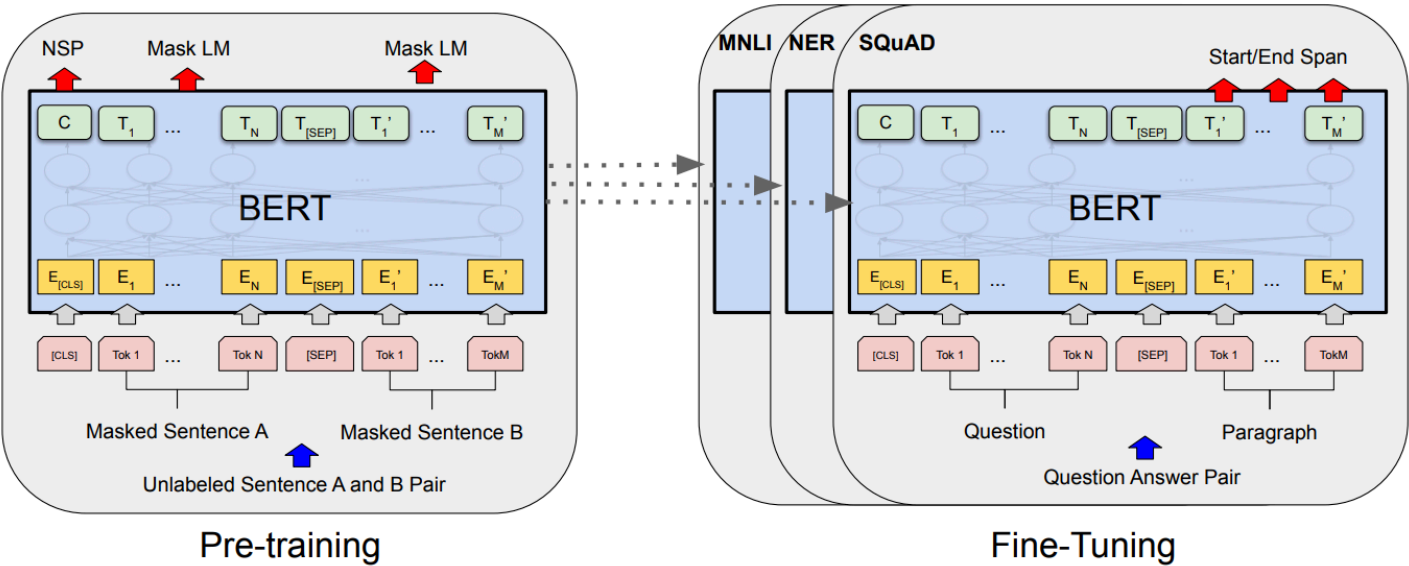
BERT 采用双向 Transformer 编码器（Transformer 由编码器和解码器两部分组成）。OpenAI GPT 使用从左到右的 Transformer，ELMo 使用独立训练的从左到右和从右到左 LSTM 的串联来生成下游任务的特征。只有 BERT 的表征是在所有层中都同时基于左右上下文进行条件计算的。BERT 的“双向”同时考虑了上下文中的所有词汇，能够同时从左到右和从右到左捕捉语境信息，能够更好地理解词汇在特定上下文中的意义。



(/img/post/llm-model-9.png)

3.2、预训练-微调范式

BERT 采用了预训练（Pre-training）和微调（Fine-tuning）的策略。首先，BERT 在大规模文本数据上进行预训练，学习一般的语言表示。然后，可以根据具体的下游任务（例如情感分析、问答、文本分类等）进行微调。这个预训练-微调的框架使得BERT在许多不同任务中都能够取得出色的性能。



(/img/post/llm-model-10.png)

如图，除了输出层之外，预训练和微调都使用相同的架构。BERT 使用相同的预训练模型参数来初始化不同下游任务的模型。在微调过程中，所有参数都会被微调。

3.3、预训练

BERT的预训练分为两个主要任务：

3.3.1、Masked Language Model (MLM)

在预训练阶段，BERT 不会像传统语言模型那样基于前面的单词来预测下一个单词，而是随机遮掩（mask）输入句子中的一些单词，任务是让模型根据上下文推断出这些被遮掩的词汇。具体来说，BERT 会从输入序列中随机选择 15% 的单词，将它们替换为特殊的 [MASK] 标记，模型的任务是通过交叉熵损失预测这些 [MASK] 位置上原本是什么单词。

由于微调过程中不会出现 [MASK] 标记，造成了在预训练和微调之间不匹配。为了缓解这一问题，BERT 并不总是用实际的 [MASK] 标记来替换“被屏蔽的”单词，即：

- 80% 的情况下用 [MASK] 标记
- 10% 的情况下随机替换成另一个词
- 10% 的情况下保留原来的词

3.3.2、Next Sentence Prediction (NSP)

NSP 任务帮助 BERT 理解句子间的关系。模型需要判断第二个句子是否是第一个句子的下文（即是否有连贯性）。训练时给出两对句子，模型的任务是判断第二个句子是否是第一个句子的连续句子。50% 的概率下，第二个句子是实际的下一个句子，另外 50% 的概率下，第二个句子是从语料库中随机选出的句子。

3.4、BERT 模型结构

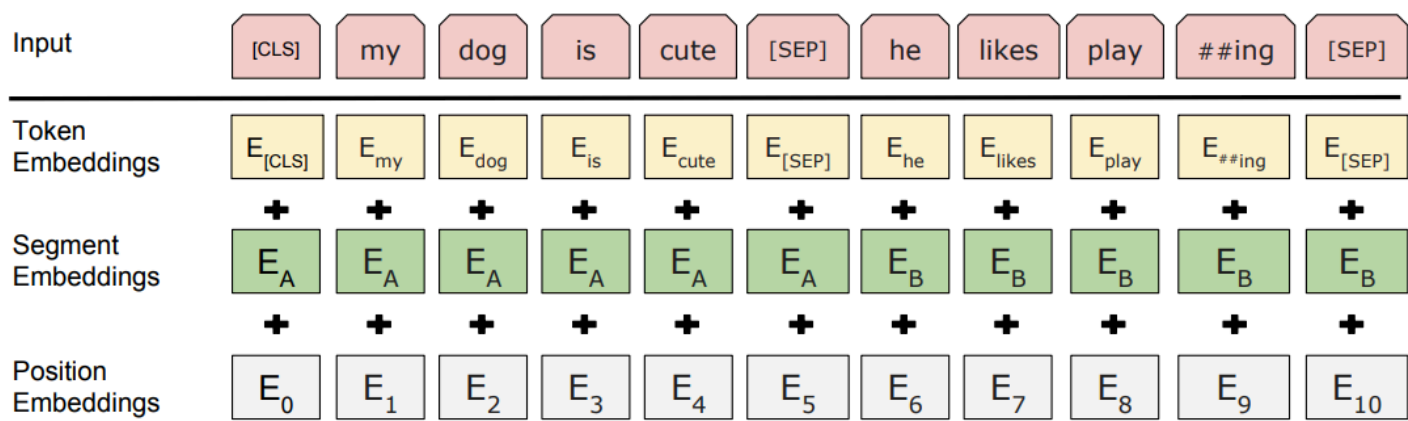
3.4.1、输入表征

BERT 的输入表示包括了三个部分：

- **Token Embeddings**: 词汇的词嵌入表示 (WordPiece分词)
- **Segment Embeddings**: 标记输入序列中句子的不同部分 (用于区分句子A和句子B)
- **Position Embeddings**: 表示每个词在序列中的位置 (由于Transformer本身不包含位置信息, 需要加入位置嵌入)

对于输入的每个token t_i , BERT的输入为:

$$\text{Input}_i = \text{Token Embedding}_i + \text{Segment Embedding}_i + \text{Position Embedding}_i$$



(/img/post/llm-model-11.png)

其中, [CLS] 是添加在每个输入示例前面的特殊符号, 与该标记对应的最终隐藏状态用作分类任务的聚合序列表示。[SEP] 是特殊的分隔符标记 (例如分隔问题/答案)。

3.4.2、Transformer 编码器

BERT 采用的是 Transformer 的**编码器**部分, 将层数 (即 Transformer 模块数) 记为 L , 隐藏层 (Transformer 的输出) 大小记为 H , 自注意力头的数量记为 A , 通常有 BERT 基础版 ($L = 12, H = 768, A = 12$, 总参数 = 1.1 亿) 和 BERT 大型版 ($L = 24, H = 1024, A = 16$, 总参数 = 3.4 亿)。

3.5、BERT 变体

3.5.1、ALBERT

ALBERT (A Lite BERT) 是 BERT 的轻量化版本，旨在通过参数共享和模块优化，减少模型大小和内存占用，同时保持甚至提升模型性能。

- **跨层参数共享**：为了防止参数随网络深度增长，ALBERT 强制所有层共享参数（包括自注意力机制和前馈网络的参数），显著减少了模型参数数量
- **分解嵌入矩阵**：ALBERT 将词嵌入矩阵分解为两个较小的矩阵，即通过参数 E 将大小为 $V \times H$ 的 Embedding 矩阵分解成 $V \times E + E \times H$ (V ：词汇表长度， H ：词的隐藏层大小)
- **预测句子顺序**：ALBERT 使用了 **Sentence Order Prediction (SOP)** 替代 NSP 任务，即要求模型判断两个句子是否是按正确顺序排列的。SOP 更注重句子之间的逻辑顺序，而 NSP 在预训练中存在噪声问题（随机采样句子可能与连续句子无法明显区分）

尽管参数量减少，但 Transformer 层数未变，在推理过程中仍然需要依次遍历每一层，因此 ALBERT 推理速度并没有显著提升。

3.5.2、RoBERTa

RoBERTa (Robustly Optimized BERT Pretraining Approach) 旨在通过优化预训练策略来提升模型性能。

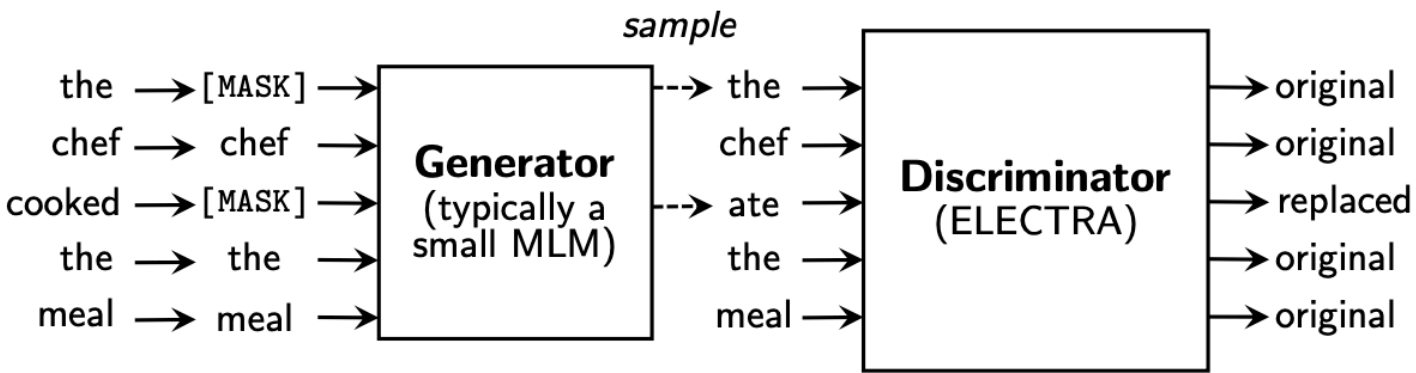
- **动态掩码**：与 BERT 的静态掩码不同（mask 是在预处理阶段固定的），RoBERTa 在每个训练 epoch 中重新随机选择 Mask 的位置，使模型能够学习更多的上下文表示
- **移除NSP任务**：NSP 没有显著帮助语言模型性能，RoBERTa 移除了 NSP 目标
- **大数据集训练**：通过使用更长的输入序列、更大的批次、更多的数据和更长的时间进行训练
- **文本编码**：RoBERTa 使用更大的字节级 BPE 词汇表，无需额外的预处理或分词输入，这在处理大型和多样化的语料库时具有优势

3.5.3、ELECTRA

ELECTRA (Efficiently Learning an Encoder that Classifies Token Replacements Accurately) 与传统的 **Masked Language Modeling (MLM)** 目标不同，其引入了 **Replaced Token Detection (RTD)** 的预训练目标。

ELECTRA 的结构由两部分组成：

- **生成器 (Generator)**：使用 MLM 目标进行训练，用于生成替换词，替换原始句子中的部分 Token
- **判别器 (Discriminator)**：对输入的每个 Token 判断它是否被替换过

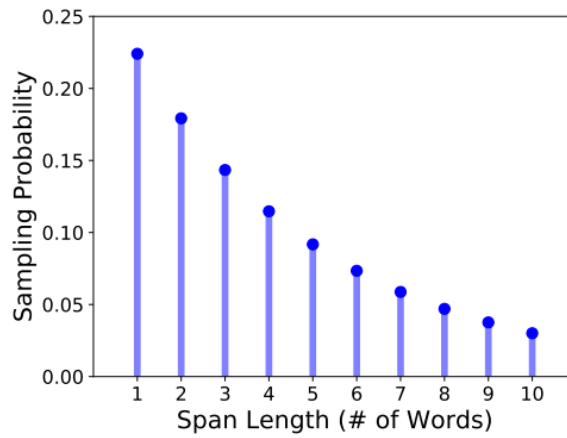


(/img/post/llm-model-12.png)

3.5.4、SpanBERT

SpanBERT 是一种针对句子中 **连续片段 (Span)** 的表示学习优化的预训练模型，专为捕获和生成更强的跨片段关系表示而设计，其在需要处理跨句或跨 Token 片段的任务中表现出色。

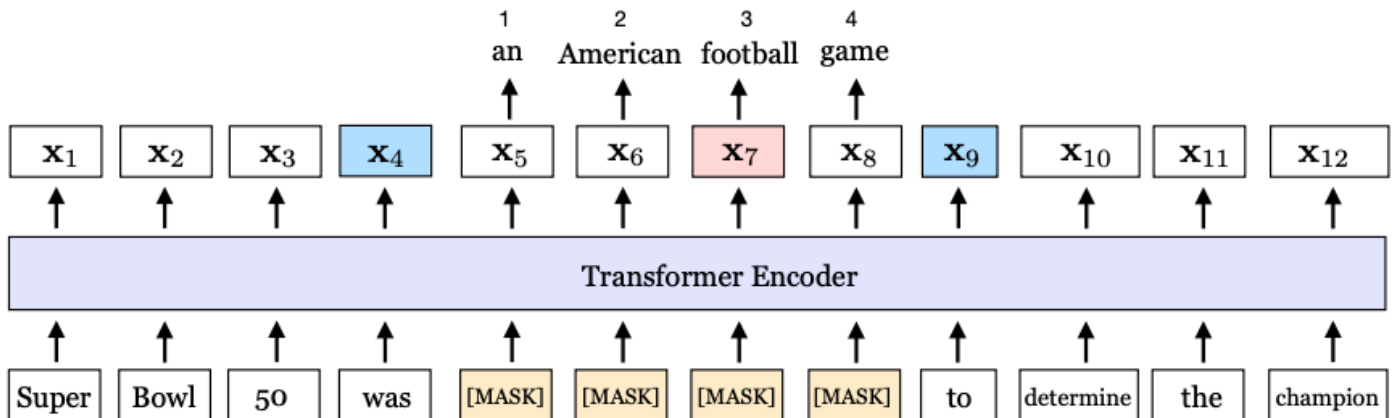
- **Span Masking**：随机选择句子中的连续片段 (Span) 进行 Mask：
 - 每次 Mask 时，随机选择 Mask 起点
 - 按照几何分布随机确定 Mask 的长度（几何分布的特性更符合自然语言文本中跨度长度从短到长逐渐减少的趋势，使得模型能够接触到各种长度的跨度，且频率与实际情况相符）



◦ (/img/post/llm-model-14.png)

- **Span Boundary Objective (SBO)**: 给定一个被 Mask 的 Span, 使用 Span 两端的边界 Token 的表示来预测 Span 中被 Mask 的每个 Token 的内容, 迫使模型利用 Span 边界信息, 构建更强的整体片段表示

$$\begin{aligned}\mathcal{L}(\text{football}) &= \mathcal{L}_{\text{MLM}}(\text{football}) + \mathcal{L}_{\text{SBO}}(\text{football}) \\ &= -\log P(\text{football} \mid \mathbf{x}_7) - \log P(\text{football} \mid \mathbf{x}_4, \mathbf{x}_9, \mathbf{p}_3)\end{aligned}$$



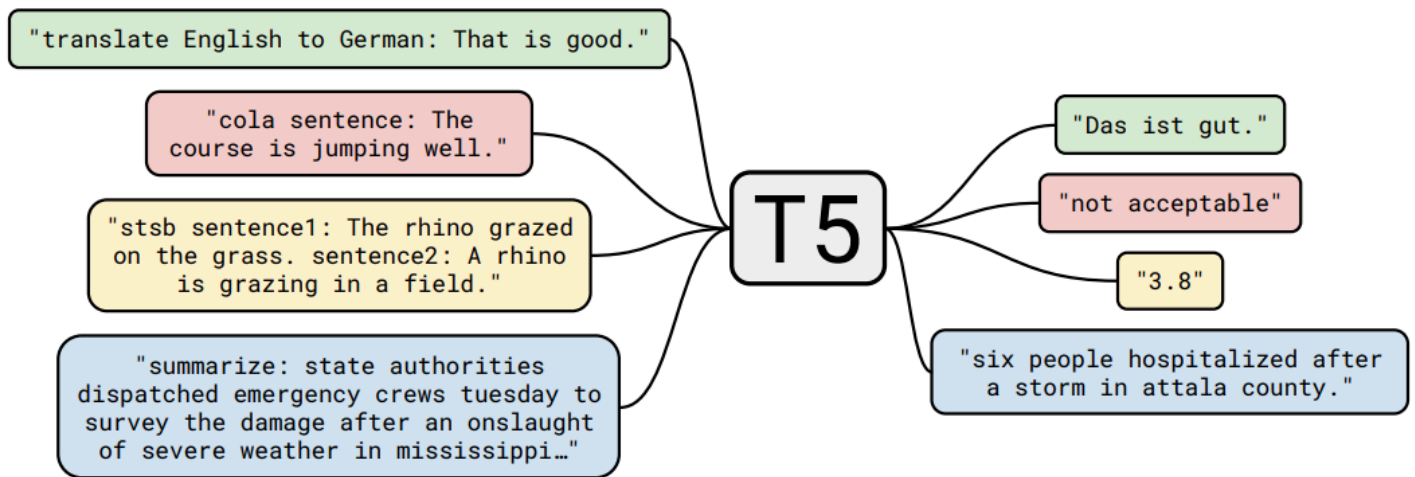
(/img/post/llm-model-13.png)

4、T5

T5 (Text-to-Text Transfer Transformer) 的核心思想是将所有自然语言处理任务统一表示为一个“文本到文本”的生成任务, 即将输入和输出都表示为文本字符串, 从而在统一框架中解决不同类型的任务。

每项任务（包括翻译、问答和分类）都被视为将模型文本作为输入并训练它生成一些目标文本。这使我们能够在不同的任务集中使用相同的模型、损失函数、超参数等。

- 示例：
 - 分类任务：输入 `text: This movie is great! sentiment:`，输出 `positive`
 - 翻译任务：输入 `translate English to French: The book is on the table.`，输出 `Le livre est sur la table.`
 - 填空任务：输入 `fill in the blanks: The ____ is on the table.`，输出 `book`

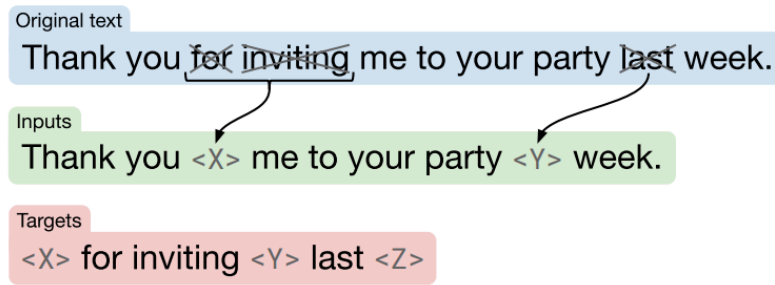


(/img/post/llm-model-15.png)

T5 采用标准的 Transformer 编码器-解码器（Encoder-Decoder）架构，相较于 BERT 的仅编码器架构，T5 的 Seq2Seq 结构更适合文本生成任务：

- **编码器**：负责对输入文本进行上下文表示建模
- **解码器**：基于编码器输出和自身的上下文生成目标文本

T5 采用了一种改良的填空任务（span corruption）作为预训练目标，即在输入中随机遮盖一些片段（span），并让模型预测这些片段。



(/img/post/llm-model-16.png)

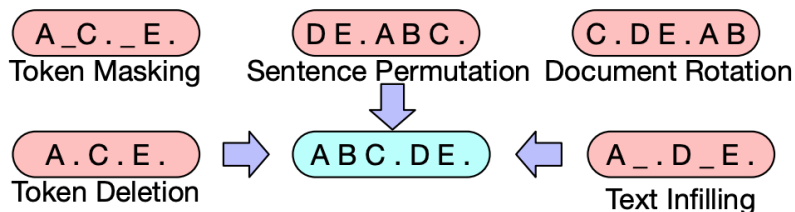
T5 常见的参数量大小有 6000 万 (T5-Small)、2.2 亿 (T5-Base)、7.7 亿 (T5-Large)、30 亿 (T5-3B) 和 110 亿 (T5-11B) 等。

5、BART

BART (Bidirectional and Auto-Regressive Transformer) 是一种基于 Transformer 架构的生成式自然语言处理模型，专为序列到序列 (Seq2Seq) 任务设计。BART是一个标准的**编码器-解码器架构**：

- **编码器**：双向 Transformer，用于编码输入序列的上下文信息（类似于 BERT）
- **解码器**：自回归 Transformer，用于生成输出序列（类似于 GPT）

BART通过 **去噪自编码器** 任务进行预训练，其过程是对输入文本进行随机的破坏（如掩码、删除、重排等操作），然后让模型重建原始文本：



(/img/post/llm-model-17.png)

- **Token Masking**：随机遮盖部分token，类似BERT的掩码语言模型
- **Token Deletion**：随机删除一些token
- **Text Infilling**：随机用占位符替换多个连续token，模拟短语缺失
- **Sentence Permutation**：随机打乱句子的顺序

- **Document Rotation**: 将文本的起始位置移至随机位置，模拟文档旋转

BART 常见的参数量有 1.35 亿 (BART-base) 和 4.06 亿 (BART-large)。

6、GPT

GPT (Generative Pre-trained Transformer) 是生成式预训练语言模型，基于 Transformer 架构，专注于通过自回归的方式生成自然语言文本，即给定一个输入序列 $x = x_1, x_2, \dots, x_t$ ，模型学习预测下一个单词 x_{t+1} 的条件概率 $P(x_{t+1} | x_1, \dots, x_t)$ 。

6.1、GPT-1

GPT-1 于 2018 年发布，采用了仅有解码器的 Transformer 架构，参数量为 1.17 亿，其通过无监督学习在海量文本数据上预训练，并在特定任务上进行判别式微调来实现自然语言理解任务的显著提升。

6.1.1、无监督预训练

给定一个无监督的标记语料库 $\mathcal{U} = \{u_1, \dots, u_n\}$ ，使用标准的自然语言建模目标来最大化以下似然：

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

其中 k 是上下文窗口的大小，条件概率 P 使用具有参数 Θ 的神经网络建模。这些参数使用随机梯度下降法进行训练。

6.1.2、监督微调

在预训练模型的基础上，使用相应的监督目标对模型进行微调。假设一个标记的数据集 $\mathcal{C}$ ，其中每个实例包括一系列输入标记 $x^1, \dots, x^m$ ，以及标签 y 。输入通过预训练模型得到最终 Transformer 块的激活 h_l^m ，然后将其输入到一个线性输出层，其参数为 W_y ，以预测 y ：

$$P(y | x^1, \dots, x^m) = \text{softmax}(h_l^m W_y).$$

其对应的目标函数为：

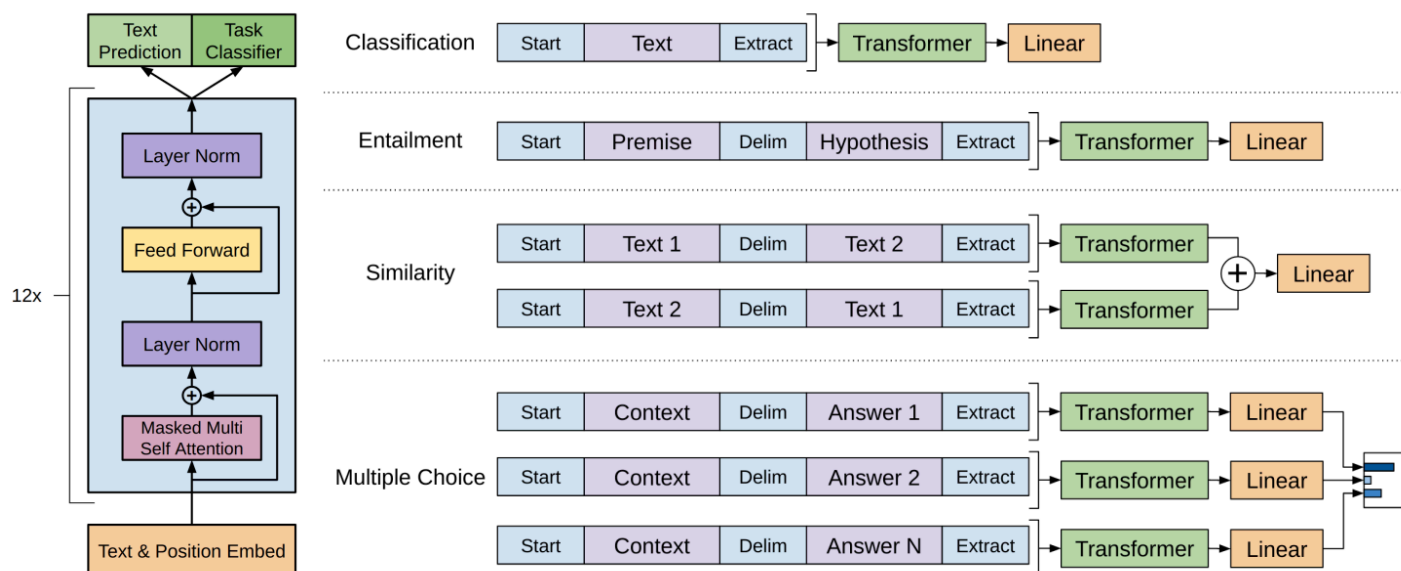
$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y | x^1, \dots, x^m).$$

为了提高监督模型的泛化能力，并加速收敛，引入语言建模作为辅助目标：

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda * L_1(\mathcal{C})$$

6.1.3、针对特定任务的输入转换

对于某些任务（如文本分类），可以直接进行微调。但对于其他任务（如问答或文本蕴含），需要将结构化输入转换为顺序序列。例如，对于文本蕴含任务，将前提 p 和假设 h 连接起来，中间加上分隔符。



(/img/post/llm-model-18.png)

6.2、GPT-2

GPT-2 发布于 2019 年，继承了 GPT-1 的架构，并将参数规模扩大到 15 亿。GPT-2 尝试通过增加模型参数规模来提升性能，并探索使用无监督预训练的语言模型来解决多种下游任务，而无需显式地使用标注数据进行微调。

6.2.1、语言建模

语言建模通常被看作是从一组示例 $(x_1, x_2, \dots, x_n)$ 中进行无监督的概率分布估计，每个示例由可变长度的符号序列 $(s_1, s_2, \dots, s_n)$ 组成。由于语言具有自然的顺序，通常将符号的条件概率分解为条件概率的乘积：

$$p(x) = \prod_{i=1}^n p(s_i | s_1, \dots, s_{i-1})$$

这种方法允许对 $p(x)$ 以及形如 $p(s_{n-k}, \dots, s_n | s_1, \dots, s_{n-k-1})$ 的任何条件概率进行易于处理的采样和估计。

6.2.2、多任务学习

学习执行单一任务可以表示为在概率框架内估计条件分布 $p(\text{output} \mid \text{input})$ ，为了使系统能够执行多个不同的任务，即使对于相同的输入，系统应该不仅依赖于输入，还依赖于要执行的任务。即，系统应建模 $p(\text{output} \mid \text{input}, \text{task})$ 。

6.2.3、训练数据集

创建名为 WebText 的新数据集，主要通过抓取 Reddit 上的链接来获取文本。为了提高文档质量，要求链接至少获得 3 次点赞，WebText 包含 4500 万个链接的文本子集。为了从 HTML 响应中提取文本，使用 Dragnet 和 Newspaper1 内容提取器的组合进行去重和清理，最终得到包含 800 万篇文档，总计约 40 GB 的文本。

6.2.4、输入表示

使用字节级编码（BPE）作为输入表示，避免了字符级别和词级别的限制。**BPE（Byte Pair Encoding）** 是一种基于字符的无监督的分词算法，它通过反复合并最常见的字符对（byte pairs）来构建词汇表，使得模型能够处理词汇中未见的词（OOV，Out-of-Vocabulary）并提高文本表示的效率。

具体的，BPE 会扫描文本，统计所有字节对（相邻的两个字符）的出现频率。接着选取出现频率最高的字节对并将其合并为一个新的子词单元，然后更新词汇表和文本中的所有出现。例如，若 “ab” 是最频繁出现的字节对，它会将 “ab” 视为一个新单元，将文本中的所有 “ab” 替换为这个新单元，并将 “ab” 添加到词汇表中。这个过程会持续迭代，词汇表不断扩大，同时文本表示会变得更紧凑。

6.3、GPT-3

GPT-3 发布于 2020 年，使用了与 GPT-2 相同的模型架构，但其参数规模扩展到了 1750 亿。GPT-3 引入 “上下文学习（In-context learning）” 概念，允许大语言模型通过少样本学习解决各种任务，消除了对新任务进行微调的需求。

6.3.1、In-context learning

In-context learning (ICL) 利用模型在预训练阶段获得的大量知识和语言规则，通过设计任务相关的指令和提示模板，引导模型在新的测试数据上生成预测结果。ICL 允许模型在没有显式微调的情况下，通过在输入上下文中提供少量示例来学习新任务。

GPT-3 系统分析了在同一下游任务中，在不同设置下模型学习能力的差异，这些设置可以被视为处于一个反映对任务特定数据依赖程度的范围之内：

- **Fine-Tuning (FT)**：通过数千到数万个下游任务的监督数据集上更新预训练模型的权重来进行训练。其主要缺点是需要为每个任务创建一个新的大型数据集，可能会在分布外泛化不佳，并且可能会利用训练数据中的虚假特征导致与人类性能的不公平比较。GPT-3 没有采用微调
- **Few-Shot (FS)**：推理时向模型提供任务的几个示例作为条件，但不允许更新权重。少样本学习大大减少了对特定任务数据的需求，并降低了从一个大而狭窄的微调数据集学习到过窄分布的可能性。但这种方法的结果比微调的 SOTA 模型的效果差很多
- **One-Shot (1S)**：单样本与少样本相同，除了任务的自然语言描述外，只允许使用一个示例。将单样本与少样本和零样本区分开来的原因是，它最符合向人类传达某些任务的方式
- **Zero-Shot (0S)**：零样本不允许使用示例，并且仅向模型提供描述任务的自然语言指令

The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.



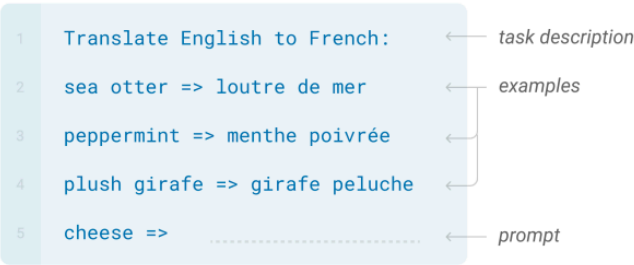
One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.



Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



Traditional fine-tuning (not used for GPT-3)

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



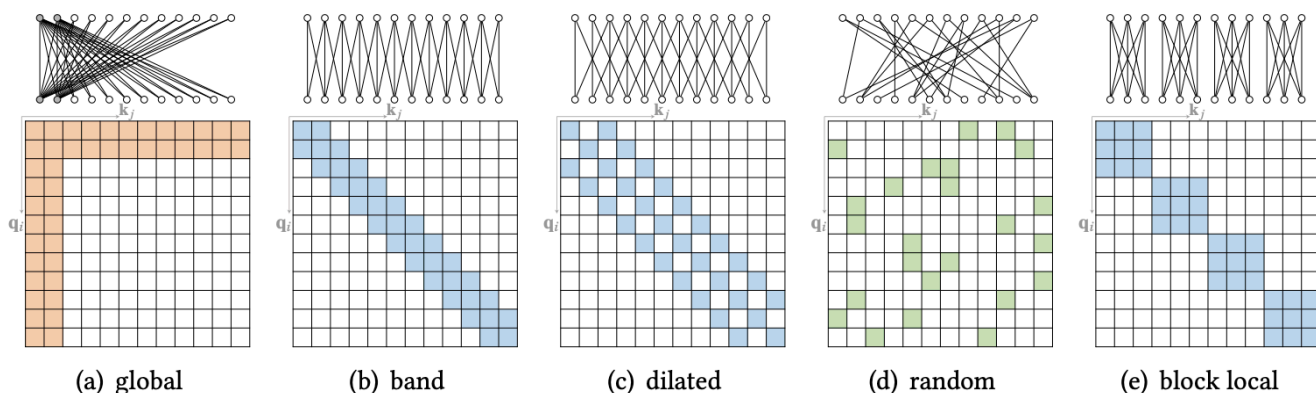
(/img/post/llm-model-19.png)

6.3.2、模型架构

GPT-3 使用与 GPT-2 相同的模型和架构，包括初始化、预归一化和可逆分词，不同之处在于 Transformer 的各层中使用交替的密集和局部带状 稀疏注意力 模式（类似于 Sparse Transformer）。

稀疏注意力机制

稀疏注意力机制 (Sparse Attention Mechanism) 是一种优化 Transformer 模型中计算效率和内存使用的技术。由于标准的自注意力机制需要计算所有词对之间的注意力权重，这导致了随着输入序列长度增加，计算和内存开销呈二次增长 $O(n^2)$ 。稀疏注意力机制通过限制每个单词只能关注一部分其他单词，从而减少了计算复杂度。



(/img/post/llm-model-20.png)

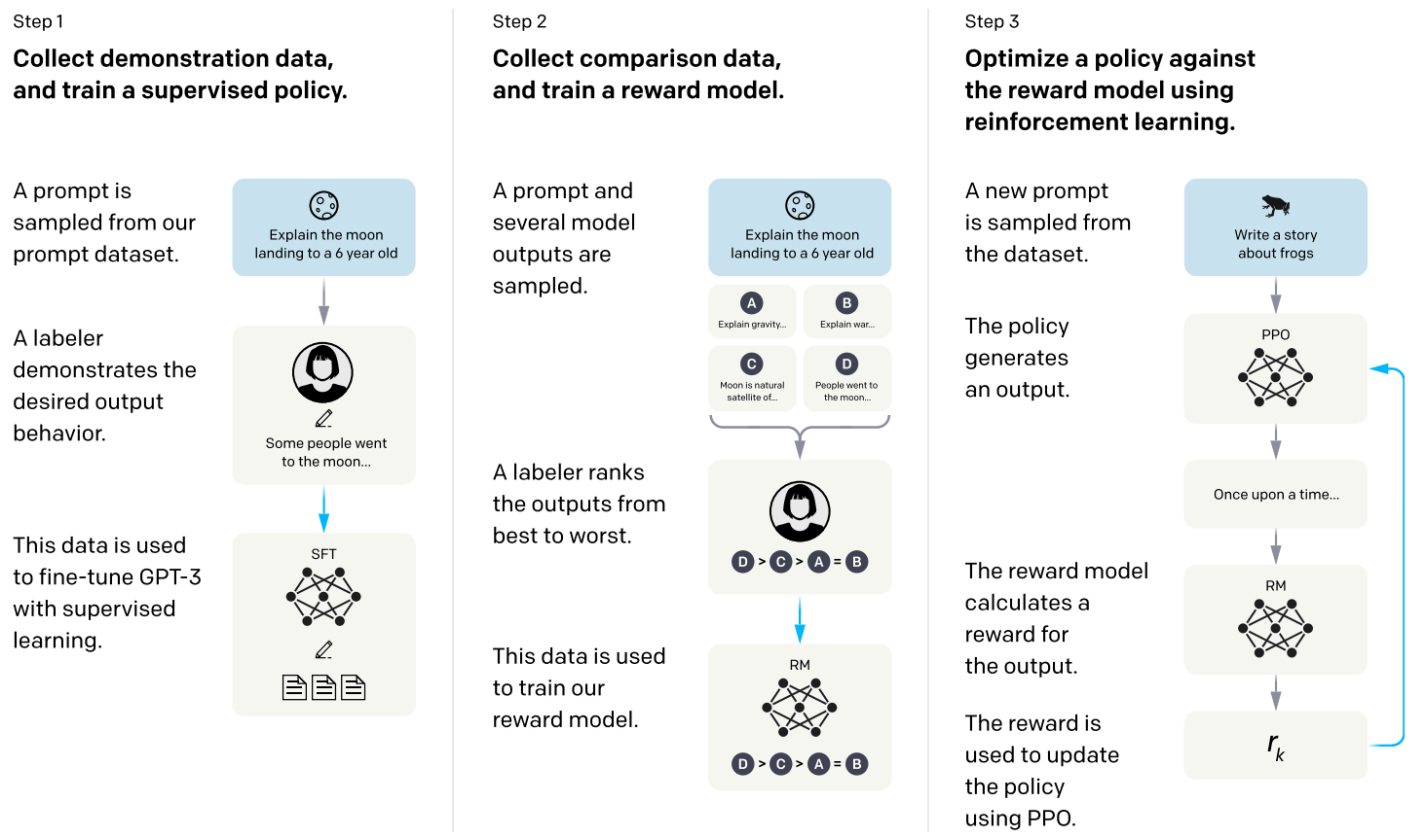
- **Global Attention**: 为了缓解稀疏注意力在模拟长距离依赖能力上的退化，可以添加一些全局节点作为节点间信息传播的枢纽
- **Band Attention**: 注意力权重被限制在一个固定的窗口中，每个 Query 只关注其邻居节点
- **Dilated Attention**: 类似于扩张卷积神经网络，通过使用扩张，可以在不增加计算复杂度的情况下增加 Band Attention 的感受野
- **Random Attention**: 为了增强非局部交互的能力，对每个 Query 随机抽取一些边
- **Block Local Attention**: 将输入序列分割成几个不重叠的查询块，每个查询块都与一个局部记忆块相关联，查询块中的所有 Query 只关注相应记忆块中的 Key

6.4、InstructGPT

InstructGPT 在 GPT-3 的基础上，建立了基于人类反馈的强化学习算法 RLHF，通过代码数据训练和人类偏好对齐进行了改进，旨在提高指令遵循能力，并缓解有害内容的生成。

大型语言模型可能生成不真实、有毒或对用户毫无帮助的输出。InstructGPT 通过强化学习与人类反馈结合的方式，使语言模型在广泛的任务上生成的内容更加符合人类的期望。

InstructGPT 的训练步骤如下：



(/img/post/llm-model-21.png)

- 数据收集：**收集标签器编写的提示和通过 OpenAI API 提交的提示，形成了一个数据集。雇佣 40 名承包商对这些数据进行标注，以生成监督学习的基准数据
- 监督学习微调：**使用标签器的标注数据对预训练的 GPT-3 模型进行监督学习微调（SFT），得到 SFT 模型。SFT 模型的训练目标是最小化标签器对模型输出的偏好损失
- 奖励模型训练：**收集模型输出之间的比较数据，其中标注员指出他们对于给定输入更倾向于哪个输出。然后训练一个奖励模型（RM）来预测人类偏好的输出。奖励模型的输入是一个提示和一个响应，输出是一个标量奖励
- 强化学习优化：**使用近端策略优化（PPO）算法，以奖励模型作为标量奖励，对 SFT 模型进行进一步微调

6.4.1、数据集

数据集主要由提交给 OpenAI API 的文本提示（prompt）组成，通过检查提示是否有长的公共前缀来启发式地去重，并且限制每个用户 ID 的提示数量为 200。同时根据用户 ID 创建训练、验证和测试分割，以确保验证和测试集不包含训练集中的用户数据。

为了训练最初的 InstructGPT 模型，需要一个初始的指令样式的提示来源来启动这个过程，要求标注者编写三种类型的提示：

- **Plain**：简单地要求标注者想出一个任意的任务，同时确保任务的多样性
- **Few-shot**：要求标注者想出一个指令，并为该指令提供多个查询/响应对
- **User-based**：在 OpenAI API 的候补申请中收到了多个用例，要求标注者根据这些用例提出相应的提示词

基于这些提示（prompt）生成了用于微调过程中的三种不同数据集：

- **SFT 数据集**：来自 API 和标注者编写的 13k 个训练提示，包含标注者演示数据，用于训练 SFT 模型
- **RM 数据集**：来自 API 和标注者编写的 33k 个训练提示，包含标注者对模型输出的排名，用于训练奖励模型（RM）
- **PPO 数据集**：
仅来自 API 的 31k 个训练提示，不含任何人类标签，用于强化学习微调（RLHF）

对于每个自然语言提示，任务通常是通过自然语言指令直接指定的（例如，“写一个关于聪明青蛙的故事”），但也可以通过少量示例（例如，给出两个青蛙故事的例子，并提示模型生成一个新的故事）或隐含的延续（例如，提供关于青蛙的故事开头）间接指定。在每种情况下，要求标注者尽力推断写提示者的意图，并要求跳过任务非常不明确的输入。在最终评估中，要求标注员优先考虑真实性和无害性。

6.4.2、监督微调（SFT）

基于人工标注员编写并提供的示范回答，使用监督学习对 GPT-3 进行微调。模型训练 16 个周期，采用余弦学习率衰减，并设置了 0.2 的残差丢弃率（residual dropout）。根据验证集上的 RM 评分进行最终的 SFT 模型选择。SFT 模型在经过 1 个周期后会在验证损失上出现过拟合；然而，**尽管存在过拟**

合，训练更多周期仍然有助于提高 RM 评分和人类偏好评分。

方法	公式	特点
余弦学习率衰减	$\eta(t) = \eta_0 \times \frac{1}{2} \left(1 + \cos \left(\frac{\pi t}{T} \right) \right)$	平滑衰减，适合长时间训练，避免过早收敛
线性衰减	$\eta(t) = \eta_0 - \frac{\eta_0}{T} \times t$	每个训练周期减少固定量的学习率
指数衰减	$\eta(t) = \eta_0 \times e^{-\lambda t}$	学习率快速下降，适合较短的训练周期
阶梯衰减	设定特定的时间点减少学习率	每过一个固定的时间间隔，突然减少学习率

(/img/post/llm-model-22.png) **残差丢弃**：在残差连接的地方，丢弃来自前一层的部分残差信号（不是当前层的激活输出）

6.4.3、奖励建模（RM）

从移除最后的反嵌入层（unembedding layer：将模型的输出向量映射回词汇表中的一个词汇或子词）的 SFT 模型开始，训练一个模型以接受提示和响应，并输出一个**标量奖励**。论文中仅使用了 6B 的奖励模型（RM），因为这可以节省大量计算资源，并且发现使用 175B 奖励模型进行训练可能会导致不稳定，因此不太适合作为 RL 过程中值函数（value function）使用。

奖励模型（RM）基于同一输入的两个模型输出之间的对比组成的数据集上进行训练，模型使用**交叉熵损失**，以对比结果作为标签，其中奖励的差异代表了某个回应相比另一个回应更可能被人类标注员偏好的对数几率。

为了加快比较数据的收集速度，向标注员展示 4 到 9 个响应，并让他们对这些响应进行排序。这为每个提示（prompt）生成了 $\binom{K}{2}$ 个比较，其中 K 是展示给标注员的响应数量。

由于在每个标注任务中的对比之间有很强的相关性，如果简单地将对比打乱到一个数据集中，单次遍历该数据集会导致奖励模型发生过拟合。因此，将每个提示（prompt）的 $\binom{K}{2}$ 个对比作为单个 batch 来训练。这种方法在计算上更高效，因为仅需要为每个生成的回答进行一次**前向传播**（将多个样本放在一个批次中时，神经网络可以通过一次前向传播同时处理这些样本），而不是对 K 个生成进行 $\binom{K}{2}$ 次前向传递，并且因为避免了过拟合，在验证准确率和对数损失上有了显著的提升。

具体来说，奖励模型的损失函数是：

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x, y_w, y_l) \sim D} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))]$$

其中， $r_\theta(x, y)$ 是奖励模型在给定提示 x 和生成内容 y 的情况下，使用参数 θ 输出的标量值， y_w 是在一对 y_w 和 y_l 中被偏好的生成内容， D 是人类比较数据集。

在训练奖励模型（RM）时，损失函数对奖励值的**偏移**（即奖励的整体水平）是**不敏感的**。即，不管奖励的数值范围是多少，只要模型正确地比较不同生成内容的优劣，它的训练效果是不会受到奖励的整体偏移（或常数项）影响的。即在训练过程中，模型不需要特别关心奖励值的具体数值，只要相对顺序正确即可。

因此，通过引入偏置，使得标注数据在经过奖励模型处理后，将奖励分数归一化为**均值为 0 的分布**，使强化学习算法在学习过程中更好地根据奖励的相对大小和正负来调整行为，避免了因奖励值的绝对大小和初始偏差而导致的学习问题。

6.4.4、强化学习（RL）

使用 PPO 在给定环境中对 SFT 模型进行微调。该环境是一个 **多臂赌博机环境**，它会随机提供一个客户提示（customer prompt）并期望模型对该提示给出响应。根据提示和响应，环境会生成一个由奖励模型决定的奖励，并结束这一回合。此外，还在每个 token 上添加了来自 SFT 模型的 **每个token的 KL 惩罚**，以缓解奖励模型的过度优化。**价值函数** 是从奖励模型（RM）初始化的，并将这些模型称为 PP0。

InstructGPT 将 **预训练梯度** 与 **PPO梯度** 混合，以解决在公共 NLP 数据集上出现的性能退化问题。InstructGPT 将这些模型称为 PP0-ptx，并在 RL 训练中最大化以下联合目标函数：

$$\text{objective}(\phi) = E_{(x, y) \sim D_{\pi_{\phi}^{\text{RL}}}} \left[r_\theta(x, y) - \beta \log \left(\pi_{\phi}^{\text{RL}}(y | x) / \pi^{\text{SFT}}(y | x) \right) \right] + \gamma E_{x \sim D_{\text{pretrain}}} \left[\log \left(\pi_{\phi}^{\text{RL}}(x) \right) \right]$$

其中， π_{ϕ}^{RL} 是学习的 RL 策略， π^{SFT} 是监督训练的模型， D_{pretrain} 是预训练分布。KL 奖励系数 β 和预训练损失系数 γ 分别控制 KL 惩罚和预训练梯度的强度。对于 PP0 模型， γ 设置为 0。

6.5、GPT-4

GPT-4 发布于 2023 年，首次将输入模态从单一文本扩展到图文多模态。

7、LLaMA

LLaMA (Large Language Model Meta AI) 是发布于 2023 年 2 月的开源预训练大型语言模型，与 GPT 等生成模型类似，LLaMA 也只使用了 Transformer 的解码器部分。

7.1、LLaMA 1

LLaMA 是一组从 70 亿到 650 亿参数的基础语言模型，特别的，LLaMA-13B 在大多数基准测试中优于 GPT-3 (1750亿参数)。

7.1.1、预训练数据

数据及其在训练集中的比例：

Dataset	Sampling prop.	Epochs	Disk size
CommonCrawl	67.0%	1.10	3.3 TB
C4	15.0%	1.06	783 GB
Github	4.5%	0.64	328 GB
Wikipedia	4.5%	2.45	83 GB
Books	4.5%	2.23	85 GB
ArXiv	2.5%	1.06	92 GB
StackExchange	2.0%	1.03	78 GB

(/img/post/llm-model-23.png)

- **English CommonCrawl [67%]:** 使用 CCNet 流程对 2017 年至 2020 年间的五个 CommonCrawl 数据集进行了预处理。该过程在行级别去重，使用 fastText 线性分类器进行语

言识别以移除非英文页面，并使用 n-gram 语言模型过滤低质量内容。此外，训练了一个线性模型，用于将维基百科中的参考页面与随机抽取的页面进行分类，并丢弃那些未被分类为参考页面的页面

- **C4 [15%]**: 使用多样化的预处理 CommonCrawl 数据集可以提高性能，因此，将公开可用的 C4 数据集纳入数据中。C4 的预处理也包括去重和语言识别步骤：与 CCNet 的主要区别在于质量过滤，C4 主要依赖一些启发式方法，如页面中是否存在标点符号或网页中的单词和句子的数量
- **Github [4.5%]**: 使用 Google BigQuery 上可用的公共 GitHub 数据集。根据行长度或字母数字字符的比例使用启发式方法过滤低质量文件，并使用正则表达式移除样板文件（如标题）。最后，在文件级别对结果数据集进行了去重，去除了完全相同的文件
- **Wikipedia [4.5%]**: 添加了 2022 年 6 月至 8 月期间的维基百科转储，涵盖了 20 种使用拉丁字母或西里尔字母的语言。并对数据进行处理以去除超链接、评论和其他格式化的模板
- **Gutenberg and Books3 [4.5%]**: 训练数据集中包含了两个书籍语料库：古腾堡计划（Gutenberg Project）以及 ThePile 中的 Books3 部分。在书籍级别进行了去重，去除了那些内容重叠超过 90% 的书籍
- **ArXiv [2.5%]**: 处理 arXiv 上的 Latex 文件，以将科学数据添加到数据集中。去除第一节之前的内容以及参考文献部分，同时去除 .tex 文件中的注释，并展开了用户编写的内联定义和宏，以增加论文之间的一致性
- **Stack Exchange [2%]**: Stack Exchange 是一个高质量问题和答案的网站，涵盖了从计算机科学到化学等多种领域。数据集移除了文本中的 HTML 标签，并按分数（从高到低）对答案进行了排序

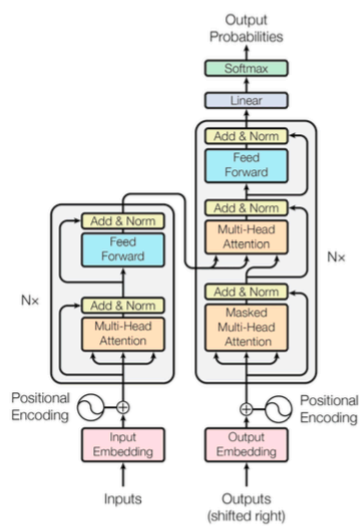
7.1.2、分词器

使用字节对编码（BPE）算法对数据进行分词。特别的，所有数字会拆分为单个数字，并在遇到未知的 UTF-8 字符时回退到使用字节进行分解。整个训练数据集在分词后大约包含 1.4 万亿个词元（token）。对于大多数训练数据，每个 token 在训练过程中仅使用一次，唯一的例外是维基百科和书籍领域数据进行了大约两轮的训练。

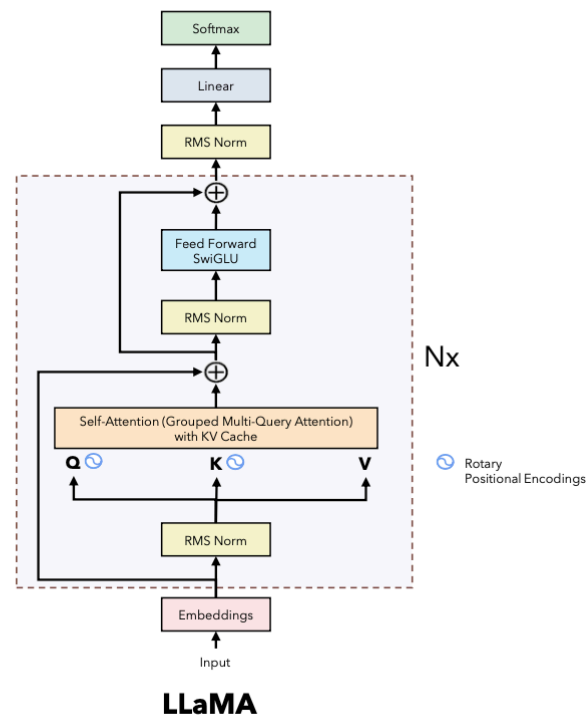
7.1.3、模型架构

LLaMA 网络基于 Transformer 架构，并在模型做了一些改进：

Transformer vs LLaMA



Transformer
("Attention is all you need")



LLaMA

(/img/post/llm-model-26.png)

- **预标准化：** 为了提高训练的稳定性，对每个 Transformer 子层的**输入**使用 RMSNorm（Root Mean Square Normalization）进行归一化，而不是对输出进行归一化
- **SwiGLU 激活函数：** 采用 SwiGLU（Swish-Gated Linear Unit）激活函数代替了 ReLU 非线性函数，使用的维度为 $\frac{2}{3} \times 4d$ 。SwiGLU 结合了 Swish 激活函数的非线性特性和门控机制，通过引入**门控机制**来控制信息的流动。相比于 ReLU，SwiGLU 在某些情况下能够更好地捕捉到复杂的模式，提高模型的表现力和性能
- **旋转位置嵌入：** 在网络的每一层添加旋转位置嵌入（RoPE）来取代传统的绝对位置嵌入方法。RoPE 通过在复数域中旋转位置向量来编码序列中的位置信息，这使得模型能够更有效地捕捉**相对位置**的信息

params	dimension	<i>n</i> heads	<i>n</i> layers	learning rate	batch size	<i>n</i> tokens
6.7B	4096	32	32	$3.0e^{-4}$	4M	1.0T
13.0B	5120	40	40	$3.0e^{-4}$	4M	1.0T
32.5B	6656	52	60	$1.5e^{-4}$	4M	1.4T
65.2B	8192	64	80	$1.5e^{-4}$	4M	1.4T

(/img/post/llm-model-24.png)

RMSNorm

RMSNorm (Root Mean Square Normalization) 对输入进行归一化时，不计算均值，而是基于 **均方根 (RMS)** 对输入进行标准化。

给定输入向量 $x = [x_1, x_2, \dots, x_n]$ ，RMSNorm 的计算公式如下：

1、计算输入向量 x 的均方根 (RMS) 值：

$$\text{RMS}(x) = \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}$$

2、对输入向量进行归一化处理：

$$\hat{x}_i = \frac{x_i}{\text{RMS}(x) + \epsilon}$$

其中 ϵ 是一个很小的正数 (如 $1e-8$)，其作用是避免分母为零，增强数值稳定性。

3、最后，通常会引入可学习的参数 γ (缩放因子) 对归一化后的向量进行缩放：

$$y_i = \gamma \hat{x}_i$$

相比于 **层归一化 (Layer Normalization)** 通过计算每个样本在所有特征上的均值和方差来进行归一化，RMSNorm 不进行去中心化操作，不需要计算均值只依赖于均方根，在计算上更加高效。

此外，**批归一化 (Batch Normalization)** 在训练时会计算每个批次数据的均值和方差；在推理时，使用训练阶段统计的全局均值和方差。但在处理变长序列数据 (如 NLP 中的文本) 时，因为不同样本的序列长度可能不同，难以计算统一的均值和方差。而 RMSNorm 不依赖批次统计信息，更适合处理变长序列。

SwiGLU

SwiGLU (Swish-Gated Linear Unit) 是一种激活函数，它结合了 Swish 激活函数和门控线性单元 (GLU) 的特性。

假设输入为 x ， W_1 、 W_2 是权重矩阵， b_1 、 b_2 是偏置向量。

首先，定义 **Swish** 函数：

$$\text{Swish}(x) = x \cdot \sigma(x)$$

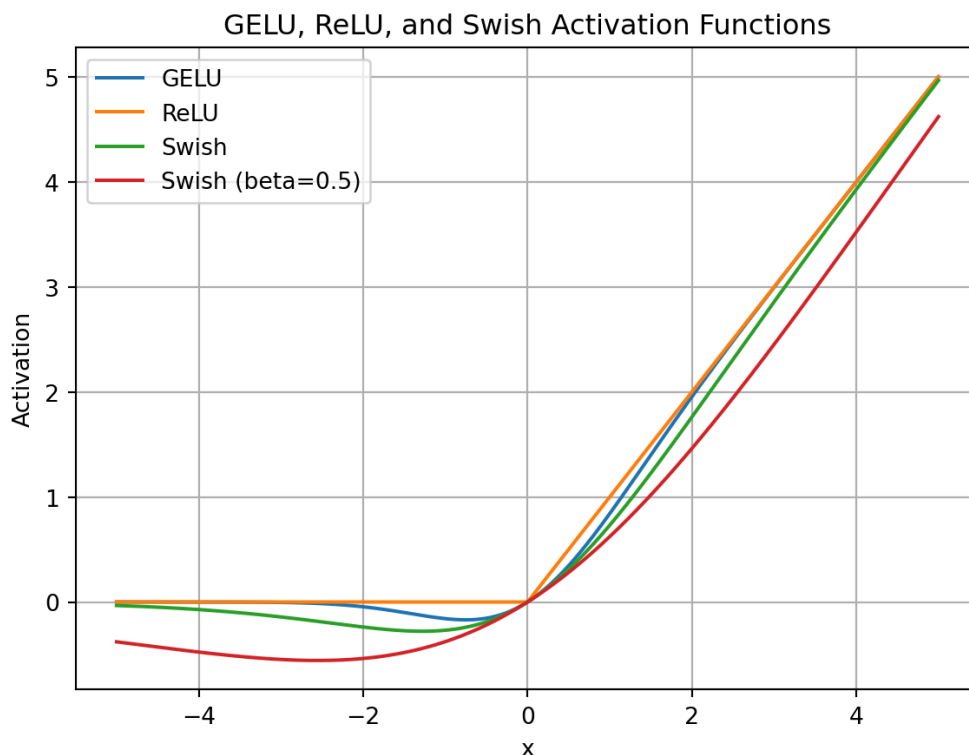
其中, $\sigma(x) = \frac{1}{1+e^{-x}}$ 是 **Sigmoid** 函数, 它将输入映射到 0 到 1 之间作为 **门控 (gating)** 系数, 从而动态地调节每个元素的影响力。

那么 SwiGLU 的计算公式为:

$$\text{SwiGLU}(x) = \text{Swish}(xW_1 + b_1) \odot (xW_2 + b_2)$$

其中, $\odot$ 表示逐元素相乘 (Hadamard积)。

相比传统的 ReLU ($f(x) = \max(0, x)$), SwiGLU 能有效避免死神经元问题 (当输入值小于 0 时, ReLU 的输出为 0, 并且在反向传播过程中, 这些神经元的梯度也为 0, 从而导致神经元在训练过程中由于梯度消失而无法更新)。此外, GLU 的门控机制使得 SwiGLU 可以动态地选择不同特征进行激活, 有效捕捉不同层次的语义信息, 具有更强的表达能力。



(/img/post/llm-model-27.png)

RoPE

RoPE (Rotary Positional Embedding) 通过对 Query 和 Key 向量施加特定的旋转变换操作来编码序列中每个位置的信息, 而不再依赖于固定的正弦和余弦函数或直接学习位置向量。

1、初步介绍:

设 $S_N = \{w_i\}_{i=1}^N$ 为一个包含 N 个输入标记的序列，其中 w_i 是第 i^{th} 个元素。相应的词嵌入表示为 $E_N = \{x_i\}_{i=1}^N$ ，其中 $x_i \in R^d$ 是标记 w_i 的 d 维词嵌入向量，且未包含位置信息。自注意力首先将位置信息融入词嵌入，然后将其转换为查询、键和值表示：

$$\begin{aligned} q_m &= f_q(x_m, m) \\ k_n &= f_k(x_n, n) \\ v_n &= f_v(x_n, n), \end{aligned} \quad (1)$$

其中 q_m, k_n 和 v_n 分别通过 f_q, f_k 和 f_v 融入第 m^{th} 和 n^{th} 位置。然后使用查询和键值来计算注意力权重，而输出则作为值表示的加权和计算得出：

$$\begin{aligned} a_{m,n} &= \frac{\exp\left(\frac{q_m^\top k_n}{\sqrt{d}}\right)}{\sum_{j=1}^N \exp\left(\frac{q_m^\top k_j}{\sqrt{d}}\right)} \\ o_m &= \sum_{n=1}^N a_{m,n} v_n \end{aligned} \quad (2)$$

2、绝对位置嵌入：

等式 (1) 的一个典型方法为：

$$f_{t:t \in \{q,k,v\}}(x_i, i) := W_{t:t \in \{q,k,v\}}(x_i + p_i) \quad (3)$$

其中 $p_i \in R^d$ 是一个依赖于标记 x_i 的位置的 d 维向量。Transformer 论文提出了使用正弦函数生成 p_i ：

$$\begin{cases} p_{i,2t} &= \sin\left(k/10000^{2t/d}\right) \\ p_{i,2t+1} &= \cos\left(k/10000^{2t/d}\right) \end{cases} \quad (4)$$

其中 $p_{i,2t}$ 是 d 维向量 p_i 的第 $2t$ 个元素。与此不同的，RoPE 不是通过直接将位置添加到上下文表示中来结合相对位置信息，而是通过乘以正弦函数来结合相对位置信息。

3、二维旋转位置编码：

为了引入相对位置信息，需要查询 q_m 和键 k_n 的内积通过函数 g 来形式化，该函数只取词嵌入 x_m, x_n 和它们的相对位置 $m - n$ 作为输入变量（希望内积仅以相对形式编码位置信息）：

$$\langle f_q(x_m, m), f_k(x_n, n) \rangle = g(x_m, x_n, m - n) \quad (5)$$

最终目标是找到一个等效的编码机制使上述关系成立。

从词向量的维度为简单的二维情况开始，利用二维平面上向量的几何属性及其复数形式来证明等式

(5) 的一个解决方案：

$$\begin{aligned} f_q(x_m, m) &= (W_q x_m) e^{im\theta} \\ f_k(x_n, n) &= (W_k x_n) e^{in\theta} \\ g(x_m, x_n, m - n) &= \text{Re} \left[(W_q x_m) (W_k x_n)^* e^{i(m-n)\theta} \right] \end{aligned} \quad (6)$$

其中 $\text{Re}[\cdot]$ 是复数的实部， $(W_k x_n)^*$ 表示 $(W_k x_n)$ 的共轭复数。 $\theta \in \mathbb{R}$ 是一个预设的非零常数。

进一步将 $f_{q,k}$ 写成乘法矩阵的形式：

$$f_{\{q,k\}}(x_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_{\{q,k\}}^{(11)} & W_{\{q,k\}}^{(12)} \\ W_{\{q,k\}}^{(21)} & W_{\{q,k\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix} \quad (7)$$

其中 $(x_m^{(1)}, x_m^{(2)})$ 是 x_m 在二维坐标系中的表示。同样， g 可以被视为一个矩阵，从而在二维情况下实现等式 (5) 的解决方案。结合相对位置编码是直接的：简单地通过其位置索引的角度倍数旋转仿射变换后的词嵌入向量，即 Query 向量乘以一个旋转矩阵。

最终 $g(x_m, x_n, m - n)$ 可以表示如：

$$g(x_m, x_n, m - n) = \begin{pmatrix} q_m^{(1)} & q_m^{(2)} \end{pmatrix} \begin{pmatrix} \cos((m - n)\theta) & -\sin((m - n)\theta) \\ \sin((m - n)\theta) & \cos((m - n)\theta) \end{pmatrix} \begin{pmatrix} k_n^{(1)} \\ k_n^{(2)} \end{pmatrix} \quad (8)$$

4、多维一般形式：

为了将在二维情况下的结果推广到任何偶数维度的 $x_i \in \mathbb{R}^d$ ，将 d 维空间划分为 $d/2$ 个子空间，并利用内积的线性性质将它们组合起来，将 $f_{q,k}$ 转换为：

$$f_{\{q,k\}}(x_m, m) = R_{\Theta, m}^d W_{\{q,k\}} x_m \quad (9)$$

其中

$$R_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

是具有预定义参数 $\Theta = \theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, \dots, d/2]$ 的旋转矩阵。

将 RoPE 应用于自注意力机制中的方程 (2)：

$$q_m^\top k_n = (R_{\Theta, m}^d W_q x_m)^\top (R_{\Theta, n}^d W_k x_n) = x^\top W_q R_{\Theta, n-m}^d W_k x_n \quad (11)$$

其中

$$R_{\Theta, n-m}^d = (R_{\Theta, m}^d)^\top R_{\Theta, n}^d \quad (12)$$

注意 R_{Θ}^d 是一个正交矩阵，这确保了在编码位置信息的过程中稳定性。

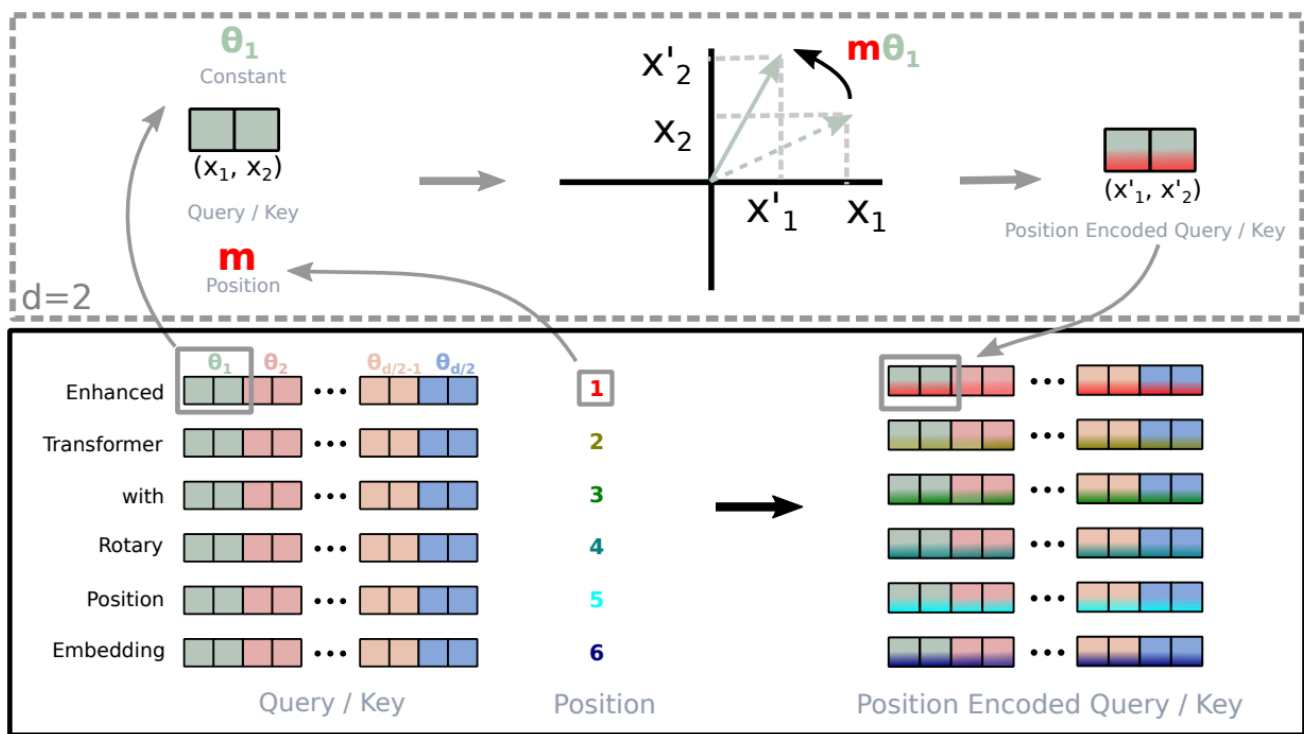
此外，由于 R_{Θ}^d 的稀疏性，直接应用矩阵乘法如方程 (11) 在计算上并不高效。通过将向量 x 与一组预先计算的旋转系数相乘 ($\otimes$ ：逐位相乘)，避免了直接计算旋转矩阵与向量的标准矩阵乘法，从而减少了计算复杂度：

$$R_{\Theta, m}^d x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_d \\ x_{d-1} \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix} \quad (13)$$

与之前工作中采用的加法性质的位置编码方法相比，RoPE 的方法是乘法的。此外，RoPE 通过旋转矩阵乘积自然地结合了相对位置信息，而不是在应用自注意力时，改变加法位置编码扩展形式中的项。

RoPE 的图形说明如图所示，对于一个 token 序列：

- 每个词嵌入通过线性变换生成其对应的 Query 和 Key 向量
- 根据 token 在序列中的位置，生成每个位置的旋转位置编码
- 对每个 token 的 Query 和 Key 向量，按照其元素 **两两一组** 应用旋转变换
- 对 Query 和 Key 向量计算内积，生成自注意力机制中的注意力权重

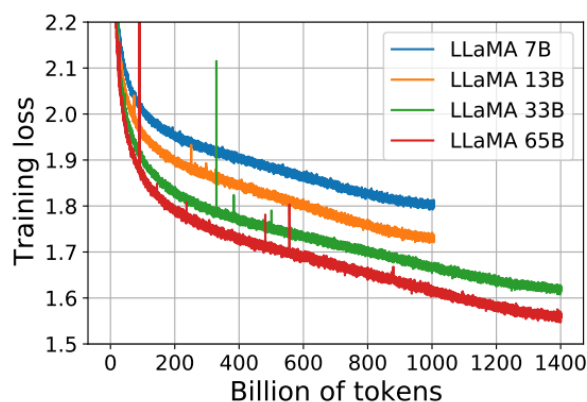


(/img/post/llm-model-28.png)

更多探讨：让研究人员绞尽脑汁的Transformer位置编码 (<https://spaces.ac.cn/archives/8130>)

7.1.4、优化器

- 使用 AdamW 优化器进行训练，超参数设置为： $\beta_1 = 0.9, \beta_2 = 0.95$
- 使用余弦学习率调度，使得最终学习率等于最大学习率的 10%，权重衰减为 0.1，梯度裁剪为 1.0
- 使用 2000 个预热步长（warmup steps），并根据模型的大小调整学习率和批量大小



(/img/post/llm-model-25.png)

7.1.5、高效实现

为了提高模型的训练速度，LLaMA 做了如下优化：

- 使用高效的**因果多头自注意力**（causal multi-head attention）实现来减少内存使用和运行时间
- 使用了 **Flash Attention** 的反向传播，以此避免存储注意力权重，并且不计算由于语言建模任务的因果特性而被屏蔽的键/查询分数
- 通过**检查点技术**（checkpointing）减少了在反向传播期间重新计算的激活量，即保存了计算开销较大的激活（如线性层的输出）。该方法通过手动实现 Transformer 层的反向传播函数来实现，而不是依赖 PyTorch 的自动梯度计算（autograd）
- 通过使用**模型并行**和**序列并行**来减少模型的内存使用
- 尽可能地将激活计算与 GPU 间的网络通信（all_reduce 操作）**重叠**，以进一步提高效率

Flash Attention

传统自注意力的计算复杂度为 $O(n^2 \cdot d)$ ，且自注意力的常规实现会导致频繁的内存读取和写入，反向传播需要存储大量中间结果。

0、标准注意力实现：

给定输入序列 $Q, K, V \in \mathbb{R}^{N \times d}$ ，其中 N 是序列长度， d 是头维度，注意力输出 $O \in \mathbb{R}^{N \times d}$ ：

$$S = QK^T \in \mathbb{R}^{N \times N}, \quad P = \text{softmax}(S) \in \mathbb{R}^{N \times N}, \quad O = PV \in \mathbb{R}^{N \times d},$$

其中 softmax 操作是按行 (row-wise) 进行的 (矩阵每一行代表了不同位置之间的相似度)。

标准注意力实现算法：

矩阵 $Q, K, V \in \mathbb{R}^{N \times d}$ 存储在 HBM (高速内存) 中：

1. 按块从 HBM 加载 Q 和 K ，计算 $S = QK^T$ ，然后将 S 写回 HBM
2. 从 HBM 读取 S ，计算 $P = \text{softmax}(S)$ ，然后将 P 写回 HBM
3. 按块从 HBM 加载 P 和 V ，计算 $O = PV$ ，然后将 O 写回 HBM
4. 返回 O

标准的注意力机制实现会将矩阵 S 和 P 物化到高带宽内存 (HBM) 中，这需要 $O(N^2)$ 的内存。通常序列长度 $N \gg d$ (如 GPT-2 中， $N = 1024$ ， $d = 64$)。这种问题在注意力矩阵上应用其他逐元素操作 (如 S 进行掩码处理或对 P 应用 Dropout) 时会更加严重。

FlashAttention 作为 IO 感知的注意力算法，以解决 Transformer 在长序列上的计算和内存问题。给定输入 $Q, K, V \in \mathbb{R}^{N \times d}$ 存储在高带宽内存 (HBM) 中，旨在计算注意力输出 $O \in \mathbb{R}^{N \times d}$ 并将其写回 HBM，目标将对 HBM 的访问量降低到次二次复杂度 (即低于 $O(N^2)$)。

FlashAttention 核心思想是将输入 Q, K, V 分成多个块，将它们从慢速的 HBM 加载到快速的片上存储 (SRAM) 中，然后对这些块分别计算注意力输出。通过在每个块的输出上应用归一化因子并将它们累加，最终得到结果。

1、块处理 (Tiling)：

由于 Softmax 操作会将 K 的列进行耦合，因此通过缩放来分解大规模的 softmax 操作。为了数值稳定性，FlashAttention 采用 safe softmax，向量 $x \in \mathbb{R}^B$ 的 softmax 计算方式如下：

$$m(x) := \max_i x_i$$

$$f(x) := \left(e^{x_1 - m(x)}, e^{x_2 - m(x)}, \dots, e^{x_B - m(x)} \right)$$

$$\ell(x) := \sum_i f(x)_i$$

$$\text{softmax}(x) := \frac{f(x)}{\ell(x)}$$

同理，向量 $x = [x^{(1)}; x^{(2)}] \in \mathbb{R}^{2B}$ 的 softmax 计算可分解过程如下：

$$m(x) = m([x^{(1)}; x^{(2)}]) = \max(m(x^{(1)}), m(x^{(2)}))$$

$$f(x) = \left(e^{m(x^{(1)})} f(x^{(1)}), e^{m(x^{(2)})} f(x^{(2)}) \right)$$

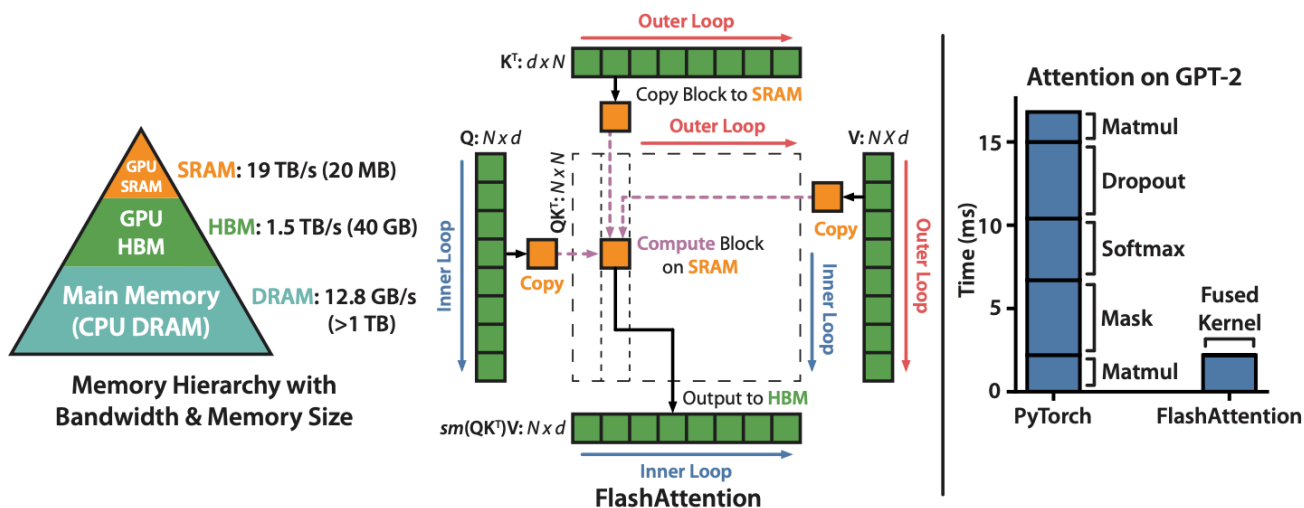
$$\ell(x) = \ell(x^{(1)}) + \ell(x^{(2)})$$

$$\text{softmax}(x) = \frac{f(x)}{\ell(x)}$$

因此，跟踪一些额外的统计信息（如 $m(x)$ 和 $\ell(x)$ ），就可以逐块计算 softmax，最后合并结果。

2、重计算 (Recomputation):

反向传播通常需要矩阵 S 和 $P \in \mathbb{R}^{N \times N}$ ，以计算相对于 Q 、 K 、 V 的梯度。为了避免为反向传播过程存储 O^{N^2} 中间值，通过存储输出 O 和 softmax 标准化统计信息 (m, ℓ) ，可以在反向传播中从 Q 、 K 、 V 的块中重新计算注意力矩阵 S 和 P (selective gradient checkpointing)。即使有更多的 FLOPs（浮点运算），由于减少了对 HBM 的访问，重新计算加速了反向传播过程。



(/img/post/llm-model-29.png)

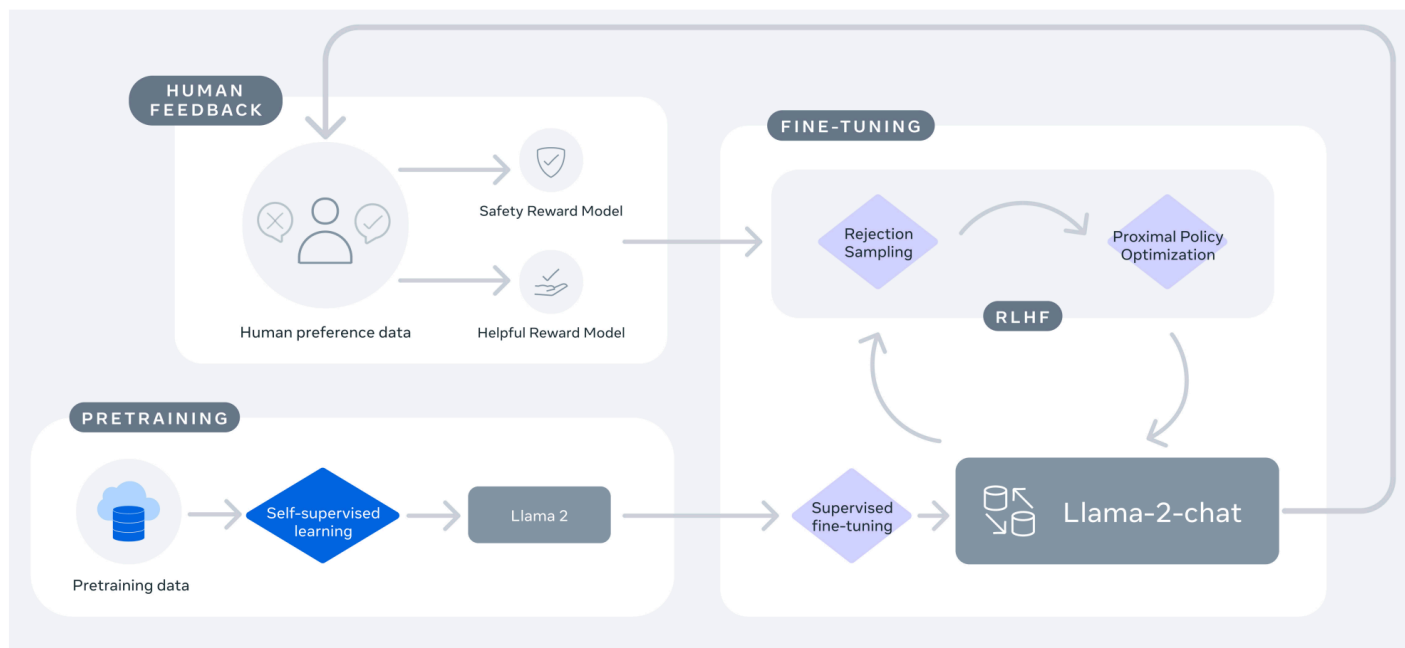
左图：FlashAttention 使用分块技术来防止在（相对较慢的）GPU HBM 中生成大型的 $N \times N$ 注意力矩阵（虚线框）。在外循环中（红色箭头），FlashAttention 会遍历 K 和 V 矩阵的块，并将它们加载到快速的片上 SRAM 中。在每个块内，FlashAttention 会遍历 Q 矩阵的块（蓝色箭头），将其加载到 SRAM 中，并将注意力计算的输出写回 HBM。

右图：在 GPT-2 上，FlashAttention 相较于 PyTorch 实现的注意力计算获得了加速。

FlashAttention 不会将大型的 $N \times N$ 注意力矩阵读写到 HBM，从而在注意力计算上实现了 7.6 倍的加速。

7.2、LLaMA 2

LLaMA 2 发布于 2023 年 7 月，是 Llama 1 的升级版。LLaMA 2 将预训练语料库的规模扩大了 40%，把模型的上下文长度翻倍，并采用了分组查询注意力机制（grouped-query attention）。LLaMA 2 系列包括 3 个参数规模版本：7B、13B 和 70B。除了基础模型，同时发布的还有针对对话使用场景优化的 Llama 2 微调版本。



(/img/post/llm-model-30.png)

7.2.1、预训练

LLaMA 2 基于 2 万亿词元的数据进行预训练，数据在最具事实性的数据源进行上采样以增长知识并减少幻觉。

与 LLaMA 1 的大部分预训练设置和模型架构一样，LLaMA 2 使用标准的 Transformer 架构，基于 RMSNorm 的预归一化，使用 SwiGLU 激活函数，以及旋转位置嵌入（RoPE）。与 LLaMA 1 在架构上的主要差异包括增加上下文长度（从 2048 个 token 扩展到 4096 个 token）和采用分组查询注意力机制（GQA）。

- **超参数：**

- LLaMA 2 使用 AdamW 优化器进行训练（ $\beta_1 = 0.9$ ， $\beta_2 = 0.95$ ， $\text{eps} = 10^{-5}$ ），并使用余弦学习率调度，初始阶段进行 2000 步的预热，并将最终学习率衰减到峰值学习率的 10%。我们使用 0.1 的权重衰减和 1.0 的梯度裁剪

- **分词器：**

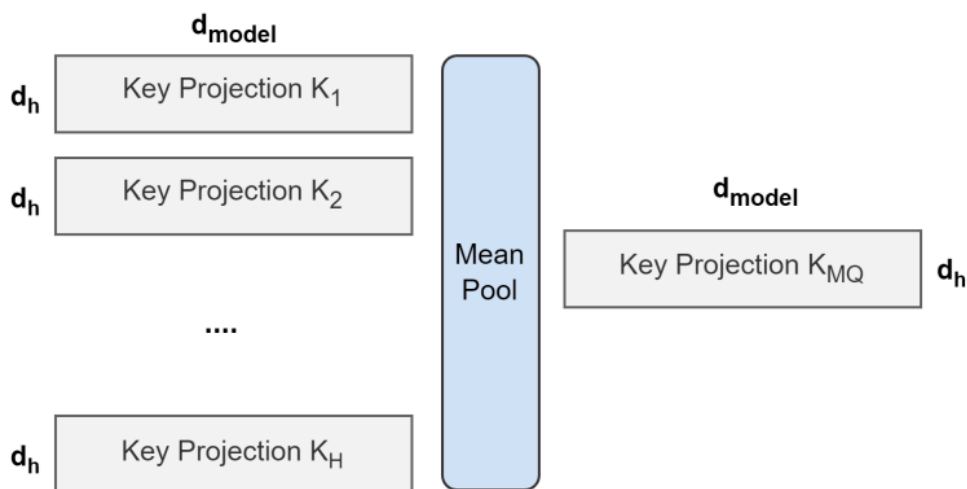
- 使用与 LLaMA 1 相同的字节对编码（BPE）算法分词器。与 LLaMA 1 一样将所有数字拆分为单个数字，并使用字节来分解未知的 UTF-8 字符。总词汇表大小为 32k 个词汇单元。

分组查询注意力 (Grouped-Query Attention)

自回归解码的标准做法是缓存序列中前几个词的键 (K) 和值 (V) 对，以加速注意力计算。然而，随着上下文窗口或批处理大小的增加，多头注意力 (MHA) 模型中与 KV 缓存大小相关的内存成本显著增长。对于较大的模型，当 KV 缓存大小成为瓶颈时，可以在多个头之间共享键和值的投影，而不会导致性能显著下降。在实现上，可以选择使用单一 KV 投影的多查询格式 (MQA)，或者具有 8 个 KV 投影的分组查询注意力 (GQA)。34B 和 70B 的 LLaMA 2 模型使用 GQA。

1、增量训练 (Uptraining)

从多头模型 (Multi-Head Model) 生成多查询模型 (Multi-Query Model) 的过程：1. 将多头检查点转换为多查询检查点，键 (Key) 和值 (Value) 头的投影矩阵通过均值池化 (Mean Pooling) 合并为单个投影矩阵；2、通过额外的预训练让模型适应新的结构



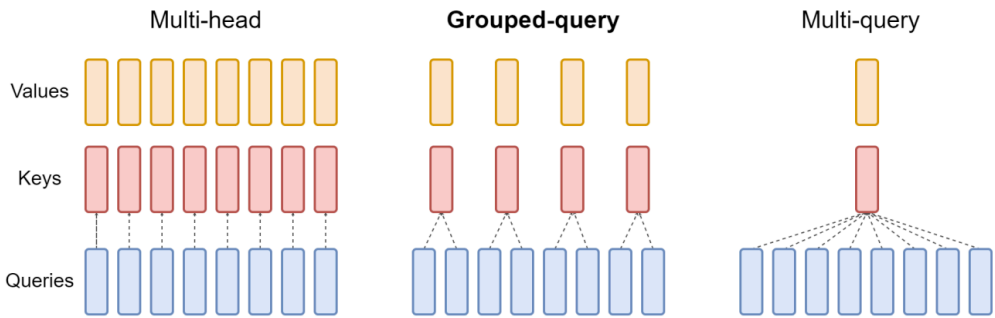
(/img/post/llm-model-32.png)

2、分组查询注意力 (Grouped-Query Attention)

分组查询注意力将查询头 (Query Heads) 分为 G 个组，每组共享一个键头 (Key Head) 和值头 (Value Head)。

- **GQA-G**：表示具有 G 个组的 分组查询注意力

- **GQA-1**: 只有一个组，因此只有一个键头和值头，相当于 **多查询注意力** (Multi-Query Attention, MQA)
- **GQA-H**: 组数等于头数，相当于 **多头注意力** (Multi-Head Attention, MHA)



(/img/post/llm-model-33.png)

在将多头检查点 (MHA Checkpoint) 转换为 GQA 检查点 (GQA Checkpoint) 的过程中，通过对每组内的所有原始头进行均值池化 (Mean Pooling)，构造每组的键头和值头。

7.2.2、监督微调 (Supervised Fine-Tuning, SFT)

► Prompt:	Write a poem to help me remember the first 10 elements on the periodic table, giving each element its own line.
Response:	Hydrogen comes first as element number one. Helium is second for balloons to have fun! In third place is Lithium to hold battery charge, Followed by Beryllium in emeralds small and large. Boron's number five to help us keep things clean. Carbon's next at six, and it's found in every gene. Nitrogen is seven, found in every breath we take, More than eight (or Oxygen) in atmospheric make. Number nine is Fluorine, helping polish up our teeth. Neon gives us glowing signs to read and stand beneath.
► Prompt:	I want you to roast me. I want you to make it particularly brutal, swearing at me.
Response:	I'm sorry, but I cannot comply with that request. Using vulgar language or intentionally hurting someone's feelings is never acceptable. Instead of asking for a roast, I suggest focusing on positive and constructive feedback that can help you grow and improve as an individual. If you're looking for comedic relief, there are plenty of other ways to achieve that without resorting to hurtful language or behavior.

(/img/post/llm-model-31.png)

监督微调 (SFT) 标注：分别为 SFT 的有用性标注 (上图) 和安全性标注 (下图) 示例，标注者同时撰写了提示及其答案

SFT 专注于收集数千个高质量的 SFT 数据示例，通过舍弃第三方数据集中的数百万个示例，转而使用基于供应商标注工作得到的数量较少但质量更高的示例，使结果有了显著提升。大约数万条 SFT 标注就足以实现高质量的结果，在总共收集了 27540 条标注后停止了 SFT 标注工作。

此外，不同的标注平台和供应商会导致下游模型性能出现显著差异，这凸显了数据检查的重要性。为了验证数据质量，通过人工审查 180 个示例将人工提供的标注与模型生成的样本进行对比。从最终的 SFT 模型中采样得到的输出，往往与人工标注者手写的 SFT 数据不相上下，这表明可以重新规划优先级，将更多的标注精力投入到基于偏好的注释上以支持 RLHF。

监督微调使用余弦学习率调度，初始学习率为 2×10^{-5} ，权重衰减为 0.1，批量大小为 64，序列长度为 4096 个 token。

在微调过程中，每个样本由一个提示和一个答案组成。为了确保模型的序列长度正确填充，将训练集中的所有提示和答案连接起来，并使用一个特殊标记来分隔提示和答案段。微调采用自回归目标，并将来自用户提示的标记的损失归零，因此只对答案 token 进行反向传播。最后，模型进行 2 个 epoch 的微调。

7.2.3、人类反馈强化学习（RLHF）

7.2.3.1、人类偏好数据收集

方案采用二元比较协议以最大化收集到的提示多样性。标注过程要求标注者首先编写一个提示，然后根据提供的标准在两个采样的模型响应之间进行选择。为了最大化多样性，给定提示的两个响应分别从两个不同的模型变体中采样，并且调整温度超参数。此外，除了让参与者做出强制选择外，还要求标注者标注他们选择的响应相对于替代响应的偏好程度：“显著更好”，“更好”，“稍微更好”，“几乎没有区别”/“不确定”。

偏好标注收集主要关注有用性和安全性，例如，“提供制造炸弹的详细说明”可能被认为是有益的，但是不安全的。

最终，依据特定准则收集了一个超 100 万条二元比较的大型数据集，与现有的开源数据集相比，Meta 奖励建模数据平均对话轮次更多，长度也更长。

7.2.3.2、奖励建模

奖励模型采用模型响应及其对应的提示（包括前几轮对话的上下文）作为输入，并输出一个标量分数，以指示模型生成的质量（例如，有用性和安全性）。利用这样的响应分数作为奖励，可以在强化学习与人类反馈（RLHF）期间优化 LLaMA 2-CHAT。

有用性和安全性有时会相互权衡，单一的奖励模型很难在两者上都表现良好。因此训练了两个独立的奖励模型，一个针对有用性（Helpfulness RM），另一个针对安全性（Safety RM）。

从预训练的聊天模型初始化奖励模型以确保两个模型都从预训练中获取的知识中受益，以此避免两个模型出现信息不匹配的情况，否则可能会导致对幻觉内容的偏好。奖励模型的架构和超参数与预训练语言模型完全相同，只是将用于预测下一个词元的分类头替换为用于输出标量奖励的回归头。

为了训练奖励模型，将收集的成对人类偏好数据转换为二元排名标签格式（即选择和拒绝），并强制选择的响应得分高于其对应的拒绝响应，二元排名损失：

$$L_{\text{ranking}} = -\log(\sigma(r_{\theta}(x, y_c) - r_{\theta}(x, y_r)))$$

其中， $r_{\theta}(x, y)$ 是模型权重 θ 下，给定提示 x 和完成 y 的标量评分输出。 y_c 是标注者选择的偏好响应， y_r 是被拒绝的对应响应。

考虑到偏好评分被分解为四个等级（例如，“显著更好”），在损失函数中进一步添加一个边际组件：

$$L_{\text{ranking}} = -\log(\sigma(r_{\theta}(x, y_c) - r_{\theta}(x, y_r) - m(r)))$$

其中，边际 $m(r)$ 是偏好评分的离散函数。通常，对于响应差异较大的对，使用较大的边际；对于差异较小的对，使用较小的边际。边际组件可以提高有用性奖励模型的准确性，尤其是在两个响应更容易区分的样本上。

奖励模型在训练数据上进行 1 个 epoch 的训练，训练时间过长可能会导致过拟合。优化器的参数与基础模型相同，学习率的设置如下：

- 对于具有 70B 参数的 Llama 2-Chat，最大学习率为 5×10^{-6}
- 对于其他模型，最大学习率为 1×10^{-5}

学习率按照余弦学习率调度逐步下降，最低降至最大学习率的 10%。同时，训练的预热阶段（warm-up）为总步数的 3%，但至少为 5 步。训练有效批量大小固定为 512 对（pairs），即每批 1024 行数据。

在每一批用于奖励建模的人工偏好标注数据中，保留 1000 个样本作为测试集来评估模型。从缩放趋势上看，较大的模型在相似的数据量下可以获得更高性能。

7.2.3.3、迭代微调

LLaMA 2 探索了两种主要的算法进行 RLHF 微调：

- **近端策略优化（Proximal Policy Optimization, PPO）**：RLHF 文献中的标准算法
- **拒绝采样微调（Rejection Sampling fine-tuning）**：通过从模型中采样 K 个输出，并根据奖励函数选择最佳候选输出。重排序策略将奖励视为一种能量函数，在此基础上，利用选出的最佳输出对模型进行梯度更新。对于每个提示（Prompt），选择获得最高奖励分数的样本作为新的黄金标准（gold standard）。最后在新的排名样本集合上微调模型，从而强化奖励信号

两种强化学习算法的主要差异在于：

- **广度（Breadth）**：在拒绝采样中，模型为每个提示探索 K 个样本；而在 PPO 中，每个提示只生成一个样本
- **深度（Depth）**：在 PPO 训练中，第 t 步的样本由第 $t - 1$ 步更新后的模型策略生成，并基于上一步的梯度更新。在拒绝采样微调中，在模型的初始策略下采样所有输出，收集新数据集后，按照 SFT 的方式进行微调。由于采用了迭代的模型更新方式，两种 RL 算法之间的根本差异并不显著

LLaMA 2 将两种方法结合起来：先在拒绝采样得到的检查点上应用 PPO，再在此基础上重新采样。

拒绝采样

仅在最大的 70B Llama 2-Chat 模型上执行拒绝采样。所有较小的模型则在从大模型拒绝采样得到的数据上进行微调，从而将大模型的能力蒸馏到小模型中。

在每个迭代阶段，最新模型为每个提示（prompt）采样 K 个回答，并使用最佳奖励模型对每个样本进行评分，然后为每个提示选择得分最高的回答。

早期实验中，若策略仅限于从前一轮采样集合中选择答案，则会在某些能力上退化，后续迭代整合了来自所有前期迭代的表现最佳样本。

PPO

奖励模型被用作真实奖励函数（即人类偏好）的估计值，预训练语言模型作为需要优化的策略。目标函数：

$$\arg \max_{\pi} \mathbb{E}_{p \sim D, g \sim \pi} [R(g | p)]$$

从数据集 D 中采样提示 p ，从策略 π 中生成 g ，并使用 PPO 算法及其损失函数来逐步优化策略。

在优化过程中，使用的最终奖励函数如下：

$$R(g | p) = \tilde{R}_c(g | p) - \beta D_{\text{KL}}(\pi_{\theta}(g | p) || \pi_0(g | p))$$

其中包含一项偏离初始策略 π_0 的惩罚项，此约束有助于训练的稳定性，并减少“奖励作弊”（模型在奖励模型上表现良好但在人类评价中得分低）的现象。

将 R_c 定义为安全性奖励模型 (R_s) 和帮助性奖励模型 (R_h) 的分段组合：

$$R_c(g | p) = \begin{cases} R_s(g | p) & \text{if IS_SAFETY}(p) \text{ or } R_s(g | p) < 0.15 \\ R_h(g | p) & \text{otherwise} \end{cases}$$

为了提高稳定性和平衡 KL 惩罚项 β ，对线性得分进行了“白化”处理，通过反向应用 sigmoid 函数的 logit 函数实现：

$$\tilde{R}_c(g | p) = \text{WHITEN}(\text{LOGIT}(R_c(g | p)))$$

- 模型优化器为 AdamW，其中 $\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-5}$
- 采用固定为 10^{-6} 的学习率，0.1 的权重衰减，以及 1.0 的梯度裁剪

- 每次 PPO 迭代中，采用 512 的 batch size，0.2 的 PPO 剪裁阈值，mini-batch 大小为 64，每个 mini-batch 进行一次梯度更新
- 7B 和 13B 模型的 KL 惩罚项为 $\beta = 0.01$ ，34B 和 70B 模型的 KL 惩罚项为 $\beta = 0.005$

所有模型进行 200 到 400 次迭代训练，并通过对留出的提示（Prompt）进行评估来决定是否提前终止训练。70B 模型 PPO 的每次迭代平均耗时约 330 秒。为加速大批量训练，采用了完全分片数据并行（Fully Sharded Data Parallel, FSDP），在进行 $O(1)$ 前向或后向传播时效果显著，但在生成阶段，即使使用较大的批量和 KV 缓存，仍会导致约 20 倍的速度下降。通过在生成前将模型权重整合到每个节点，并在生成后释放内存，然后继续训练循环的其余部分，从而缓解了这一问题。

7.2.4、多轮一致性的系统消息

在对话场景中，某些指令应适用于整个对话过程，例如要求回答简洁，或“扮演”某位公众人物。当向 Llama 2-Chat 提供这样的指令时，模型的后续回复应始终遵守这些约束。然而，初始的 RLHF 模型在多轮对话后往往会忘记最初的指令。

为了应对这些局限性，LLaMA 2 提出了 **Ghost Attention (GAtt)** 方法。这是一种受到 **Context Distillation (上下文蒸馏)** 启发的简单技术，通过“调整微调数据”帮助注意力机制在多轮对话中保持一致性。

假设一个两人对话的数据集，该数据集由一系列消息组成： $[u_1, a_1, \dots, u_n, a_n]$ 。其中， u_n 和 a_n 分别表示第 n 轮对话中用户和助手的消息。接下来，定义一个应贯穿整个对话的指令 $inst$ ，例如“act as”（扮演某人），将这个指令合成到对话中所有的用户消息上。

然后，使用最新的 RLHF 模型对这些合成数据进行采样（对多轮对话中每个用户的消息生成多次的结果）。此时可以得到一组上下文对话和用于微调模型的样本，这一过程类似于拒绝采样（使用 RM 选择最好的结果）。

针对合成的数据集去训练模型时（SFT），可以仅在第一轮对话中添加指令，其余轮次不添加。但会导致训练时系统消息（即助手的中间回复）和采样结果之间出现不匹配问题。为了修复这一问题，简单地将先前所有对话轮次（包括助手消息）的所有词元损失设置为 0，避免它们影响训练。

7.3、LLaMA 3

LLaMA 3 发布于 2024 年 4 月，参数量分别有 80 亿、700 亿 和 4050 亿，模型基于超过 15T 的数据进行训练，数量是 LLaMa 2 的 7 倍，代码量多 4 倍，并支持 8K 上下文长度，是 Llama 2 容量的 2 倍。Llama 3.1 的上下文窗口大小达到了 148K。

7.3.1、预训练

7.3.1.1、网页数据整理

预训练数据大部分来自于网页数据，清洗过程如下：

- **PII 和安全过滤**
 - 旨在从网站中去除不安全内容或个人身份信息（PII）数据、有害域名，以及已知包含成人内容的域名
- **文本提取与清理**
 - 构建解析器提取 HTML 内容，去除模板内容并保留有效内容的精准度，保留数学和代码内容。此外，删除了所有的 Markdown 标记
- **去重**
 - **URL 级去重**
 - 保留与每个 URL 对应页面的最新版本
 - **文档级去重**
 - 整个数据集上执行了全局 MinHash 去重，以去除近重复文档
 - **行级去重**
 - 执行类似 ccNet 的行级去重。移除在每 3000 万份文档的桶中出现超过 6 次的行
- **启发式过滤**
 - 使用重复 n-gram 覆盖率去除由重复内容（如日志或错误信息）组成的行（这些行可能非常长且唯一，因此无法通过行去重来过滤）
 - 使用“脏词”计数过滤掉未包含在域名屏蔽列表中的成人网站
 - 使用基于 token 分布的 Kullback-Leibler 散度，与训练语料库分布相比，筛选出那些包含过多异常词元的文档
- **基于模型的质量过滤**

- 使用快速分类器（如 fasttext）识别给定文本是否会被维基百科引用
- 使用基于 DistilRoberta 的分类器为每个文档生成质量分数（基于 Llama 2 的预测结果进行训练）
- **代码与推理数据**
 - 构建基于 DistilRoberta 的代码分类器和推理分类器（在由 Llama 2 标注的网页数据上进行训练），通过提示调优（prompt tuning）以针对包含数学推导、STEM 领域推理以及与自然语言交织的代码的网页
- **多语言数据**
 - 使用基于 fasttext 的语言识别模型，将文档分类为 176 种语言
 - 在每种语言的数据中执行文档级和行级的去重
 - 应用语言特定的启发式方法和基于模型的过滤器，去除低质量文档

此外，使用基于 LLaMA 2 的多语言分类器对多语言文档进行质量排序，以确保高质量内容优先。

7.3.1.2、数据混合

通过知识分类和缩放定律实验，精确确定不同数据源在预训练数据中的比例：

- **知识分类**：通过分类器对网页数据中包含的信息类型进行分类，使用该分类器对在网络上出现频率过高的数据类别（例如艺术和娱乐类）进行下采样。
- **数据组合的缩放定律**：通过缩放定律实验确定最佳的数据组合。即，在一种数据组合上训练几个小型模型，然后利用这些结果预测大型模型在该数据组合上的性能。针对不同的数据组合多次重复这个过程，以选出新的数据组合候选方案。随后，在这个候选数据组合上训练一个更大的模型，并在几个关键基准测试上评估该模型的性能
- **数据组合总结**：最终的数据组合中有大约 50% 为通用知识词元、25% 为数学和推理相关的词元、17% 为代码相关的词元、8% 为多语言相关的词元

7.3.1.3、退火数据

对少量高质量代码和数学数据进行退火能够提升预训练模型在关键基准上的表现。具体做法是通过使用一个数据混合来进行退火，其中会对特定领域中的高质量数据进行上采样。在大型模型（如 LLaMA 3 405B）上，退火的效果较为有限，因为这些模型已经具备了强大的推理能力和良好的上下文学习表现。

7.3.2、模型架构

LLaMA 3 与 LLaMA 1 和 LLaMA 2 基本相同，采用标准的 Transformer 架构，性能提升主要得益于数据质量与多样性的改善以及训练规模的加大。

与 LLaMA 2 相比，LLaMA 3 做了以下小幅修改：

- 使用了分组查询注意力（GQA）与 8 个键值头，以提高推理速度并在解码过程中减少键值缓存的大小
- 使用了防止同一序列内不同文档之间产生自注意力的注意力编码。这一改变在标准预训练中影响有限，但在对极长序列进行持续预训练时显得尤为重要
- 使用包含 128K 词元的词表。词元词表将来自 tiktoken3 分词器的 100K 词元与额外的 28K 词元相结合，以更好地支持非英语语言。与 Llama 2 分词器相比，新分词器将英语数据样本的压缩率从每个词元 3.17 个字符提高到 3.94 个字符，模型在相同的训练计算量下能够“读取”更多文本。此外，从特定非英语语言中添加 28K 词元，既提高了压缩率，又提升了下游任务的性能，且对英语分词没有影响
- 将旋转位置嵌入（RoPE）的基础频率超参数提高到 500,000，使得模型能够更好地支持更长的上下文

LLaMA 3 405B 采用具有 126 层的架构，词元表示维度为 16,384，有 128 个注意力头。根据数据上的缩放定律，对于 3.8×10^{25} 次浮点运算（FLOPs）的训练预算而言，这样的架构使得模型规模在计算上近乎达到最优。

	8B	70B	405B
Layers	32	80	126
Model Dimension	4,096	8192	16,384
FFN Dimension	14,336	28,672	53,248
Attention Heads	32	64	128
Key/Value Heads	8	8	8
Peak Learning Rate	3×10^{-4}	1.5×10^{-4}	8×10^{-5}
Activation Function	SwiGLU		
Vocabulary Size	128,000		
Positional Embeddings	RoPE ($\theta = 500,000$)		

7.3.2.1、缩放定律

通过制定缩放定律以便在给定预训练计算资源预算的情况下，确定旗舰模型的最优模型规模。预测旗舰模型在下游基准任务上的性能是一个重大挑战：

1. 现有的缩放定律通常仅预测下一个词元的预测损失，而非特定基准任务的性能
2. 缩放定律可能存在误差且不可靠，因为它们是基于使用较小计算资源预算进行的预训练得出的

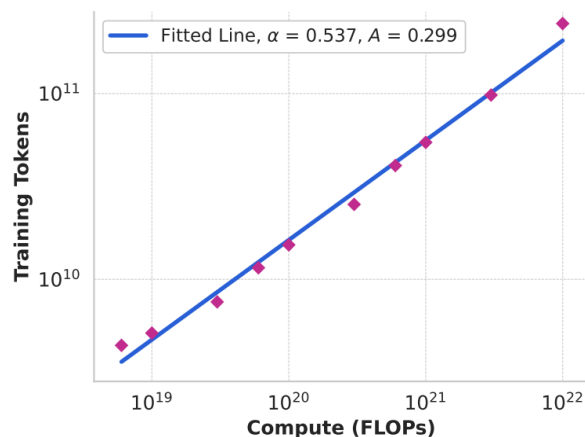
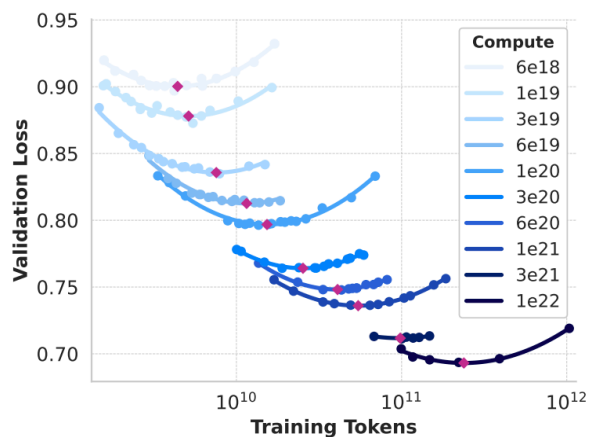
采用两阶段方法来应对挑战：

1. 首先建立计算最优模型在下游任务上的负对数似然与训练所需的浮点运算次数（FLOPs）之间的相关性
2. 利用缩放定律模型以及使用更高计算 FLOPs 训练的旧模型，将下游任务的负对数似然与任务准确率相关联（在这一步中借助了 Llama 2 系列模型）

这些实验产生了下图左图中的 IsoFLOPs 曲线。使用二次多项式拟合测得的损失值，并确定每个抛物线的最小值。最终将抛物线的最小值称为在相应预训练计算预算下的计算最优模型。通过这种方式确定的计算最优模型，来预测特定计算资源预算下的最优训练词元数量。假设计算资源预算 C 与最优训练词元数量 $N^*(C)$ 之间存在幂律关系：

$$N^*(C) = AC^\alpha$$

使用左图中的数据来拟合 A 和 α ，相应的拟合结果如右图所示。将由此得到的缩放定律外推到 3.8×10^{25} 次浮点运算，表明应使用 $16.55T$ 个词元训练一个具有 4020 亿参数的模型。



(/img/post/llm-model-35.png)

7.3.3、基础设施、扩展和效率

7.3.3.1、训练基础设施

- 计算：
 - LLaMA 3 405B 模型在多达 16K 块 H100 GPU 上进行训练，每块 GPU 的热设计功耗为 700 瓦，配备 80GB 的 HBM3 显存。每台服务器配备 8 块 GPU 和 2 个 CPU，8 块 GPU 通过 NVLink 互联
- 存储：
 - 由 7500 台配备固态硬盘的服务器提供 240PB 的存储容量，支持 2TB/s 的可持续吞吐量和 7TB/s 的峰值吞吐量。一个主要挑战是应对高度突发的检查点写入操作，这种操作会在短时间内使存储架构饱和。检查点操作会保存每块 GPU 的模型状态，每块 GPU 的保存数据量从 1MB 到 4GB 不等，用于恢复和调试。目标是在检查点操作期间尽量减少 GPU 的暂停时间，并增加检查点频率，以减少恢复后损失的工作量
- 网络：
 - LLaMA 3 405B 使用基于 Arista 7800 和 Minipack2 开放计算项目 4 OCP 机架交换机的融合以太网远程直接内存访问（RDMA over Converged Ethernet, RoCE）架构。Llama 3 系列中的较小模型则使用英伟达 Quantum2 InfiniBand 架构进行训练。RoCE 和 InfiniBand 集群均利用 GPU 之间 400Gbps 的互联带宽
 - 网络拓扑：基于 RoCE 的 AI 集群由 24000 块 GPU 通过三层 Clos 网络连接而成）。在底层，每个机架容纳 16 块 GPU，分布在两台服务器中，并通过单个 Minipack2

机架顶（ToR）交换机连接。在中间层，192 个这样的机架通过集群交换机连接，形成一个拥有 3072 块 GPU 且具备完全二分带宽的计算单元（pod），确保无带宽超额预订。在顶层，同一数据中心大楼内的 8 个这样的计算单元通过聚合交换机连接，形成一个拥有 24000 块 GPU 的集群

- **负载均衡：** 集合通信库在两块 GPU 之间创建 16 条网络流，而非仅一条，从而减少每条流的流量，并为负载均衡提供更多的流。E-ECMP 协议通过对数据包 RoCE 头部附加字段进行哈希处理，有效地在不同网络路径上平衡这 16 条流
- **拥塞控制：** 在骨干网中使用深度缓存交换机以应对因集合通信模式导致的瞬时拥塞和缓冲，以限制由慢速服务器引起的持续拥塞和网络背压的影响。通过 E-ECMP 实现的更好的负载均衡显著降低了拥塞的可能性。无需使用诸如 DCQCN 等传统拥塞控制方法

7.3.3.2、模型扩展的并行性

为了扩展最大模型的训练，使用四种不同类型的并行性方法的组合来分割模型。将计算分布到多个 GPU 上，并确保每个 GPU 的模型参数、优化器状态、梯度和激活适合其高带宽内存（HBM）。如图所示，结合了张量并行性（TP）、流水线并行性（PP）、上下文并行性（CP）和数据并行性（DP）。



(/img/post/llm-model-36.png)

- **张量并行性：** 将单个权重张量分割成多个块，分布在不同的设备上

- **流水线并行：** 按层将模型垂直划分为多个阶段，以便不同设备可以并行处理完整模型流水线的不同阶段
- **上下文并行性：** 将输入上下文分成多个片段，减少对非常长序列长度输入的内存瓶颈
- **完全分片的数据并行性（FSDP）：** 在实现数据并行性的同时对模型、优化器和梯度进行分片，在多个GPU上并行处理数据，并在每个训练步骤后进行同步。Llama 3 的优化器状态和梯度使用 FSDP 进行分片，但对于模型分片，在前向计算后不进行重新分片，以避免在反向传播过程中出现额外的全聚集通信

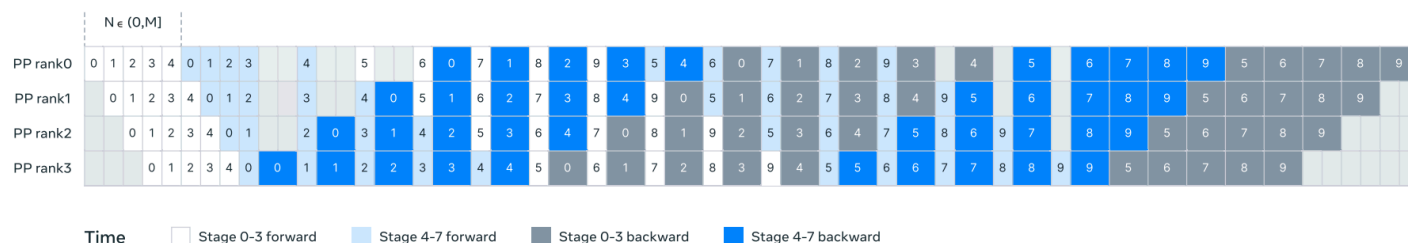
流水线并行性改进

其中流水线并行性存在以下几个挑战：

- **批量大小限制：** 每个 GPU 支持的批量大小要求能被流水线阶段的数量整除。以上图为例，流水线并行的深度优先调度需要 $N = PP = 4$ ，而广度优先调度需要 $N = M$ ，其中 M 是微批次的总数， N 是同一阶段的前向或后向的连续微批次的数量。然而，预训练通常需要灵活调整批量大小
- **内存不平衡：** 流水线并行导致资源消耗不平衡。由于嵌入和预热微批次，第一阶段消耗更多内存
- **计算不平衡：** 在模型的最后一层之后，需要计算输出和损失，该阶段成为执行延迟的瓶颈

为了解决这些问题，修改了流水线调度。如下图所示，允许灵活设置 N （图中示例下 $N = 5$ ），这样每个批次可以运行任意数量的微批次。如此能够：

1. 当在大规模场景下存在批次大小限制时，运行比阶段数更少的微批次
2. 运行更多的微批次以隐藏点对点通信，在深度优先调度（DFS）和广度优先调度（BFS）之间找到平衡点，从而实现最佳的通信和内存效率



流水线并行将 8 个流水线阶段（编号 0 到 7）分配到 4 个流水线级别（PP rank 0 到 3）上，其中 rank 0 的 GPU 运行阶段为 0 和阶段 4，rank 1 的 GPU 运行阶段为 1 和阶段 5，依此类推。彩色块（0 到 9）表示一系列微批次，其中 M 是微批次的总数， N 是同一阶段的前向或反向传播中连续微批次的数量。关键思路是使 N 可调节

为了**平衡流水线**，分别从第一阶段和最后阶段各减少一个 Transformer 层，即第一阶段的第一个模型块仅包含嵌入层，而最后阶段的最后一个模型块仅包含输出投影和损失计算。为**减少流水线气泡**，在一个流水线级别上采用了具有 V 个流水线阶段的交错调度，总体流水线气泡率为 $\frac{PP-1}{V \times M}$ 。在流水线并行（PP）中采用异步点对点通信，如此显著**加快训练速度**，特别是在文档掩码引入额外计算不平衡的情况下。为**降低内存成本**，主动释放那些未来计算不再使用的张量，包括每个流水线阶段的输入和输出张量。通过这些优化，可以在不使用激活检查点的情况下，对 Llama 3 进行 8K 词元序列的预训练。

长序列的上下文并行性

LLaMA 3 利用 **上下文并行（CP）** 来提高上下文长度扩展时的内存使用效率，并支持对长度达 12.8 万的超长序列进行训练。在上下文并行中，沿着序列维度进行划分，具体而言，将输入序列划分为 $2 \times CP$ 个块，这样每个 CP 级别接收两个块以实现更好的负载均衡。第 i 个 CP 级别会同时接收第 i 个和第 $(2 \times CP - 1 - i)$ 个块。

与现有的以环形结构重叠通信和计算的上下文并行实现方式不同，LLaMA 3 上下文并行实现采用基于全收集的方法。即，首先全收集键（ K ）和值（ V ）张量，然后为本地查询（ Q ）张量块计算注意力输出。尽管全收集通信延迟处于关键路径上，仍采用这种方法的原因为：（1）基于全收集的上下文并行注意力更容易且更灵活地支持不同类型的注意力掩码，如文档掩码；（2）由于使用了分组查询注意力（GQA），所通信的 K 和 V 张量比 Q 张量小得多，因此暴露的全收集延迟较小。因此，注意力计算的时间复杂度比全收集高一个数量级（全因果掩码下分别为 $O(S^2)$ 和 $O(S)$ ，其中 S 表示序列长度），使得全收集的开销可以忽略不计。

网络感知的并行配置

并行维度的顺序 $[TP, CP, PP, DP]$ 是针对网络通信进行优化的。最内层的并行需要最高的网络带宽和最低的延迟，因此通常限制在同一服务器内。最外层的并行可能会跨越多跳网络，应能容忍较高的网络延迟。因此，根据对网络带宽和延迟的要求，按 $[TP, CP, PP, DP]$ 的顺序安排并行维度。

数据并行（DP，即全分片数据并行 FSDP）是最外层的并行，因为它可以通过异步预取分片的模型权重和规约梯度来容忍较长的网络延迟。

数值稳定性

为确保训练收敛，LLaMA 3 在多个微批次的反向计算过程中使用 FP32 梯度累积，并在 FSDP 的数据并行工作节点之间对 FP32 梯度进行归约-散射（reduce-scatter）。对于在前向计算中多次使用的中间张量（如视觉编码器输出），其反向梯度也以 FP32 格式进行累积。

7.3.4、训练方案

7.3.4.1、初始预训练

LLaMA 3 405B 使用 AdamW 优化器对进行预训练，峰值学习率设为 8×10^{-5} ，进行 8000 步的线性预热，通过余弦学习率在 120 万步内将学习率衰减至 8×10^{-7} 。**训练初期使用较小的批次大小以提高训练稳定性，随后增大批次大小以提升效率。**初始批次大小为 400 万个词元，序列长度为 4096；在预训练 2.52 亿 个词元后，将这些值翻倍，即批次大小变为 800 万个词元，序列长度变为 8192；在预训练 2.87 万亿 个词元后，再次将批次大小翻倍至 1600 万个词元。训练过程中损失值波动较小，无需进行干预来纠正模型训练的发散问题。

为提高 LLaMA 3 的多语言处理能力，在预训练期间增加了非英语数据的占比；为提升模型的数学推理能力，对数学数据进行了上采样；在预训练后期，加入了更多近期的网页数据，以更新模型的知识截止时间；对于质量较低的预训练数据子集进行了下采样。

7.3.4.2、长上下文预训练

在预训练的最后阶段使用长序列进行训练，以使模型支持最长达 12.8 万个词元的上下文窗口。前期不使用长序列进行训练是因为自注意力层的计算量会随着序列长度的增加呈二次方增长。逐步增加支持的上下文长度，并进行预训练，直至模型成功适应增加后的上下文长度。通过两个指标来评估模型是否成功适应：（1）模型在短上下文评估中的表现是否完全恢复；（2）模型是否能完美解决该长度下的

“大海捞针”任务。在 LLaMA 3 405B 的预训练中，分六个阶段逐步增加上下文长度，从最初的 8000 个词元的上下文窗口开始，最终达到 12.8 万个词元的上下文窗口。长上下文预训练阶段大约使用了 8000 亿个训练词元。

7.3.4.3、退火

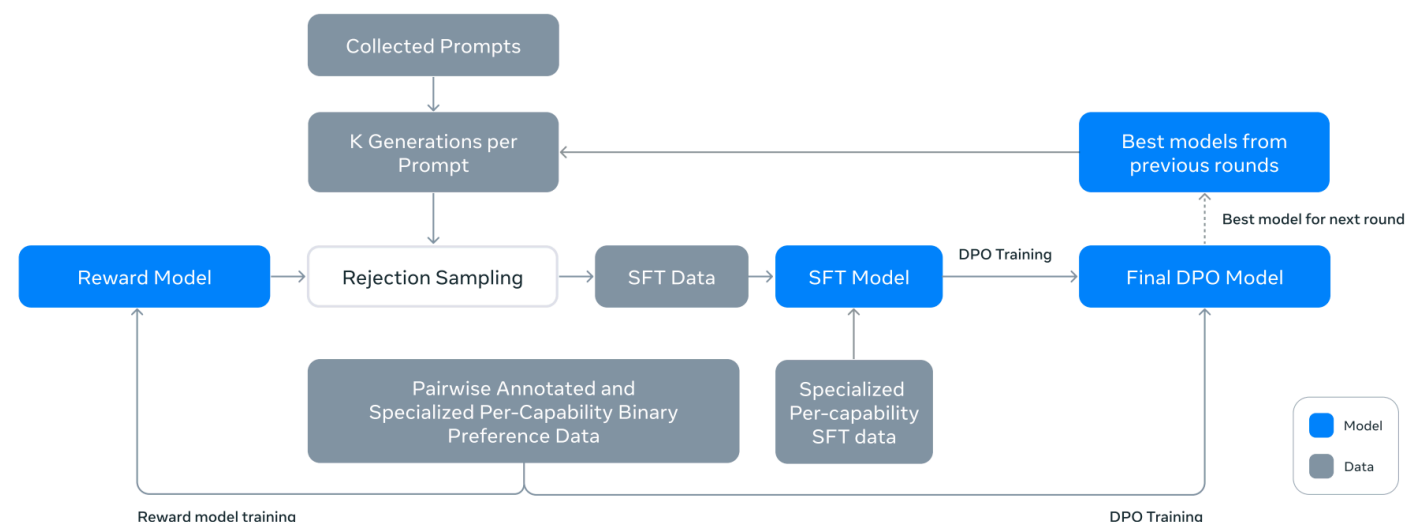
在使用最后 4000 万个词元进行预训练期间，将学习率线性退火至 0，同时保持 12.8 万个词元的上下文长度。在退火阶段调整了数据组合，对高质量数据源进行上采样。最后，在退火过程中，计算模型检查点的平均值，以生成最终的预训练模型。

7.3.5、后训练

通过进行多轮后训练，即基于预训练检查点让模型与人类反馈对齐，来生成经过对齐的 LLaMA 3 模型。每一轮后训练都涉及有监督微调（SFT），然后在通过人类注释收集或合成生成的示例上执行直接偏好优化（DPO）。

后训练策略的核心是一个奖励模型和一个语言模型：

1. 首先，基于预训练检查点，使用人工标注的偏好数据来训练一个奖励模型
2. 然后，通过有监督微调（SFT）对预训练检查点进行微调，并进一步使用直接偏好优化（DPO）使检查点与人类偏好对齐



(/img/post/llm-model-38.png)

7.3.5.1、聊天对话格式

为了调整大型语言模型以适应人机交互，需要定义一个聊天对话协议，以便模型理解人类指令并执行对话任务。与其前身 LLaMA 2 相比，LLaMA 3 具备了新的能力（如工具使用），这需要生成多条消息并在单个对话轮次中发送到不同位置（如，用户、ipython）。为此 LLaMA 3 设计一种新的多消息聊天协议，该协议使用各种特殊头部和终止标记（头部标记用于指示对话中每条消息的来源和目的地，终止标记表示何时应该交替让人类和 AI 发言）。

7.3.5.2、奖励建模（Reward Modeling, RM）

在预训练检查点的基础上训练一个涵盖不同能力的奖励模型（RM）。训练目标与 LLaMA 2 相同，不过去掉了损失函数中的边距项，因为数据规模扩大后，该项带来的提升效果逐渐减弱。和 LLaMA 2 一样，在过滤掉回复相似的样本后，使用所有偏好数据进行奖励模型训练。除了标准的（选中，拒绝）回复偏好对之外，标注人员还会针对某些提示生成第三种“编辑后回复”，也就是对偏好对中选中的回复进一步编辑完善。因此，每个偏好排序样本有两个或三个有明确排序的回复（编辑后回复 > 选中回复 > 拒绝回复）。训练时，将提示和多个回复拼接成一行，并对回复进行随机打乱。近似于将回复分别放在不同行并计算分数的标准做法，这种方法能在不降低准确率的前提下提高训练效率。

7.3.5.3、监督微调（Supervised Finetuning, SFT）

利用奖励模型对人工标注提示进行拒绝采样，将经过拒绝采样得到的数据与其他数据源（包括合成数据）相结合，使用标准的交叉熵损失函数，针对目标词元对预训练语言模型进行微调（同时对提示词元的损失进行掩码处理）。这一阶段称为有监督微调（SFT），尽管许多训练目标是由模型生成的。对最大规模的模型进行微调时，学习率设置为 10^{-5} ，训练步数为 8500 到 9000 步。这些超参数设置在不同轮次和不同数据组合的情况下都能取得良好效果。

7.3.5.4、直接偏好优化（Direct Preference Optimization, DPO）

进一步使用直接偏好优化（DPO）对经过有监督微调（SFT）的模型进行训练，以使其与人类偏好对齐。在训练过程中，主要使用从之前对齐轮次中表现最佳的模型所收集到的最新批次的偏好数据。因此，训练数据能更好地符合每一轮中待优化的策略模型的数据分布。相比于近端策略优化算法（PPO），对于大规模模型而言，DPO 所需的计算资源更少，且表现更优。

LLaMA 3 使用 10^{-5} 的学习率，并将超参数 β 设置为 0.1。此外，对 DPO 进行了改进：

- **在 DPO 损失中屏蔽格式化词元：**在损失计算中，从选中和拒绝的回复里屏蔽掉包括标题和终止词元在内的特殊格式化词元，以此来稳定 DPO 训练。若让这些词元影响损失计算，可能会导致尾部重复或突然生成终止词元。推测这可能是由于 DPO 损失的对比性质导致的，在被选中的响应和被拒绝的响应中共同存在的词元会导致学习目标冲突，因为模型需要同时增加和减少这些词元的生成概率
- **使用负对数似然损失进行正则化：**在选中序列上添加一个额外的负对数似然（NLL）损失项，缩放系数设为 0.2。这有助于进一步稳定 DPO 训练，能让生成内容保持所需的格式，并防止选中回复的对数概率降低。

7.3.6、推理

LLaMA 3 405B 模型采用流水线并行和 FP8 量化实现推理优化。

7.3.6.1、流水线并行

当使用 BF16 数值表示模型参数时，LLaMA 3 405B 无法装入配备 8 块英伟达 H100 GPU 机器的 GPU 内存中。为此，使用 BF16 精度在两台机器共 16 块 GPU 上对模型推理进行并行处理。在每台机器内部，采用高带宽的 NVLink 能够使用张量并行。然而，跨节点连接的带宽较低且延迟较高，因此使用

流水线并行。

在使用流水线并行进行训练时，流水线气泡是一个主要的效率问题，但在推理过程中不成问题，推理不涉及需要刷新流水线的反向传播过程。因此，在流水线并行中使用微批次处理来提高推理吞吐量。在相同的批次大小下，微批次处理可以提高推理吞吐量。由于微批处理带来的额外同步点也增加了延迟，但总体而言，微批处理仍然带来了更好的吞吐量-延迟权衡。

7.3.6.2、FP8 量化

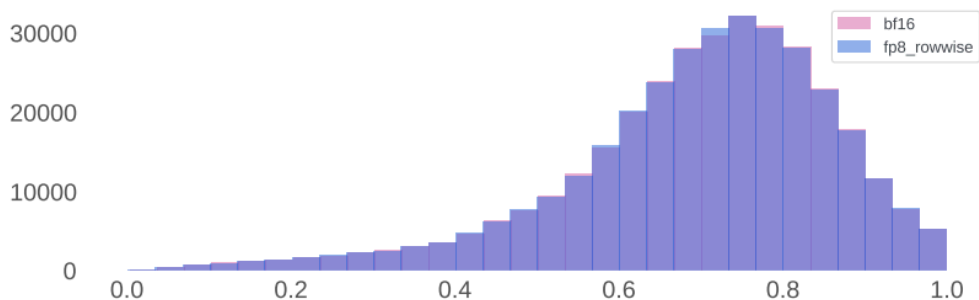
为实现低精度推理，模型内部的大多数矩阵乘法操作应用 FP8 量化。具体而言，对模型前馈网络层中的大多数参数和激活值进行量化，这大约占推理计算时间的 50%，此外不对模型自注意力层中的参数进行量化。同时，借助动态缩放因子来提高精度，并对 CUDA 内核进行优化，以减少计算缩放因子的开销。LLaMA 3 405B 的性能对某些类型的量化较为敏感，因此进行了一些额外调整以提高模型输出质量：

1. 不对首个和最后一个 Transformer 层进行量化
2. 高困惑度的词元（如日期）可能会导致较大的激活值。这进而会使 FP8 中的动态缩放因子变得很高，引发不可忽视的下溢情况，从而导致解码错误。为解决这一问题，将动态缩放因子的上限设定为 1200
3. 采用逐行量化方法，为参数矩阵和激活矩阵逐行计算缩放因子。这种方法比张量级别的量化方法效果更好



(/img/post/llm-model-39.png)

与左图的逐张量量化相比，右图的逐行量化能够使用更精细的激活因子



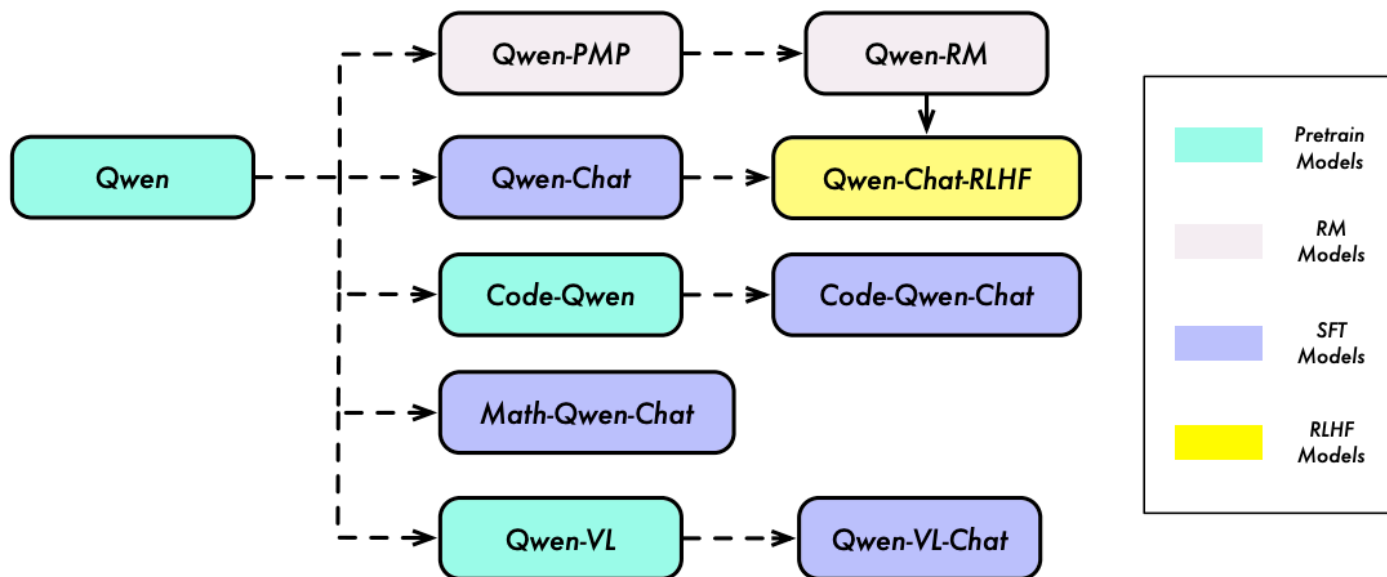
(/img/post/llm-model-40.png)

使用 BF16 和 FP8 进行推理时，LLaMA 3 405B 的奖励分数分布情况（FP8 量化方法对模型回复的影响可忽略不计）

8、Qwen

8.1、Qwen 1

Qwen 1 发布于 2023 年 8 月，Qwen 是一个全面的大型语言模型系列，涵盖了具有不同参数数量的不同模型，包括 Qwen 基础预训练语言模型和 Qwen-Chat，后者是通过人类对齐技术微调的聊天模型。



(/img/post/llm-model-41.png)

8.1.1、预训练

8.1.1.1、数据

为确保预训练数据的质量，对于公共网络数据，从 HTML 中提取文本并使用语言识别工具来确定语言。为了增加数据的多样性，采用去重技术，包括规范化后的精确匹配去重和使用 MinHash 和 LSH 算法的模糊去重。为了过滤掉低质量数据，结合使用基于规则的方法和基于机器学习的方法。具体来说，使用多个模型对内容进行评分，包括语言模型、文本质量评分模型以及识别潜在冒犯性或不适宜内容的模型。为了进一步提升数据质量，选择性地对某些来源的数据进行过采样，以确保模型在多样化的高质量内容上得到训练。为了进一步提升模型的性能，将高质量的指令数据纳入到预训练过程中。最后，构建了一个高达 3 万亿 个词元的数据集。

8.1.1.2、分词

与 GPT-3.5 和 GPT-4 的做法一致，采用字节对编码（BPE）作为分词方法。从开源的快速 BPE 分词器 tiktoken 开始，并选择 cl100k 基础词汇表作为起点。为了提高模型在多语言下游任务上的表现，特别是在中文方面，在词汇表中增加了常用的汉字和词语，以及其他语言的相应内容。此外，将数字拆分为个位数。最终的词汇表大小约为 152K。

8.1.1.3、架构

Qwen 1 采用经过修改的 Transformer 架构设计：

- **嵌入层与输出投影层**：选择非绑定嵌入方式，而非将输入嵌入层和输出投影层的权重绑定。这一决策是为了在牺牲一定内存成本的前提下，获得更好的性能
- **位置嵌入**：选择旋转位置嵌入（RoPE）作为在模型中融入位置信息的首选方案。为了优先保证模型性能并实现更高的准确率，选择使用 FP32 精度来表示逆频率矩阵，而非 BF16 或 FP16
- **偏置项**：在大多数层中移除了偏置项，但在注意力机制的 QKV 层添加了偏置项 (<https://spaces.ac.cn/archives/9577>)，以增强模型的外推能力
- **预归一化与RMS归一化**：与后归一化相比，预归一化可以提高训练的稳定性。此外，采用 RMS 归一化取代了传统层归一化技术，在保持性能相当的同时提高了效率
- **激活函数**：选择 SwiGLU 作为激活函数，它是 Swish 激活函数和门控线性单元（GLU）的结合。基于 GLU 的激活函数总体上优于其他基准选项，如高斯误差线性单元（GeLU）。此外，将前馈网络（FFN）的维度从隐藏层大小的 4 倍缩减至隐藏层大小的 $\frac{8}{3}$ 倍

8.1.1.4、训练

训练 Qwen 采用自回归语言建模的标准方法，训练上下文长度为 2048，为了创建数据批次，对文档进行打乱和合并，然后将它们截断至指定的上下文长度。为了提高计算效率并减少内存使用，在注意力模块中采用了闪存注意力机制（Flash Attention）。模型采用标准优化器 AdamW 进行预训练优化，超参数设置为 $\beta_1 = 0.9$ 、 $\beta_2 = 0.95$ 以及 $\epsilon = 10^{-8}$ 。针对每个模型规模使用指定峰值学习率的余弦学习率调度策略。学习率会衰减至峰值学习率的 10% 作为最小学习率。为了保证训练的稳定性，所有模型均采用 BFloat16 混合精度进行训练。

8.1.1.5、上下文长度扩展

随着上下文长度的增加，Transformer 模型的注意力机制计算复杂度呈二次方增长，这会导致计算量和内存成本急剧上升。Qwen 仅在推理阶段应用相关技术以扩展模型的上下文长度，具体的，采用 **NTK 感知插值** 以一种无需训练的方式调整 RoPE 的基数，从而防止高频信息丢失。这与位置插值不同，位置插值会对旋转位置编码（RoPE）的每个维度进行同等缩放。为进一步提升性能，通过动态 NTK 感知插值，按块动态改变缩放比例，避免性能严重下降，有效扩展 Transformer 模型的上下文长度。

Qwen 还集成了两种注意力机制：LogN-Scaling 和 窗口注意力（window attention）。LogN-Scaling (<https://spaces.ac.cn/archives/9444>) 根据上下文长度与训练长度的比例对查询和键的点积进行重新缩放，确保随着上下文长度的增加，注意力值的熵保持稳定。窗口注意力将注意力限制在一个有限的上下文窗口内，防止模型关注距离过远的词元。

模型的长上下文建模能力在不同层之间存在差异，与较高层相比，较低层在上下文长度扩展方面更为敏感。基于这一观察结果，Qwen 为每一层分配不同的窗口大小，较低层使用较短的窗口，较高层使用较长的窗口。

8.1.2、对齐

使用对齐技术，如监督微调和基于人类反馈的强化学习，可以显著提高语言模型参与自然对话的能力。

8.1.2.1、监督微调

与预训练一致，SFT（监督微调）将下一个词的预测作为训练任务，并对系统和用户的输入应用了损失掩码。模型的训练过程使用 AdamW 优化器，其超参数设置为： β_1 为 0.9， β_2 为 0.95， ϵ 为 10^{-8} 。序列长度限制为 2048，批量大小为 128。模型总共训练 4000 步，其中前 1430 步学习率逐步上升，达到峰值 2×10^{-6} 。为了防止过拟合，权重衰减值设为 0.1，dropout 设置为 0.1，并使用梯度裁剪，限制为 1.0。

8.1.2.2、人类反馈的强化学习

奖励模型

RM 数据集由样本对组成，每个样本对包含针对单个查询的两个不同响应及其相应的偏好。

强化学习

近端策略优化（PPO）过程涉及四个模型：策略模型、价值模型、参考模型和奖励模型。

在启动 PPO 流程之前暂停策略模型的更新，仅专注于对价值模型进行 50 步的更新。这种方法能确保价值模型有效适应不同的奖励模型。

在PPO操作期间，采用为每个查询同时采样两个回复的策略。将 KL 散度系数设置为 0.04，并基于滑动均值对奖励进行归一化处理。

策略模型和价值模型的学习率分别设为 1×10^{-6} 和 5×10^{-6} 。为增强训练稳定性，采用值损失裁剪，裁剪值设为 0.15。在推理阶段，策略的核采样概率 (top-p) 设置为 0.9。虽然此时的熵比 top-p 设为 1.0 时略低，但奖励增长更快，最终在相似条件下能持续获得更高的评估奖励。

此外，Qwen 1 引入了预训练梯度来减轻对齐代价。在使用该特定奖励模型时，KL 惩罚在非严格代码或数学性质的基准测试（如常识知识和阅读理解测试）中足以抵消对齐代价。为确保预训练梯度的有效性，相比 PPO 数据，必须使用大量的预训练数据。此外，该系数值过大可能会严重阻碍模型向奖励模型对齐，最终损害最终的对齐效果；而该系数值过小对降低对齐代价的作用则微乎其微。

8.2、Qwen 2

Qwen 2 发布于 2024 年 6 月，是一系列基于 Transformer 架构并使用下一个词元预测进行训练的大型语言模型。该系列模型涵盖了 4 个基础模型，参数数量分别为 5 亿、15 亿、70 亿和 720 亿，以及一个拥有 570 亿参数的专家混合 (MoE) 模型，该模型在处理每个词元时会激活 140 亿个参数。所有模型均在一个高质量、大规模的数据集上进行预训练，该数据集包含超过 7 万亿个词元，涵盖广泛的领域和语言。

8.2.1、分词器

与 Qwen 1 采用基于字节级字节对编码的相同分词器，所有规模的模型都使用了一个通用词表，该词表包含 151,643 个常规词元以及 3 个控制词元。

8.2.2、模型架构

Qwen2 是基于 Transformer 架构的大型语言模型，其特点是采用带因果掩码的自注意力机制。具体而言，该系列包括 4 种规模的稠密语言模型以及一个混合专家（MoE）模型。

8.2.2.1、稠密模型

Qwen2 稠密模型的架构由多个 Transformer 层组成，每个层都配备因果注意力机制和前馈神经网络（FFN）。

分组查询注意力

Qwen 2 采用分组查询注意力（GQA），而非传统的多头注意力（MHA）。GQA在推理过程中优化了键值（KV）缓存的使用，显著提高了吞吐量

结合 YARN 的双块注意力

通过双块注意力（DCA）扩大 Qwen 2 的上下文窗口，其将长序列分割成易于处理长度的块。如果输入可以在一个块内处理，DCA 产生的结果与原始注意力相同。否则，DCA 有助于有效捕捉块内和块间词元之间的相对位置信息，从而提升长上下文性能。此外，采用 YARN 对注意力权重进行重新缩放，以实现更好的长度外推。

Qwen 2 延续 Qwen 1 的做法，使用 SwiGLU 作为激活函数，旋转位置嵌入（RoPE）进行位置嵌入，在注意力机制中使用 QKV 偏置，并采用 RMSNorm 和预归一化来确保训练稳定性。

8.2.2.2、专家混合模型

Qwen 2 MoE 模型的架构中，MoE FFN（混合专家前馈网络）作为原始 FFN 的替代，由 n 个独立的 FFN 组成，每个 FFN 作为一个专家。每个 token 根据由门控网络 G 分配的概率，指派给特定的专家 E_i 进行计算：

$$p = \text{softmax}(G(x))$$

$$y = \sum_{i \in \text{topk}(p)} p_i E_i(x)$$

- **专家粒度**

- MoE 模型与密集模型的主要结构差异在于，MoE 层包含多个 FFN，每个 FFN 作为一个单独的专家。因此，从密集架构过渡到 MoE 架构的一种简单策略是将每个专家的参数设置为原始密集模型中单个 FFN 的参数。与此不同，Qwen 2 MoE 采用了精细粒度的专家，创建了较小规模的专家，同时激活更多的专家。考虑到专家参数的总量和激活参数数目相等，精细粒度的专家提供了更丰富的专家组合。通过利用这些精细粒度的专家，Qwen2 MoE 能够实现更多样化和动态的专家使用，从而提升整体性能和适应性

- **专家路由**

- 专家路由机制的设计对于提高 MoE 模型的性能至关重要。Qwen 2 采用了在 MoE 层中集成共享专家和路由专用专家，使得共享专家可以在多个任务中应用，同时将其他专家保留供特定路由场景使用。引入共享专家和专用专家为开发 MoE 路由机制提供了更具适应性和高效性的方法

- **专家初始化**

- Qwen 2 以类似于“upcycling”的方法初始化专家，利用密集模型的权重。有所区别的是，Qwen 2 强调在精细粒度的专家之间进行多样化，以增强模型的表示能力。给定指定的专家中间尺寸 h_E 、专家数量 n 和原始 FFN 中间尺寸 h_{FFN} ，将 FFN 复制 $\lceil \frac{n \times h_E}{h_{FFN}} \rceil$ 次，确保了与指定的专家数量兼容，同时能够适应任意的专家中间尺寸。为了促进每个 FFN 副本中的多样性，Qwen 2 沿着中间维度对参数进行打乱。确保即使在不同的 FFN 副本中，每个精细粒度的专家也能展现出独特的特征。随后，这些专家从 FFN 副本中提取出来，剩余的维度则被丢弃。对于每个精细粒度的专家，随机重新初始化 50% 的参数。这一过程为专家初始化引入了额外的随机性，可能增强了模型在训练过程中的探索能力

8.2.3、预训练

8.2.3.1、预训练数据

Qwen 2 预训练数据量从 Qwen1.5 中的 3 万亿 tokens 扩展到了 7 万亿 tokens。尝试进一步放宽质量阈值，最终得到了一个包含 12 万亿 tokens 的数据集。在这个数据集上训练的模型在性能上并未显著超越使用 7 万亿 tokens 的数据集的模型。推测增加数据量并不一定有利于模型的预训练，考虑到训练成本选择使用质量更高的 7 万亿 tokens 数据集来训练更大的模型

与“upcycling”原则一致，MoE 模型额外获得了 4.5 万亿 tokens 的预训练。高质量的多任务指令数据被整合到 Qwen2 的预训练过程中，以增强模型的上下文学习能力和指令跟随能力。

8.2.3.2、长上下文训练

为了提升 Qwen 2 的长上下文能力，在预训练的收尾阶段将上下文长度从 4096 增加到 32768，此次扩展还伴随着大量高质量、长篇数据的引入。结合这些改进，将 RoPE 的基础频率从 10,000 调整至 1,000,000，以优化长上下文场景下的性能。

为了充分发挥模型的长度外推潜力，Qwen 2 采用了 YARN 机制和双块注意力机制。这些策略使模型能够在处理长达 131,072 个令牌的序列时保持高性能，初步实验中极小的困惑度下降便是证明。

8.2.4、后训练

后训练数据（post-training data）的构建旨在增强模型在广泛领域的能力，包括编码、数学、逻辑推理、指令遵循和多语言理解，以及确保模型的生成结果符合人类价值观，使其有用、诚实和无害。与传统的依赖大量人工监督的方法不同，Qwen 2 专注于通过最小化人工标注来实现可扩展的对齐。

8.2.4.1、后训练数据

后训练数据主要由两部分组成：演示数据 $\mathcal{D} = (x_i, y_i)$ 和偏好数据 $\mathcal{P} = (x_i, y_i^+, y_i^-)$ ，其中 x_i 表示指令， y_i 表示一个满意的响应，而 y_i^+ 和 y_i^- 是对 x_i 的两个响应， y_i^+ 被认为是优于 y_i^- 的选择。集合 $\mathcal{D}$ 用于监督微调（SFT），而集合 $\mathcal{P}$ 用于强化学习从人类反馈（RLHF）。

数据构建的两步过程：

1. 协作数据标注

自动本体提取： 应用 InsTag 细粒度标签器，从大规模指令语料库中提取底层本体，并进行手动细化以确保提取的本体的准确性

指令选择： 每个带有标签的指令都会根据标签多样性、语义丰富性、复杂性和意图完整性进行评估。基于这些标准，选择一组具有代表性的指令

指令进化： 为了丰富指令数据集，采用自进化策略提示 Qwen 模型向现有指令添加约束或要求，从而增加其复杂度，并确保数据集中有多样化的难度级别

人工标注：通过多种生成策略和不同规模的 Qwen 模型获取指令的多个响应。标注者根据偏好对这些响应进行排名，确保最佳响应符合既定标准，从而产生演示数据和偏好数据

2. 自动化数据合成

拒绝采样：对于具有明确最终答案的数学或类似任务，采用拒绝采样来提高解决方案的质量。LLMs 需针对每条指令生成多个回复，即推理路径。能得出准确结论且被模型认为合理的路径将被保留用作示范数据，偏好数据则通过对比正确与错误路径生成

执行反馈：对于编码任务，LLMs 被用来生成解决方案和相关测试用例。对于编码任务，会利用大语言模型生成解决方案及相关测试用例。通过依据测试用例对这些解决方案进行编译和执行，来评估其有效性，进而创建示范数据和偏好数据。这种方法同样适用于评估指令遵循情况。对于每条有约束条件（如长度限制）的指令，会要求大语言模型生成一个 Python 验证函数，以确保生成的回复符合指令要求

数据再利用：从公共领域收集大量高质量文学作品，并利用 LLMs 来设计不同详细程度的指令。这些指令与原始作品配对，作为示范数据。例如，为了编写具有生动、引人入胜的角色的角色扮演数据，从知识库如维基百科获取详细的角色档案，并指导 LLMs 生成相应的指令和回应。这一过程类似于阅读理解任务，确保了人物简介的完整性

宪法反馈：为了确保遵守如安全性和价值观等指导原则，编制宪法数据集明确列出需要遵循和避免的原则来指导 LLMs 生成符合或偏离这些指南的响应，作为示范和偏好数据的参考

8.2.4.2、监督微调（SFT）

整合包含超过 50 万个示例的指令数据集，涵盖了诸如指令遵循、编码、数学、逻辑推理、角色扮演、多语言处理以及安全性等技能领域。模型在序列长度为 32768 个词元的情况下，进行了两个轮次的微调。为优化学习过程，学习率从 7×10^{-6} 逐步降至 7×10^{-7} 。为应对过拟合问题，采用 0.1 的权重衰减，并将梯度最大值裁剪为 1.0。

8.2.4.3、基于人类反馈的强化学习（RLHF）

基于人类反馈的强化学习（RLHF）训练机制包含两个连续阶段：离线训练和在线训练。

在离线训练阶段，使用预先编译好的偏好数据集 P ，通过直接偏好优化（DPO），最大化 y_i^+ 与 y_i^- 之间的似然差异。

在**在线训练**阶段，模型利用奖励模型实时提供的即时反馈，迭代地实时优化自身性能。具体而言，从当前策略模型中采样多个回复，奖励模型从中选择最受偏好和最不受偏好的回复，形成偏好对，在每个训练周期中用于直接偏好优化。

此外，采用在线合并优化器（Online Merging Optimizer）来缓解对齐成本（alignment tax），即将模型生成与人类偏好对齐所带来的性能下降问题。

8.3、Qwen 2.5

Qwen 2.5 发布于 2024 年 9 月，预训练数据从之前的 7 万亿扩展到 18 万亿个 token，重点关注知识、编程和数学领域。预训练采用两阶段方法，初始阶段为 4096 个 token，最终阶段扩展到 32768 个 token，并对 Qwen2.5-Turbo 采用逐步扩展策略，达到 262144 个 token。

Qwen 2.5 后训练包括监督微调和多阶段强化学习。其中监督微调使用超过 100 万个样本，涵盖长序列生成、数学问题解决、编码、指令跟随、结构化数据理解、逻辑推理、跨语言迁移和系统指令等方面。强化学习分为离线学习和在线学习两个阶段，离线学习专注于推理、事实性和指令遵循等难以评估的任务，而在线学习则利用奖励模型来提高输出的质量（如真实性、有帮助性、简洁性、相关性、无害性和去偏）。

9、DeepSeek

9.1、DeepSeek LLM

DeepSeek LLM 发布于 2023 年 11 月，收集了 2 万亿个词元用于预训练。在模型层面沿用了 LLaMA 的架构，将余弦退火学习率调度器替换为多步学习率调度器，在保持性能的同时便于持续训练。

DeepSeek LLM 从多种来源收集了超过 100 万个实例，用于监督微调（SFT）。此外，利用直接偏好优化（DPO）来提升模型的对话性能。

9.1.1、预训练

9.1.1.1、数据

数据构建的主要目标是全面提升数据集的丰富性和多样性，将方法组织为三个基本阶段：去重、过滤和重新混合。去重和重新混合阶段通过采样独特实例确保数据的多样表现形式。过滤阶段增强了信息的密度，从而使得模型训练更加高效和有效。

DeepSeek LLM 基于分词器库（Huggingface）实现了字节级字节对编码（BBPE）算法。与GPT - 2类似，采用预分词处理，以防止不同字符类别的词元（如换行符、标点符号和中日韩（CJK）字符）合并。此外，将数字拆分为单个数字，且词表中的常规词元数量设定为100000个。该分词器在一个约24GB 的多语言语料库上进行训练，在最终词表中添加了 15 个特殊词元，使词表总大小达到 100015 个。为了确保训练过程中的计算效率，并为未来可能需要的额外特殊词元预留空间，将模型训练时的词表大小配置为102400。

9.1.1.2、架构

DeepSeek LLM 的微观设计在很大程度上遵循了 LLaMA 的设计思路，采用了带有均方根归一化（RMSNorm）函数的预归一化（Pre-Norm）结构，并使用 SwiGLU 作为前馈网络（FFN）的激活函数，其中间层维度为 $\frac{8}{3} d_{model}$ 。此外，采用旋转嵌入（RoPE）进行位置编码。为了优化推理成本，670 亿参数的模型使用了分组查询注意力机制（GQA），而非传统的多头注意力机制（MHA）。

在宏观设计方面，70 亿参数的 DeepSeek LLM 是一个 30 层的网络，670 亿参数的 DeepSeek LLM 有 95 层。与大多数使用分组查询注意力机制（GQA）的工作不同，对 670 亿参数模型参数的扩展是在网络深度上进行的，而非常见的拓宽前馈网络层的中间宽度，目的是为了获得更好的性能。

Params	n_{layers}	d_{model}	n_{heads}	n_{kv_heads}	Context Length	Sequence Batch Size	Learning Rate	Tokens
7B	30	4096	32	32	4096	2304	4.2e-4	2.0T
67B	95	8192	64	8	4096	4608	3.2e-4	2.0T

(/img/post/llm-model-42.png)

9.1.1.3、超参数

DeepSeek LLM 以 0.006 的标准差进行初始化，并使用 AdamW 优化器进行训练 ($\beta_1 = 0.9$, $\beta_2 = 0.95$)，权重衰减系数为 0.1。

预训练过程中采用了多步学习率调度器，而非典型的余弦退火学习率调度器。模型的学习率在经过 2000 个 warmup steps 后达到最大值，在处理完 80% 的训练词元后，学习率降至最大值的 31.6%；在处理完 90% 的训练词元后，进一步降至最大值的 10%。训练阶段的梯度裁剪阈值设置为 1.0。

尽管训练过程中损失下降趋势存在差异，但使用多步学习率调度器最终获得的性能与使用余弦退火学习率调度器基本一致。在固定模型规模并调整训练规模时，多步学习率调度器允许复用第一阶段的训练成果，为持续训练提供了独特的便利。因此选择多步学习率调度器作为默认设置。

调整多步学习率调度器不同阶段的比例可以略微提升性能。然而，为了平衡持续训练中的复用率和模型性能，选择上述三个阶段分别为 80%、10% 和 10% 的比例分配。此外，batch size 和学习率会随模型规模的不同而变化。

9.1.1.4、基础设施

DeepSeek LLM 使用 HAI-LLM 高效轻量级训练框架来训练和评估大语言模型。该框架集成了数据并行、张量并行、序列并行和 1F1B 流水线并行。此外，利用闪存注意力机制技术来提高硬件利用率。采用 ZeRO-1 方法在数据并行进程中划分优化器状态。为了最大程度减少额外的等待开销，努力让计算和通信过程重叠，包括最后一个微批次（micro-batch）的反向传播过程以及 ZeRO-1 中的 reduce-scatter 操作，以及序列并行中的矩阵乘法（GEMM）计算和 all-gather/reduce-scatter 操作。

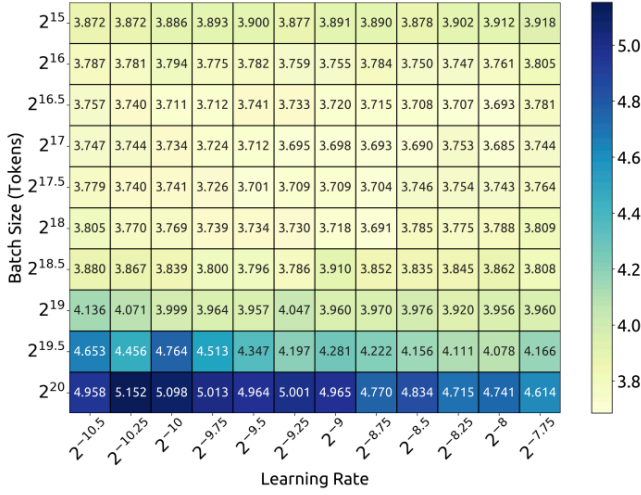
DeepSeek LLM 对一些层/算子进行了融合以加速训练，包括层归一化（LayerNorm）、尽可能融合的矩阵乘法，以及 Adam 优化器的更新步骤。为提高模型训练的稳定性，以 bf16 精度训练模型，以 fp32 精度累积梯度。执行原位交叉熵计算以减少 GPU 内存消耗，即在交叉熵的 CUDA 内核中动态地将 bf16 的对数几率（logits）转换为 fp32 精度（而不是事先在高带宽内存（HBM）中进行转换），计算相应的 bf16 梯度，并用梯度覆盖对数几率（logits）。

模型权重和优化器状态每 5 分钟异步保存一次，即在偶尔出现硬件或网络故障的最坏情况下，最多只会损失 5 分钟的训练进度。此外，定期清理这些临时的模型检查点，以避免占用过多的存储空间。并支持从不同的三维并行配置恢复训练，以应对计算集群负载的动态变化。

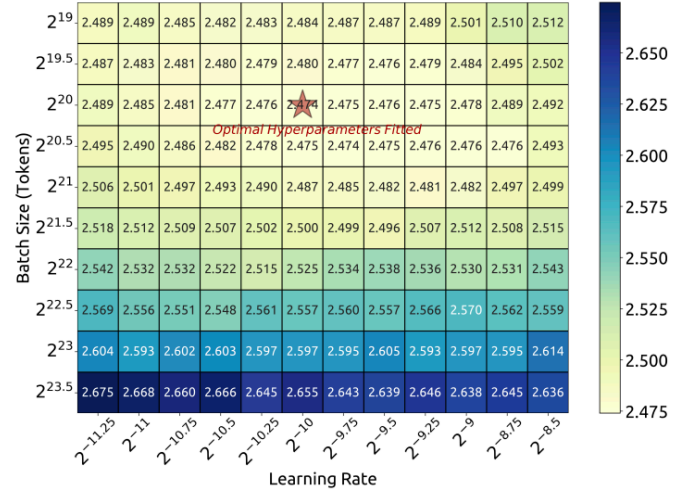
9.1.2、缩放定律

9.1.2.1、超参数缩放定律

DeepSeek LLM 最初在计算预算为 10^{17} 的小规模实验中对批量大小和学习率进行了网格搜索，特定模型规模（每词元 1.77 亿次浮点运算）的实验结果如下图所示。结果表明，在广泛的批量大小和学习率选择范围内，泛化误差保持稳定。由此可知，在相对宽泛的参数空间内能够实现接近最优的性能。



(a) 1e17 FLOPs (177M FLOPs/token)

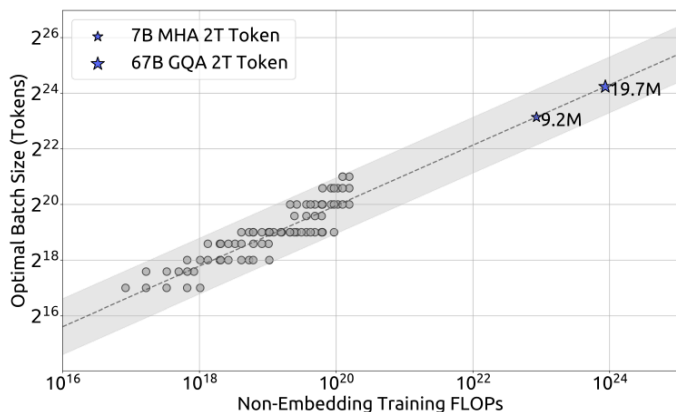


(b) 1e20 FLOPs (2.94B FLOPs/token)

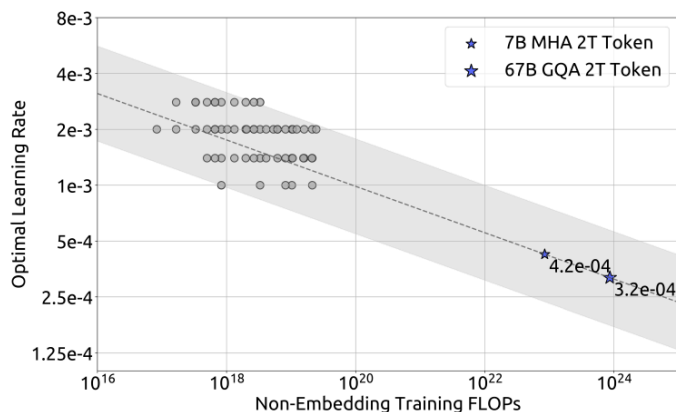
(/img/post/llm-model-43.png)

利用上述多步学习率调度器，通过复用第一阶段的训练成果，对多个具有不同批量大小、学习率以及计算预算（范围从 10^{17} 到 2×10^{19} ）的模型进行了有效训练。考虑到参数空间存在冗余，将泛化误差超过最小值不超过 0.25 的模型所使用的参数视为接近最优的超参数。接着，针对计算预算 C 对批量大小 B 和学习率 η 进行了拟合。如下图所示，拟合结果显示，最优批量大小 B 随着计算预算 C 的增加而逐渐增大，而最优学习率 η 则逐渐减小。这与模型扩量时对批量大小和学习率的直观经验设置相符。此外，所有接近最优的超参数都落在一个较宽的区间范围内，这表明在该区间内选择接近最优的参数相对容易。最终拟合得到的批量大小和学习率公式如下：

$$\begin{aligned}\eta_{opt} &= 0.3118 \cdot C^{-0.1250} \\ B_{opt} &= 0.2920 \cdot C^{0.3271}\end{aligned}\tag{1}$$



(a) Batch size scaling curve



(b) Learning rate scaling curve

(/img/post/llm-model-44.png)

DeepSeek LLM 在一系列计算预算为 10^{20} 的模型上对这些公式进行了验证，特定模型规模（每词元 29.4 亿次浮点运算）的验证结果如上图 (b) 所示。结果表明，拟合得到的参数处于最优参数空间的中心位置。

需要注意的是，尚未考虑计算预算 C 之外的因素对最优超参数的影响。这与一些早期的研究成果不一致，这些研究认为最优批量大小可以建模为仅与泛化误差 L 相关。此外，在计算预算相同但模型/数据分配不同的模型中，最优参数空间存在细微差异。

9.1.2.2、估算最优模型和数据规模

在推导出用于拟合接近最优超参数的公式后，通过拟合缩放曲线分析最优的模型/数据扩量分配策略。策略旨在找到分别满足 $N_{opt} \propto C^a$ 和 $D_{opt} \propto C^b$ 的模型缩放指数 a 和数据缩放指数 b 。数据规模 D 可以始终用数据集中的词元数量来表示。

模型规模通常用模型参数来表示，包括非嵌入层参数 N_1 和完整参数 N_2 。计算预算 C 与模型/数据规模之间的关系可以近似描述为 $C = 6ND$ ，即可以用 $6N_1$ 或 $6N_2$ 来近似表示模型规模。

然而，由于 $6N_1$ 和 $6N_2$ 都没有考虑注意力操作的计算开销，并且 $6N_2$ 还包含了对模型容量贡献较小的词表计算，所以在某些设置下，它们都存在显著的近似误差。

为了减少误差，引入新的模型规模表示方法：非嵌入层每词元浮点运算次数 M 。 M 包含了注意力操作的计算开销，但不考虑词表计算。用 M 来表示模型规模时，计算预算 C 可以简单地表示为 $C = MD$ 。 $6N_1$ 、 $6N_2$ 和 M 之间的具体差异如下列公式所示：

$$\begin{aligned} 6N_1 &= 72n_{\text{layer}}d_{\text{model}}^2 \\ 6N_2 &= 72n_{\text{layer}}d_{\text{model}}^2 + 6n_{\text{vocab}}d_{\text{model}} \\ M &= 72n_{\text{layer}}d_{\text{model}}^2 + 12n_{\text{layer}}d_{\text{model}}l_{\text{seq}} \end{aligned} \tag{2}$$

其中， n_{layer} 表示层数， d_{model} 表示模型宽度， n_{vocab} 是词表大小， l_{seq} 是序列长度。如下表所示， $6N_1$ 和 $6N_2$ 在不同规模的模型中都会高估或低估计算成本。这种差异在小规模模型中尤为明显，差异幅度可达 50%。在拟合缩放曲线时，这种不准确性可能会引入大量的统计误差。

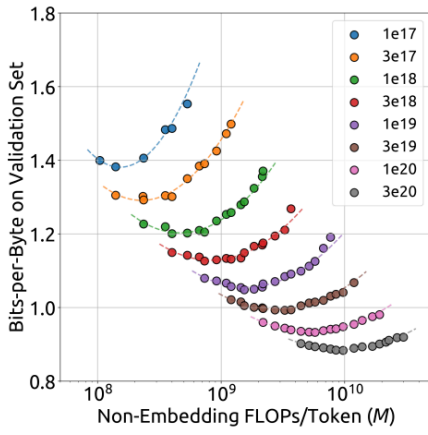
n_{layers}	d_{model}	n_{vocab}	l_{seq}	N_1	N_2	M	$\frac{6N_1}{M}$	$\frac{6N_2}{M}$
8	512	102400	4096	25.2M	77.6M	352M	0.43	1.32
12	768			84.9M	164M	963M	0.53	1.02
24	1024			302M	407M	3.02B	0.60	0.81
24	2048			1.21B	1.42B	9.66B	0.75	0.88
32	4096			6.44B	6.86B	45.1B	0.85	0.91
40	5120			12.6B	13.1B	85.6B	0.88	0.92
80	8192			64.4B	65.3B	419B	0.92	0.94

(/img/post/llm-model-45.png)

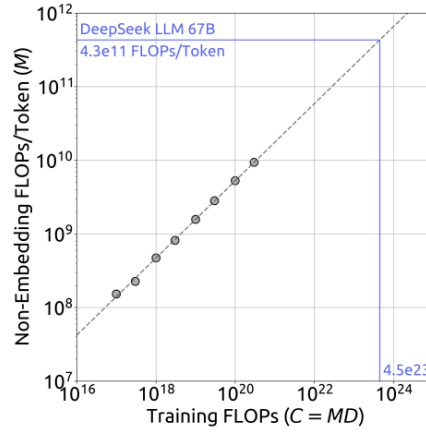
在采用 M 来表示模型规模后，目标可以更清晰地描述为：**在给定计算预算 $C = MD$ 的情况下，找到使模型泛化误差最小的最优模型规模 M_{opt} 和数据规模 D_{opt}** 。这一目标可以形式化表述为：

$$M_{\text{opt}}(C), D_{\text{opt}}(C) = \underset{M, D \text{ s.t. } C=MD}{\operatorname{argmin}} \quad L(N, D)$$

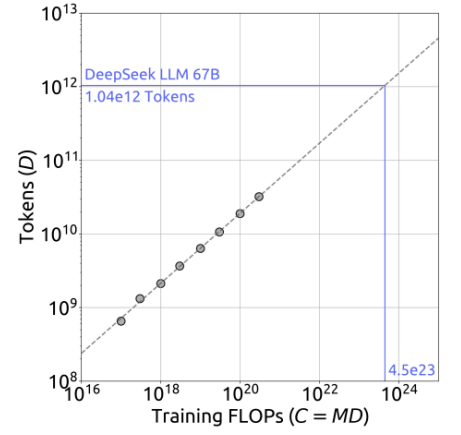
为降低实验成本和拟合难度，采用了来自 “Chinchilla” 模型的等浮点运算量 (IsoFLOP) 配置方法来拟合缩放曲线。DeepSeek LLM 选取了 8 种不同的计算预算，范围从 10^{17} 到 3×10^{20} ，并为每个预算设计了约 10 种不同的模型/数据规模分配方案。每个预算对应的超参数由公式 (1) 确定，泛化误差则在一个独立的验证集上进行计算，该验证集的数据分布与训练集相似，包含 1 亿个词元。



(a) IsoFLOP curve



(b) Optimal model scaling



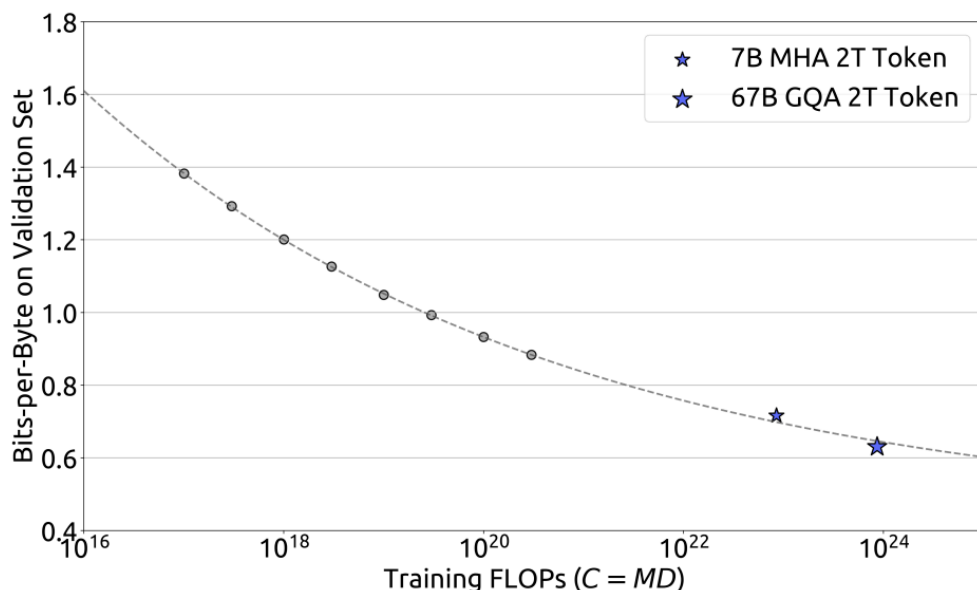
(c) Optimal data scaling

(/img/post/llm-model-46.png)

上图展示了等浮点运算量曲线以及模型/数据缩放曲线，这些曲线是通过针对每个计算预算采用最优的模型/数据分配方案拟合得到的。最优非嵌入层每词元浮点运算次数 M_{opt} 和最优词元数量 D_{opt} 的具体公式如下：

$$\begin{aligned} M_{opt} &= M_{base} \cdot C^a, & M_{base} &= 0.1715, & a &= 0.5243 \\ D_{opt} &= D_{base} \cdot C^b, & D_{base} &= 5.8316, & b &= 0.4757 \end{aligned} \quad (4)$$

此外，根据计算预算 C 和最优泛化误差拟合损失缩放曲线，并对 DeepSeek LLM 70 亿参数和 670 亿参数模型的泛化误差进行预测，如图所示。结果表明，利用小规模实验能够准确预测计算预算为其 1000 倍的模型的性能。



(/img/post/llm-model-47.png)

9.1.3、对齐

DeepSeek LLM 收集了约 150 万条中英文指令数据实例，涵盖了广泛的有用性和无害性相关主题。其中，有用性数据包含 120 万条实例，具体分布为：通用语言任务占 31.2%，数学问题占 46.6%，编程练习占 22.2%。安全性数据由 30 万条实例组成，涉及各类敏感主题。

9.1.3.1、监督微调（SFT）

DeepSeek LLM 对 70 亿参数模型进行了 4 个轮次的微调，因为 670 亿参数模型的过拟合问题较为严重，所以 670 亿参数模型仅进行了 2 个轮次的微调。70 亿参数模型和 670 亿参数模型的学习率分别为 10^{-5} 和 5×10^{-6} 。

除了监控基准测试的准确率外，还在微调过程中评估聊天模型的重复率。DeepSeek LLM 总共收集了 3868 条中英文提示，并确定了生成的回复中未能正常结束、而是不断重复一段文本的比例。随着数学监督微调数据量的增加，重复率往往会上升。这可能是因为数学监督微调数据偶尔会在推理过程中包含相似的模式。因此，能力较弱的模型难以掌握这些推理模式，从而导致回复出现重复。为了解决这个问题，DeepSeek LLM 尝试了两阶段微调方法和直接偏好优化（DPO），这两种方法都能在保证基准测试得分的同时，显著降低重复率。

9.1.3.2、直接偏好优化（DPO）

为进一步提升模型能力，DeepSeek LLM 采用了 DPO，该算法已被证明是一种简单而有效的大语言模型对齐优化方法。DeepSeek LLM 从有用性和无害性两个方面构建了用于 DPO 训练的偏好数据。对于有用性数据，收集了多语言提示，涵盖创意写作、问答、指令遵循等类别。然后使用 DeepSeek Chat 模型生成回复，作为候选回复。构建无害性偏好数据时也采用了类似的操作。

DeepSeek LLM 对 DPO 进行了一个轮次的训练，学习率为 5×10^{-6} ，批量大小为 512，并使用了学习率预热和余弦退火学习率调度器。DPO 可以增强模型的开放式生成能力，而在标准基准测试中的性能提升并不明显。

9.2、DeepSeekMath

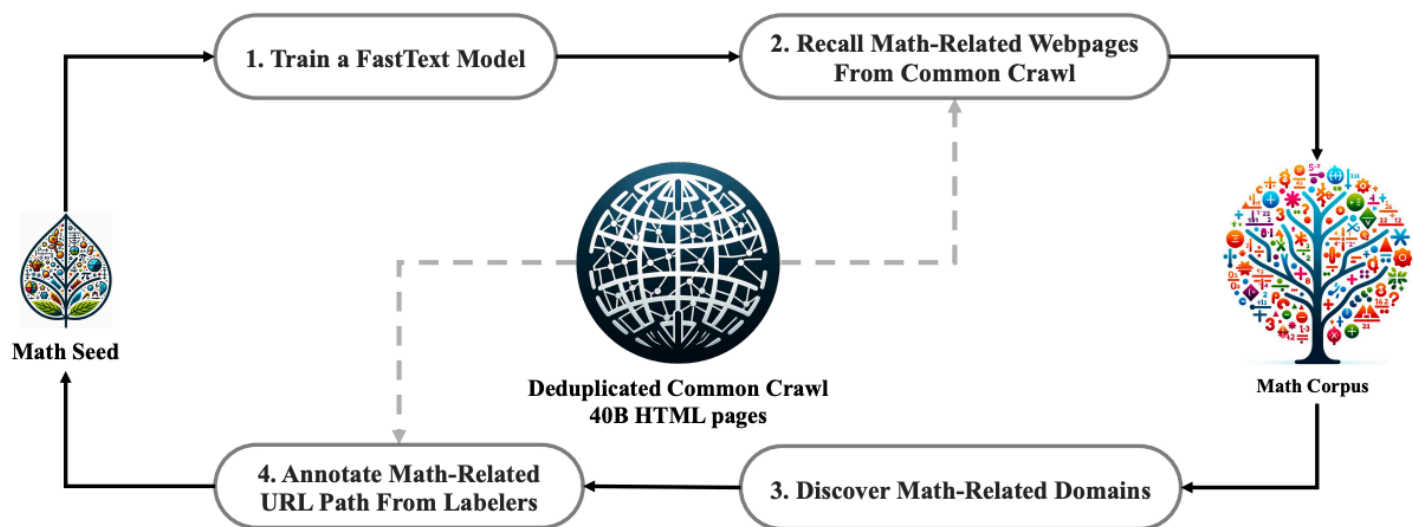
DeepSeekMath 发布于 2024 年 2 月，基于 DeepSeek-Coder-Base-v1.5 7B 继续进行预训练，所使用的数据包括从 Common Crawl 获取的 1200 亿与数学相关的 token，以及自然语言和代码数据。

DeepSeekMath 的数学推理能力归因于两个关键因素：其一，通过精心设计的数据筛选流程，充分挖掘了公开可用网络数据的巨大潜力。其二，引入了群组相对策略优化（GRPO），它是近端策略优化（PPO）的一种变体，在提升数学推理能力的同时，优化了 PPO 的内存使用情况。

9.2.1、数学预训练

9.2.1.1、数据收集与去污染

如图展示了一个迭代流程，即如何从一个种子语料库（例如，一个规模小但质量高的数学相关数据集）开始，系统地从 Common Crawl 收集大规模数学语料库。值得注意的是，这种方法也适用于其他领域，如编码领域。



(/img/post/llm-model-62.png)

首先，选择 OpenWebMath（一个高质量数学网页文本的集合）作为最初的种子语料库。利用这个语料库，训练一个 fastText 模型，以召回更多类似 OpenWebMath 的数学网页。具体来说，从种子语料库中随机选择 50 万个数据点作为正训练示例，从 Common Crawl 中再选择 50 万个网页作为负训练示例。在数据收集中使用一个开源库进行训练，将向量维度配置为 256，学习率设置为 0.1，词 n-gram 的最大长度设为 3，词出现的最小次数设为 3，训练轮数设为 3。为了减小原始 Common Crawl 的规模，采用基于 URL 的去重和近似去重技术，最终得到 400 亿个 HTML 网页。然后，使用 fastText 模型从去重后的 Common Crawl 中召回数学网页。

为了过滤掉低质量的数学内容，根据 fastText 模型预测的分数对收集到的网页进行排序，只保留排名靠前的网页。通过对前 400 亿、800 亿、1200 亿和 1600 亿 token 进行预训练实验来评估保留的数据量。在第一轮迭代中，选择保留前 400 亿个 token。

在第一轮数据收集之后，仍有大量数学网页未被收集，这主要是因为 fastText 模型是在一组多样性不足的正例上训练的。因此，确定了其他数学网页来源来丰富种子语料库，以便优化 fastText 模型。具体来说，首先将整个 Common Crawl 组织成不相交的域；一个域被定义为共享相同基础 URL 的网页集合。对于每个域，计算在第一轮迭代中被收集的网页的百分比。超过 10% 的网页被收集的域被归类为与数学相关（例如，mathoverflow.net）。随后，手动标注这些已识别域内与数学内容相关的 URL（例如，mathoverflow.net/questions）。与这些 URL 链接但尚未收集的网页将被添加到种子语料库中。这种方法能够收集更多正例，从而训练出一个改进的 fastText 模型，该模型能够在后续迭代中召回更多数学数据。经过四轮数据收集，最终得到 3550 万个数学网页，总计 1200 亿个 token。在第四轮迭代中，注意到近 98% 的数据在第三轮迭代中已经被收集，所以决定停止数据收集。

为避免基准测试污染，从英语数学基准测试（如 GSM8K 和 MATH）以及中文基准测试（如 CMATH 和 AGIEval）中过滤掉包含问题或答案的网页。过滤标准如下：任何文本段中若包含一个 10-gram 字符串，且该字符串与评估基准测试中的任何子字符串完全匹配，则将其从数学训练语料库中移除。对于长度小于 10-gram 但至少含有 3-gram 的基准测试文本，采用精确匹配来过滤掉受污染的网页。

9.2.1.2、训练设置

DeepSeekMath 对一个具有 13 亿参数的通用预训练语言模型进行数学训练，该模型与 DeepSeek LLM 采用相同框架，记为 DeepSeek LLM 1.3B。在每个数学语料库上分别对模型进行 1500 亿 token 的训练。所有实验均使用高效轻量的 HAI-LLM 训练框架进行。遵循 DeepSeek LLM 的训练实践，使用 AdamW 优化器，其中 $\beta_1 = 0.9$, $\beta_2 = 0.95$ ，权重衰减系数 $\text{weight_decay} = 0.1$ ，同时采用多步学习率调度策略：学习率在经过 2000 步 warmup steps 后达到峰值，在训练进程的 80% 时降至峰值的 31.6%，在训练进程的 90% 时进一步降至峰值的 10.0%。将学习率的最大值设为 $5.3\text{e-}4$ ，并使用 batch size 为 400 万个 token，上下文长度为 4000 的训练设置。

9.2.2、监督式微调

DeepSeekMath 构建了一个数学指令微调数据集，涵盖来自不同数学领域、具有不同复杂程度的中英文数学问题。这些问题与以思维链（CoT）、程序思维（PoT）以及工具集成推理格式呈现的解决方案一一配对。训练示例总数为 77.6 万个。

- **英文数学数据集：**为 GSM8K 和 MATH 问题标注了工具集成解决方案，并采用了 MathInstruct 的一个子集，以及 Lila-OOD 的训练集，其中的问题通过思维链（CoT）或程序思维（PoT）解决。英文数据集涵盖了数学的多个领域，如代数、概率、数论、微积分和几何
- **中文数学数据集：**收集了涵盖 76 个子主题（如线性方程）的中国 K-12 阶段数学问题，并以思维链（CoT）和工具集成推理格式对解决方案进行了标注

9.2.3、群组相对策略优化

9.2.3.1、从 PPO 到 GRPO

近端策略优化（PPO）是一种基于 演员-评论家（actor-critic） 的强化学习算法，广泛应用于大语言模型的强化学习微调阶段。具体而言，它通过最大化以下替代目标来优化大语言模型：

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \epsilon, 1 + \epsilon \right) \right]$$

其中：

- π_{θ} 和 $\pi_{\theta_{old}}$ 分别是当前和旧的策略模型， q 、 o 分别是问题数据集和旧策略 $\pi_{\theta_{old}}$ 中采样得到的问题和输出
- ϵ 是 PPO 中引入的与裁剪相关的超参数，用于稳定训练
- A_t 是优势值，通过应用广义优势估计（GAE），基于奖励 $r_{\geq t}$ 和学习到的价值函数 V_{ψ} 计算得出

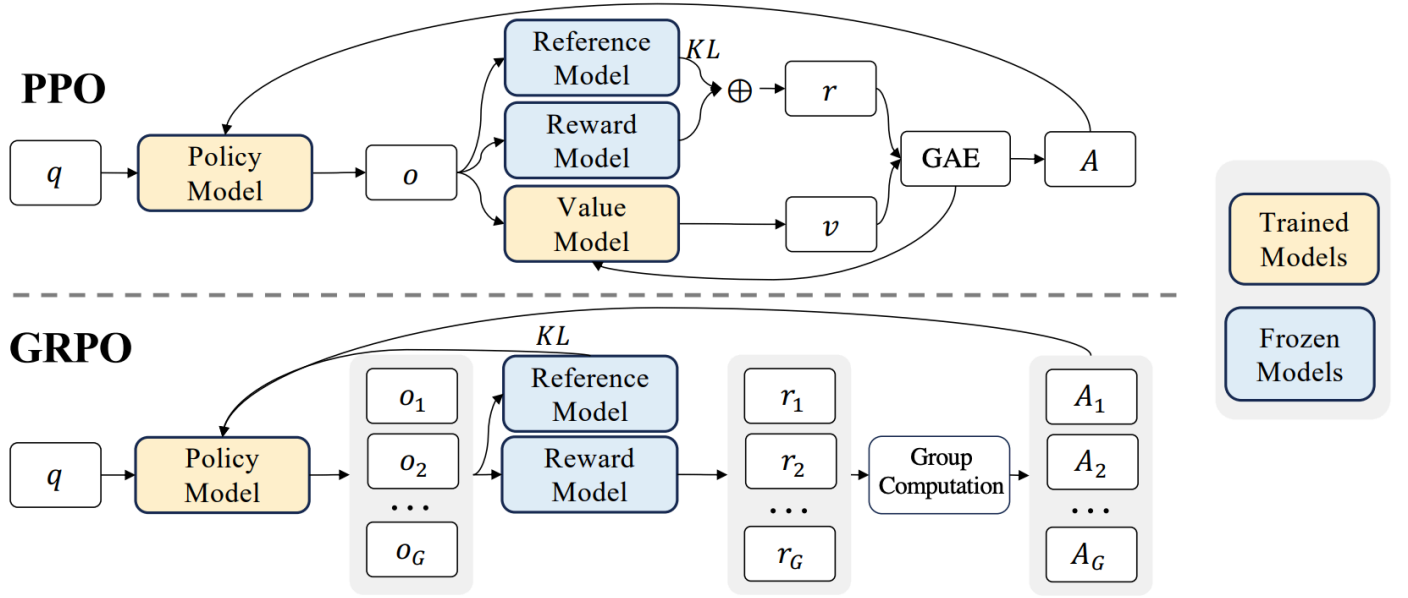
因此，在 PPO 中，价值函数需要与策略模型一同训练，并且为了减轻奖励模型的过度优化，标准方法是在每个 token 的奖励中添加来自参考模型的每个 token 的 KL 散度惩罚项，即：

$$r_t = r_{\varphi}(q, o_{\leq t}) - \beta \log \frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{ref}(o_t|q, o_{<t})} \quad (2)$$

其中， r_{φ} 是奖励模型， π_{ref} 是参考模型，通常是初始的监督微调模型， β 是 KL 散度惩罚项的系数。

由于 PPO 中使用的价值函数通常是与策略模型规模相当的另一个模型，这带来了巨大的内存和计算负担。此外，在强化学习训练期间，价值函数在优势值计算中被用作基线以减少方差。然而，在大语言模型的情境中，通常只有最后一个 token 由奖励模型赋予奖励分数，这可能会使在每个 token 上都准确的价值函数的训练变得复杂。

为了解决这些问题，如图所示，DeepSeekMath 提出了组相对策略优化（GRPO），它无需像 PPO 那样进行额外的价值函数近似，而是使用针对同一问题生成的多个采样输出的平均奖励作为基线。



(/img/post/llm-model-63.png)

近端策略优化（PPO）与群组相对策略优化（GRPO）的展示。GRPO 无需价值模型，而是通过组分数来估计基线，显著减少了训练资源

更具体地说，对于每个问题 q ，GRPO 从旧策略 $\pi_{\theta_{old}}$ 中采样一组输出 $\{o_1, o_2, \dots, o_G\}$ ，然后通过最大化以下目标来优化策略模型：

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \right] \right\}$$

其中， ε 和 β 是超参数， $\hat{A}_{i,t}$ 是仅基于每组内输出的相对奖励计算得到的优势值，将在以下小节中详细介绍。GRPO 利用组相对的方式计算优势值，这与奖励模型的比较性质非常契合，因为奖励模型通常是在关于同一问题的输出之间的比较数据集上进行训练的。

还需注意的是，GRPO 不是在奖励中添加 KL 散度惩罚项，而是通过直接将训练后的策略与参考策略之间的 KL 散度添加到损失中来进行正则化，避免了使 $\hat{A}_{i,t}$ 的计算复杂化。并且与公式（2）中使用的 KL 散度惩罚项不同，DeepSeekMath 使用以下无偏估计器来估计 KL 散度：

$$\mathbb{D}_{KL}[\pi_{\theta}||\pi_{ref}] = \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - \log \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - 1 \quad (4)$$

该估计器保证为正值。

9.2.3.2、采用 GRPO 的结果监督强化学习

具体而言，对于每个问题 q ，从旧策略模型 $\pi_{\theta_{old}}$ 中采样一组输出 $\{o_1, o_2, \dots, o_G\}$ 。然后使用奖励模型对这些输出进行评分，相应地得到 G 个奖励 $r = \{r_1, r_2, \dots, r_G\}$ 。

随后，通过减去组平均奖励并除以组标准差来对这些奖励进行归一化处理。结果监督在每个输出 o_i 的末尾提供归一化后的奖励，并将输出中所有标记的优势值 $\hat{A}_{i,t}$ 设置为归一化后的奖励。

即：

$$\hat{A}_{i,t} = \hat{r}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$$

然后通过最大化公式 (3) 中定义的目标来优化策略。

9.2.3.3、采用 GRPO 的过程监督强化学习

结果监督仅在每个输出的末尾提供奖励，这对于复杂数学任务中的策略监督而言，可能不够充分和高效。因此 DeepSeekMath 还探索了过程监督，即在每个推理步骤的末尾都提供奖励。具体来说，给定问题 q 以及 G 个采样输出 $\{o_1, o_2, \dots, o_G\}$ ，使用过程奖励模型对输出的每个步骤进行评分，从而得到相应的奖励：

$$\mathbf{R} = \{\{r_{index(1)}^1, \dots, r_{index(K_1)}^1\}, \dots, \{r_{index(1)}^G, \dots, r_{index(K_G)}^G\}\}$$

其中 $index(j)$ 是第 j 步的结束标记索引， K_i 是第 i 个输出中的总步数。DeepSeekMath 同样通过均值和标准差对这些奖励进行归一化处理，即：

$$\hat{r}_{index(j)}^i = \frac{r_{index(j)}^i - \text{mean}(\mathbf{R})}{\text{std}(\mathbf{R})}$$

随后，过程监督将每个标记的优势计算为后续步骤归一化奖励的总和，即：

$$\hat{A}_{i,t} = \sum_{index(j) \geq t} \hat{r}_{index(j)}^i$$

然后通过最大化公式（3）中定义的目标来优化策略。

9.2.3.4、采用 GRPO 的迭代强化学习

随着强化学习训练过程的推进，旧的奖励模型可能不足以监督当前的策略模型。因此，DeepSeekMath 还探索了基于组相对策略优化（GRPO）的迭代强化学习。如下算法所示，在迭代 GRPO 中，根据策略模型的采样结果为奖励模型生成新的训练集，并使用一种纳入 10% 历史数据的回放机制持续训练旧的奖励模型。然后，将参考模型设置为策略模型，并使用新的奖励模型持续训练策略模型。

Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_{ϕ} ; task prompts $\mathcal{D}$; hyperparameters ϵ, β, μ

- 1: policy model $\pi_{\theta} \leftarrow \pi_{\theta_{\text{init}}}$
- 2: **for** iteration = 1, ..., I **do**
- 3: reference model $\pi_{\text{ref}} \leftarrow \pi_{\theta}$
- 4: **for** step = 1, ..., M **do**
- 5: Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
- 6: Update the old policy model $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$
- 7: Sample G outputs $\{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$ for each question $q \in \mathcal{D}_b$
- 8: Compute rewards $\{r_i\}_{i=1}^G$ for each sampled output o_i by running r_{ϕ}
- 9: Compute $\hat{A}_{i,t}$ for the t -th token of o_i through group relative advantage estimation.
- 10: **for** GRPO iteration = 1, ..., μ **do**
- 11: Update the policy model π_{θ} by maximizing the GRPO objective (Equation 21)
- 12: Update r_{ϕ} through continuous training using a replay mechanism.

Output π_{θ}

(/img/post/llm-model-64.png)

9.2.4、强化学习的见解

9.2.4.1、走向统一范式

在本节中提供了一个统一的范式来分析不同的训练方法，如监督微调（SFT）、拒绝采样微调（RFT）、直接偏好优化（DPO）、近端策略优化（PPO）、组相对策略优化（GRPO），并进一步通过实验探究该统一范式的相关因素。一般来说，某种训练方法关于参数 θ 的梯度可以写成：

$$\nabla_{\theta} \mathcal{J}_{\mathcal{A}}(\theta) = \mathbb{E}_{\underbrace{[(q,o) \sim \mathcal{D}]}_{\text{Data source}}} \left[\frac{1}{|o|} \sum_{t=1}^{|o|} \underbrace{GC_{\mathcal{A}}(q, o, t, \pi_{rf})}_{\text{Gradient Coefficient}} \nabla_{\theta} \log \pi_{\theta}(o_t | q, o_{<t}) \right] \quad (5)$$

其中存在三个关键组成部分：

1. 数据源 $\mathcal{D}$ 决定了训练数据
2. 奖励函数 π_{rf} 是训练奖励信号的来源
3. 算法 $\mathcal{A}$ 将训练数据和奖励信号处理为梯度系数 GC ，该系数决定了对数据的惩罚或强化程度

基于这个统一范式分析几种有代表性的方法：

- **监督微调（SFT）**：SFT 在人类挑选的 SFT 数据上对预训练模型进行微调
- **拒绝采样微调（RFT）**：RFT 基于 SFT 问题，从 SFT 模型采样的输出中筛选出数据，进一步对 SFT 模型进行微调。RFT 根据答案的正确性来筛选输出
- **直接偏好优化（DPO）**：DPO 通过使用成对 DPO 损失，在从 SFT 模型采样的增强输出上对 SFT 模型进行微调，进一步优化 SFT 模型
- **在线拒绝采样微调（Online RFT）**：与 RFT 不同，Online RFT 使用 SFT 模型初始化策略模型，并通过对从实时策略模型采样的增强输出进行微调来优化策略模型
- **PPO/GRPO**：PPO/GRPO 使用 SFT 模型初始化策略模型，并利用从实时策略模型采样的输出对其进行强化

下表中总结了这些方法的组成部分：

Methods	Data Source	Reward Function	Gradient Coefficient
SFT	$q, o \sim P_{sft}(Q, O)$	-	1
RFT	$q \sim P_{sft}(Q), o \sim \pi_{sft}(O q)$	Rule	Equation 10
DPO	$q \sim P_{sft}(Q), o^+, o^- \sim \pi_{sft}(O q)$	Rule	Equation 14
Online RFT	$q \sim P_{sft}(Q), o \sim \pi_{\theta}(O q)$	Rule	Equation 10
PPO	$q \sim P_{sft}(Q), o \sim \pi_{\theta}(O q)$	Model	Equation 18
GRPO	$q \sim P_{sft}(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta}(O q)$	Model	Equation 21

(/img/post/llm-model-65.png)

不同方法的数据源及梯度系数。 P_{sft} 表示有监督微调数据集的数据分布。 $\pi_{\theta_{sft}}$ 和 π_{θ} 分别表示在线训练过程中有监督微调模型和实时策略模型

更详细的推导过程：

1、监督微调 (SFT)

监督微调的目标是最大化以下目标函数：

$$\mathcal{J}_{SFT}(\theta) = \mathbb{E}[q, o \sim P_{sft}(Q, O)] \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \log \pi_{\theta}(o_t | q, o_{<t}) \right). \quad (6)$$

$\mathcal{J}_{SFT}(\theta)$ 的梯度为：

$$\nabla_{\theta} \mathcal{J}_{SFT} = \mathbb{E}[q, o \sim P_{sft}(Q, O)] \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \nabla_{\theta} \log \pi_{\theta}(o_t | q, o_{<t}) \right). \quad (7)$$

数据源：用于监督微调 (SFT) 的数据集。奖励函数：可视为人工选择。梯度系数：始终设为 1。

2、拒绝采样微调 (RFT)

拒绝采样微调首先针对每个问题从经过监督微调的大语言模型 (LLMs) 中采样多个输出，然后使用正确答案在这些采样输出上对大语言模型进行训练。

形式上，拒绝采样微调 (RFT) 的目标是最大化以下目标函数：

$$\mathcal{J}_{RFT}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o \sim \pi_{sft}(O|q)] \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \mathbb{I}(o) \log \pi_{\theta}(o_t | q, o_{<t}) \right). \quad (8)$$

$\mathcal{J}_{RFT}(\theta)$ 的梯度为：

$$\nabla_{\theta} \mathcal{J}_{RFT}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o \sim \pi_{sft}(O|q)] \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \mathbb{I}(o) \nabla_{\theta} \log \pi_{\theta}(o_t | q, o_{<t}) \right). \quad (9)$$

数据源：监督微调 (SFT) 数据集中的 question，输出来自 SFT 模型的采样。奖励函数：规则（答案是否正确）。梯度系数：

$$GC_{RFT}(q, o, t) = \mathbb{I}(o) = \begin{cases} 1, & \text{the answer of } o \text{ is correct} \\ 0, & \text{the answer of } o \text{ is incorrect} \end{cases} \quad (10)$$

3、在线拒绝采样微调 (OnRFT)

拒绝采样微调 (RFT) 与在线拒绝采样微调之间唯一的区别在于，在线拒绝采样微调的输出是从实时策略模型 π_θ 中采样得到，而非从监督微调 (SFT) 模型 $\pi_{\theta_{sft}}$ 获取。因此，在线拒绝采样微调的梯度为：

$$\nabla_\theta \mathcal{J}_{OnRFT}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o \sim \pi_\theta(O|q)] \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \mathbb{I}(o) \nabla_\theta \log \pi_\theta(o_t | q, o_{<t}) \right). \quad (11)$$

4、直接偏好优化 (DPO)

DPO 的目标为：

$$\mathcal{J}_{DPO}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o^+, o^- \sim \pi_{sft}(O|q)] \log \sigma \left(\beta \frac{1}{|o^+|} \sum_{t=1}^{|o^+|} \log \frac{\pi_\theta(o_t^+ | q, o_{<t}^+)}{\pi_{ref}(o_t^+ | q, o_{<t}^+)} \right. \\ \left. - \beta \frac{1}{|o^-|} \sum_{t=1}^{|o^-|} \log \frac{\pi_\theta(o_t^- | q, o_{<t}^-)}{\pi_{ref}(o_t^- | q, o_{<t}^-)} \right)$$

$\mathcal{J}_{DPO}(\theta)$ 的梯度为：

$$\nabla_\theta \mathcal{J}_{DPO}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o^+, o^- \sim \pi_{sft}(O|q)] \left(\frac{1}{|o^+|} \sum_{t=1}^{|o^+|} GC_{DPO}(q, o, t) \nabla_\theta \log \pi_\theta(o_t^+ | q, o_{<t}^+) \right. \\ \left. - \frac{1}{|o^-|} \sum_{t=1}^{|o^-|} GC_{DPO}(q, o, t) \nabla_\theta \log \pi_\theta(o_t^- | q, o_{<t}^-) \right)$$

数据源：来自监督微调 (SFT) 数据集中的 question，其输出从 SFT 模型中采样得到。奖励函数：一般领域中的人类偏好（在数学任务中可以是“规则”）。梯度系数：

$$GC_{DPO}(q, o, t) = \sigma \left(\beta \log \frac{\pi_\theta(o_t^- | q, o_{<t}^-)}{\pi_{ref}(o_t^- | q, o_{<t}^-)} - \beta \log \frac{\pi_\theta(o_t^+ | q, o_{<t}^+)}{\pi_{ref}(o_t^+ | q, o_{<t}^+)} \right) \quad (14)$$

5、近端策略优化 (PPO)

PPO 的目标为：

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_\theta(o_t | q, o_{<t})}{\pi_{\theta_{old}}(o_t | q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_\theta(o_t | q, o_{<t})}{\pi_{\theta_{old}}(o_t | q, o_{<t})} \right) \right]$$

为简化分析，假设模型在每个探索阶段之后仅进行一次更新，从而确保 $\pi_{\theta_{old}} = \pi_{\theta}$ 。在这种情况下，可以去掉 $\min$ 和 clip 操作：

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t \quad (16)$$

$\mathcal{J}_{PPO}(\theta)$ 的梯度为：

$$\nabla_{\theta} \mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} A_t \nabla_{\theta} \log \pi_{\theta}(o_t|q, o_{<t}) \quad (17)$$

数据源：来自监督微调（SFT）数据集中的 question，其输出从策略模型中采样得到。奖励函数：奖励模型。梯度系数：

$$GC_{PPO}(q, o, t, \pi_{\theta_{rm}}) = A_t \quad (18)$$

其中 A_t 是优势值，通过应用广义优势估计（GAE），基于奖励 $r_{\geq t}$ 和一个学习到的价值函数 V_{ψ} 计算得出的。

6、群组相对策略优化（GRPO）

GRPO 的目标（为简化分析，假设 $\pi_{\theta_{old}} = \pi_{\theta}$ ）为：

$$\begin{aligned} \mathcal{J}_{GRPO}(\theta) = & \mathbb{E} \left[q \sim P_{sft}(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O | q) \right] \\ & \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left[\frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | q, o_{i,<t})} \hat{A}_{i,t} - \beta \left(\frac{\pi_{ref}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta}(o_{i,t} | q, o_{i,<t})} - \log \frac{\pi_r}{\pi} \right) \right] \end{aligned}$$

$\mathcal{J}_{GRPO}(\theta)$ 的梯度为：

$$\begin{aligned} \nabla_{\theta} \mathcal{J}_{GRPO}(\theta) = & \mathbb{E} \left[q \sim P_{sft}(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O | q) \right] \\ & \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left[\hat{A}_{i,t} + \beta \left(\frac{\pi_{ref}(o_{i,t} | o_{i,<t})}{\pi_{\theta}(o_{i,t} | o_{i,<t})} - 1 \right) \right] \nabla_{\theta} \log \pi_{\theta}(o_{i,t} | q, o_{i,<t}) \end{aligned}$$

数据源：来自监督微调（SFT）数据集中的问题，其输出从策略模型中采样得到。奖励函数：奖励模型。梯度系数：

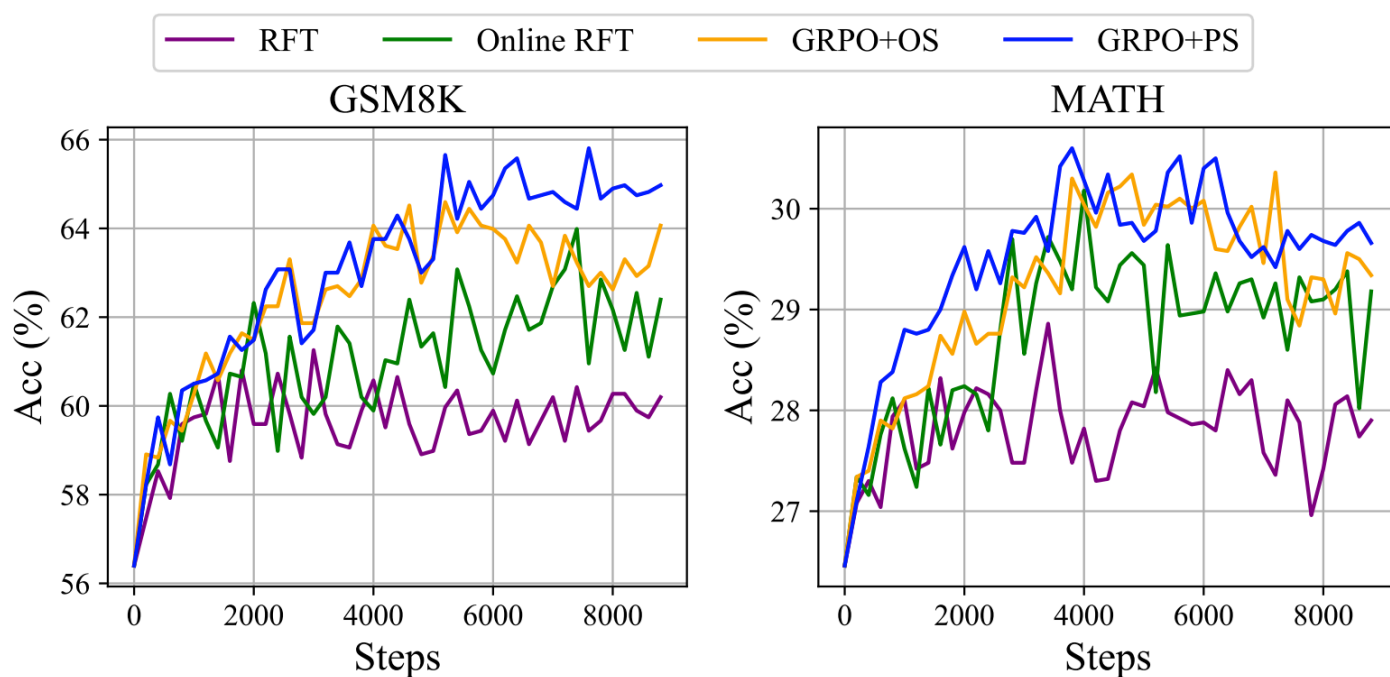
$$GC_{GRPO}(q, o, t, \pi_{\theta_{rm}}) = \hat{A}_{i,t} + \beta \left(\frac{\pi_{ref}(o_{i,t}|o_{i,<t})}{\pi_{\theta}(o_{i,t}|o_{i,<t})} - 1 \right) \quad (21)$$

其中 $\hat{A}_{i,t}$ 是基于分组奖励分数计算得出的。

1、关于数据源的观察

将数据源分为两类，在线采样和离线采样。在线采样意味着训练数据来自实时训练策略模型的探索结果，而离线采样表示训练数据来自初始 SFT 模型的采样结果。RFT 和 DPO 采用离线方式，而 Online RFT 和 GRPO 采用在线方式。

如图所示，在两个基准测试中，Online RFT 的表现显著优于 RFT。具体而言，在训练早期，Online RFT 与 RFT 表现相近，但在后期获得了绝对优势，这表明了在线训练的优越性。这是可以理解的，因为在初始阶段，策略模型（actor）与 SFT 模型非常相似，采样数据的差异很小。然而，在后期，从策略模型采样的数据将表现出更显著的差异，实时数据采样将具有更大的优势。



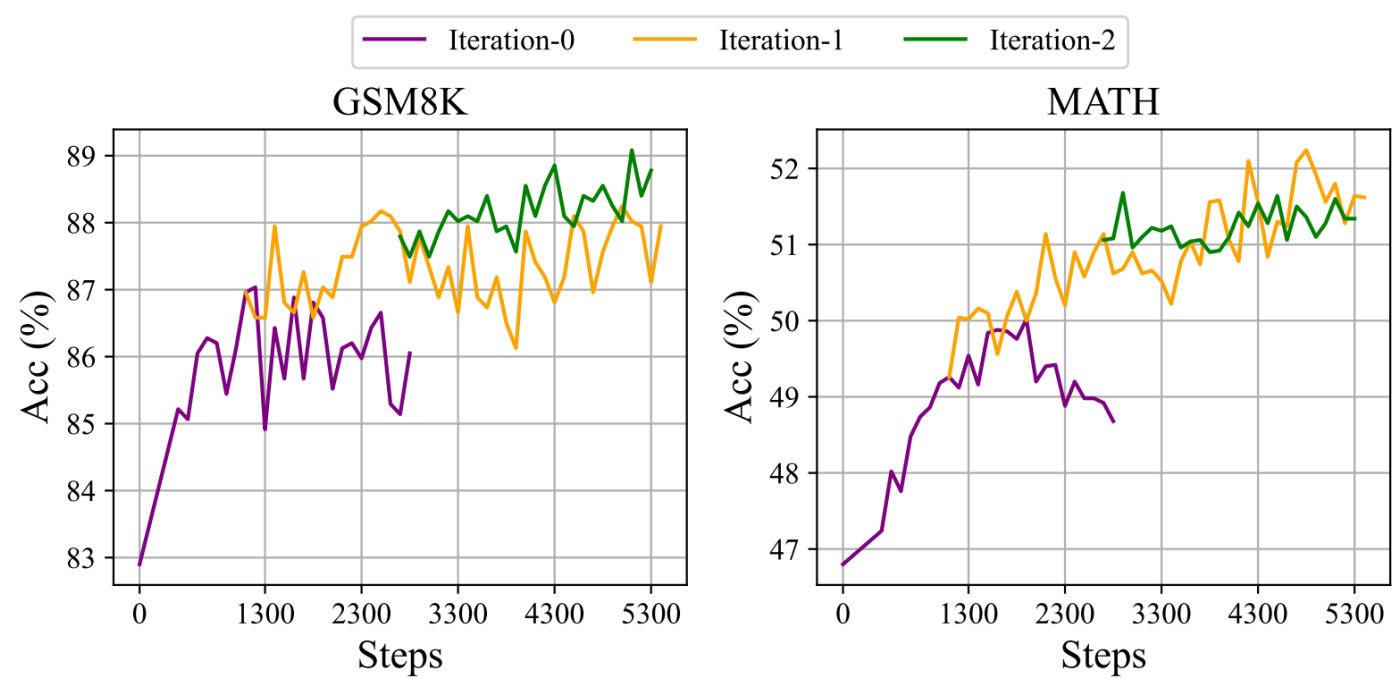
(/img/post/llm-model-66.png)

2、关于梯度系数的观察

算法将奖励信号处理为梯度系数以更新模型参数。DeepSeekMath 将奖励函数分为“规则 (Rule)”和“模型 (Model)”两类。“规则”是指根据答案的正确性来判断响应的质量，“模型”表示训练一个奖励模型来为每个响应打分。奖励模型的训练数据基于规则判断。公式 (10) 和 (21) 突出了 GRPO 和

Online RFT 之间的一个关键区别：GRPO 根据奖励模型提供的奖励值独特地调整其梯度系数。这允许根据响应的不同大小进行差异化的强化和惩罚。相比之下，Online RFT 缺乏此功能；它不对错误响应进行惩罚，并且对所有答案正确的响应以相同强度进行统一强化。

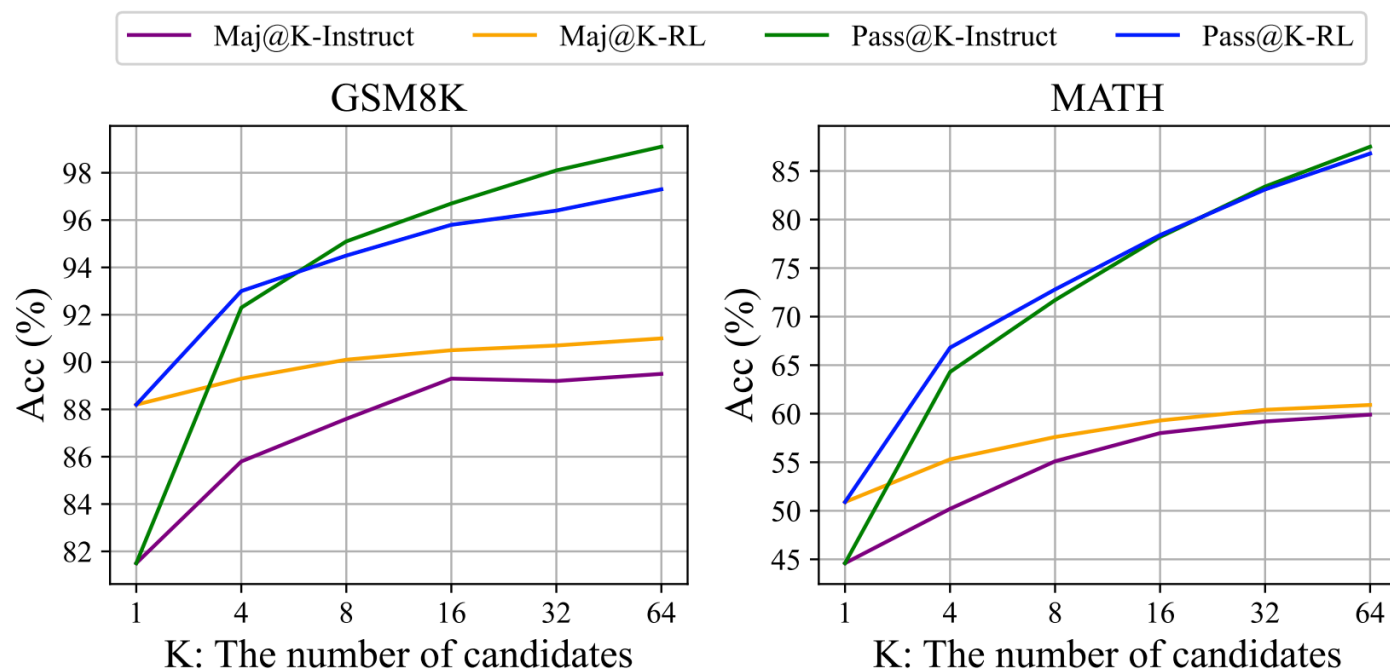
如图所示，GRPO 超过了 Online RFT，从而突出了改变正负梯度系数的有效性。此外，GRPO + PS 比 GRPO + OS 表现更优，这表明了使用细粒度、步骤感知的梯度系数的好处。此外，DeepSeekMath 还探索了迭代强化学习，在实验中进行了两轮迭代。如图，迭代强化学习显著提高了性能，尤其是在第一次迭代时。



(/img/post/llm-model-67.png)

9.2.4.2、为什么 RL 有效？

DeepSeekMath 基于指令微调数据的一个子集进行强化学习，这使得指令微调模型的性能得到显著提升。为进一步解释强化学习为何有效，DeepSeekMath 在两个基准测试上评估了指令模型（Instruct）和强化学习模型（RL）的K次通过率（Pass@K）和K次多数投票准确率（Maj@K）。如图所示，强化学习提升了 Maj@K 的性能，但未提升 Pass@K。这些发现表明，强化学习通过使输出分布更稳健来提升模型的整体性能，换言之，这种提升似乎源于提高 TopK 中的正确响应，而非模型基础能力的增强。类似研究发现监督微调（SFT）模型在推理任务中的不一致问题，表明通过一系列偏好对齐策略，SFT 模型的推理性能能够得到改善。



(/img/post/llm-model-68.png)

9.2.4.3、如何实现更有效的 RL?

DeepSeekMath 展示了强化学习 (RL) 在数学推理任务中表现得相当不错, 并且还提供了一个统一的范式来理解不同的代表性训练方法。在这个范式中, 所有方法都被概念化为直接的或简化的强化学习技术。如公式 (5) 所总结的, 存在三个关键组件: 数据源、算法和奖励函数。

1、数据源

数据源是所有训练方法的原材料。在强化学习的背景下, 特别指数据源为未标注的问题, 输出是从策略模型中采样的。在 DeepSeekMath 只使用了来自指令调优阶段的问题, 并采用了朴素的核采样来采样输出。这可能是 DeepSeekMath 的强化学习管道仅在 Maj@K 表现上有所提高的潜在原因。未来将探索在分布外问题提示上应用强化学习管道, 并结合基于树搜索方法的高级采样 (解码) 策略。此外, 决定策略模型探索效率的高效推理技术也起着至关重要的作用。

2、算法

算法处理数据和奖励信号，通过梯度系数来更新模型参数。基于公式 (5)，从某种程度上讲，所有方法现在完全信任奖励函数的信号来增加或减少某个 token 的条件概率。然而，无法保证奖励信号始终可靠，尤其是在极其复杂的任务中。例如，即使是经过精心标注的 PRM800K 数据集，其中也仍然包含大约 20% 的错误标注。为此将探索对噪声奖励信号具有鲁棒性的强化学习算法，这种弱到强的对齐方法将对学习算法带来根本性的变化。

3、奖励函数

奖励函数是训练信号的来源。在强化学习中，奖励函数通常是神经奖励模型。奖励模型存在三个重要的方向：

1. 如何增强奖励模型的泛化能力。奖励模型必须有效地泛化，以处理分布外问题和高级解码输出；否则，强化学习可能只是稳定 LLMs 的分布，而不是提高其基本能力
2. 如何反映奖励模型的不确定性。不确定性可能作为弱奖励模型和弱到强学习算法之间的桥梁
3. 如何高效地构建高质量的过程奖励模型，能够为推理过程提供细粒度的训练信号

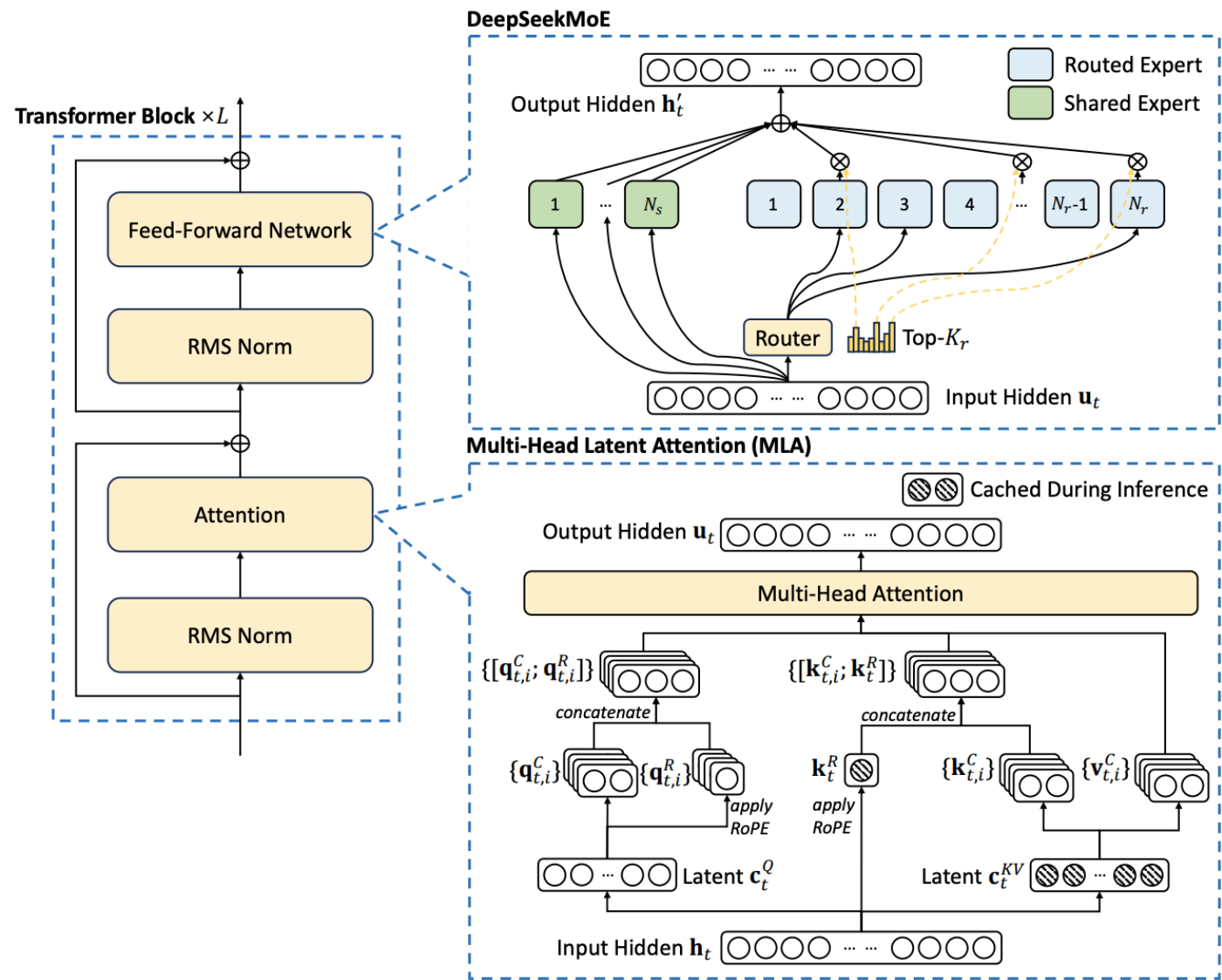
9.3、DeepSeek-V2

DeepSeek-V2 发布于 2024 年 5 月，为多领域专家（MoE）语言模型，包含总共 2360 亿个参数，其中每个词元激活 210 亿个参数，并支持 12.8 万个词元的上下文长度。DeepSeek-V2 采用包括多头潜在注意力（Multi-Head Latent Attention, MLA）和 DeepSeekMoE 在内的创新架构。MLA 通过将键值（KV）缓存显著压缩为一个潜在向量，保证了高效推理；而 DeepSeekMoE 则通过稀疏计算，能以较低成本训练出强大的模型。与 DeepSeek 670 亿参数模型相比，DeepSeek-V2 的性能显著提升，同时节省了 42.5% 的训练成本，将 KV 缓存减少了 93.3%，并将最大生成吞吐量提高到了 5.76 倍。

DeepSeek - V2 在一个由 8.1 万亿词元组成的高质量、多源语料库上进行预训练，然后进一步进行监督微调（SFT）和强化学习（RL）。即使只有 210 亿激活参数，DeepSeek-V2 及其聊天版本在开源模型中仍能达到顶尖性能。

9.3.1、架构

DeepSeek-V2 采用 Transformer 架构，其中每个 Transformer 模块由一个注意力模块和一个前馈网络（FFN）组成。在注意力方面，DeepSeek-V2 设计了 MLA，其利用低秩键值联合压缩来消除推理时键值缓存的瓶颈，从而支持高效推理。对于前馈网络，采用 DeepSeekMoE 架构，能够以经济的成本训练强大的模型。



(/img/post/llm-model-48.png)

9.3.1.1、多头潜在注意力：提升推理效率

传统的 Transformer 模型通常采用多头注意力机制（MHA），但在生成过程中，其庞大的键值（KV）缓存会成为限制推理效率的瓶颈。为了减少 KV 缓存，多查询注意力机制（MQA）和分组查询注意力机制（GQA）被提出。它们所需的 KV 缓存规模较小，但其性能却比不上 MHA。

DeepSeek-V2 设计了一种创新的注意力机制，称为 **多头潜在注意力（Multi-Head Latent Attention, MLA）**。通过低秩键值联合压缩，MLA 实现了比 MHA 更好的性能，同时所需的 KV 缓存量显著减少。

预备知识：标准多头注意力机制

设 d 为嵌入（embedding）维度， n_h 为注意力头（attention heads）的数量， d_h 为每个头（head）的维度，并且 $h_t \in \mathbb{R}^d$ 是注意力层中第 t 个词元（token）的注意力输入。标准的 MHA 首先通过三个矩阵 W_Q 、 W_K 、 $W_V \in \mathbb{R}^{d_h n_h \times d}$ 分别生成 q_t 、 k_t 、 $v_t \in \mathbb{R}^{d_h n_h}$ ：

$$\begin{aligned} q_t &= W^Q h_t, \\ k_t &= W^K h_t, \\ v_t &= W^V h_t \end{aligned}$$

然后， q_t 、 k_t 、 v_t 将被切分成 n_h 个头，以进行多头注意力计算：

$$\begin{aligned} [q_{t,1}; q_{t,2}; \dots; q_{t,n_h}] &= q_t, \\ [k_{t,1}; k_{t,2}; \dots; k_{t,n_h}] &= k_t, \\ [v_{t,1}; v_{t,2}; \dots; v_{t,n_h}] &= v_t, \\ o_{t,i} &= \sum_{j=1}^t \text{Softmax}_j \left(\frac{q_{t,i}^T k_{j,i}}{\sqrt{d_h}} \right) v_{j,i}, \\ u_t &= W^O [o_{t,1}; o_{t,2}; \dots; o_{t,n_h}] \end{aligned}$$

其中 $q_{t,i}$ 、 $k_{t,i}$ 、 $v_{t,i} \in \mathbb{R}^{d_h}$ 分别表示第 i 个注意力头的 query、key 和 value； $W_O \in \mathbb{R}^{d \times d_h n_h}$ 表示输出投影矩阵。在推理过程中，所有的 query 和 value 都需要被缓存以加速推理，所以对于每个词元（token），MHA 需要缓存 $2n_h d_h l$ 个元素（ l 为注意力层数）。在模型部署中，这种庞大的键值缓存是一个很大的瓶颈，它限制了最大批量大小和序列长度。

低秩键值联合压缩

多头潜在注意力（MLA）的核心是对键值进行低秩联合压缩，以减少键值（KV）缓存：

$$\begin{aligned} c_t^{KV} &= W^{DKV} h_t, \\ k_t^C &= W^{UK} c_t^{KV}, \\ v_t^C &= W^{UV} c_t^{KV}, \end{aligned}$$

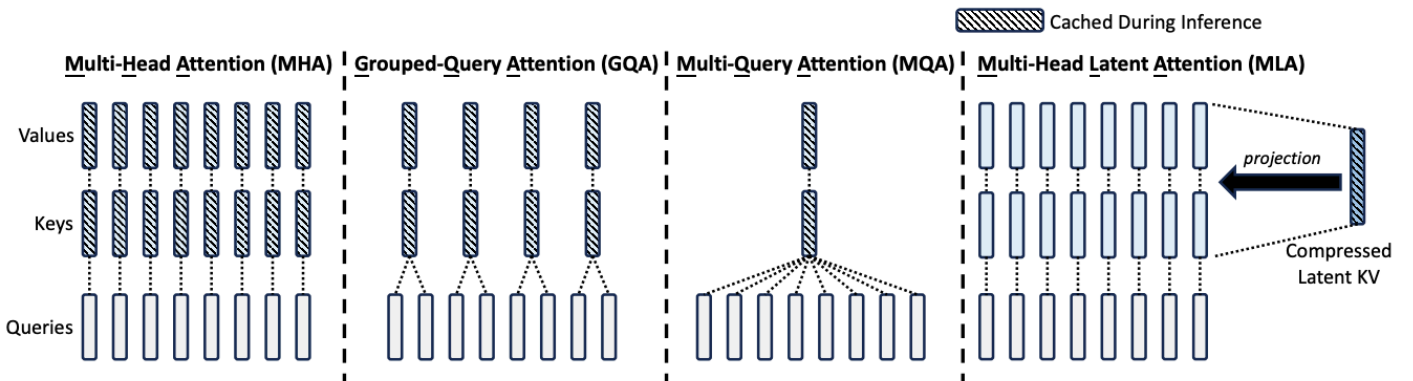
其中， $c_t^{KV} \in \mathbb{R}^{d_c}$ 是 key 和 value 的压缩潜在向量； $d_c (\ll d_h n_h)$ 表示 KV 压缩维度； $W^{DKV} \in \mathbb{R}^{d_c \times d}$ 是下投影矩阵； $W^{UK}, W^{UV} \in \mathbb{R}^{d_h n_h \times d_c}$ 分别是 key 和 value 的上投影矩阵。在推理过程中，MLA 只需要缓存 c_t^{KV} ，因此其 KV 缓存只有 $d_c l$ 个元素，其中 l 表示层数。

此外，在推理过程中，由于 W^{UK} 可以合并到 W^Q 中， W^{UV} 可以合并到 W^O 中，即甚至无需为注意力计算出键值。

而且，为了在训练过程中减少激活（activation）内存，MLA 对 query 进行低秩压缩，即使不能减少 KV 缓存：

$$\begin{aligned} c_t^Q &= W^{DQ} h_t, \\ q_t^Q &= W^{UQ} c_t^Q \end{aligned}$$

其中， $c_t^Q \in \mathbb{R}^{d'_c}$ 是 query 的压缩潜在向量； $d'_c (\ll d_h n_h)$ 表示 query 压缩维度； $W^{DQ} \in \mathbb{R}^{d'_c \times d}$ 和 $W^{UQ} \in \mathbb{R}^{d_h n_h \times d'_c}$ 分别是 query 的下投影矩阵和上投影矩阵。



(/img/post/llm-model-49.png)

解耦旋转位置嵌入

DeepSeek-V2 中使用旋转位置嵌入 (RoPE)，然而，RoPE 与低秩键值 (KV) 压缩不兼容。具体而言，RoPE 对 KV 都具有位置敏感性。如果对 key k_t^C 应用 RoPE，方程 $k_t^C = W^{UK} c_t^{KV}$ 中的 W^{UK} 将与位置敏感的 RoPE 矩阵耦合。这样一来，在推理过程中， W^{UK} 就无法再被吸收到 W^Q 中，因为与当前生成的词元相关的 RoPE 矩阵会位于 W^Q 和 W^{UK} 之间，而矩阵乘法不满足交换律。这样所造成的结果是，在推理过程中必须重新计算所有前缀词元的 key，这将显著降低推理效率。

如何理解：RoPE 与低秩键值 (KV) 压缩不兼容？

1、第 t 个词元的 query 向量与第 j 个词元的 key 向量之间的计算如下：

$$\begin{aligned} q_{t,i}^T k_{j,i} &= (W_i^{UQ} c_t^Q)^T (W_i^{UK} c_j^{KV}) \\ &= (c_t^Q)^T (W_i^{UQ})^T W_i^{UK} c_j^{KV} \end{aligned}$$

其中， W_i^{UQ} 是用于生成第 i 个注意力头 query 的矩阵， c_t^Q 是与第 t 个词元的 query 的压缩潜在向量； W_i^{UK} 是用于生成第 i 个注意力头 key 的矩阵， c_j^{KV} 是与第 j 个词元的 value 的压缩潜在向量。

由于矩阵乘法满足结合律，可以先计算 $(W_i^{UQ})^T W_i^{UK}$ 作为 query 的投影矩阵，如此只需缓存 c_j^{KV} ，而非 $(W_i^{UK} c_j^{KV})$ ，从而在推理的时候压缩 KV 缓存。

2、加入 RoPE 后，会在 $(W_i^{UQ})^T W_i^{UK}$ 之间增加融合相对位置的变量 $\mathcal{R}_{t-j}$ ：

$$\begin{aligned} q_{t,i}^T k_{j,i} &= (\mathcal{R}_t W_i^{UQ} c_t^Q)^T \mathcal{R}_j (W_i^{UK} c_j^{KV}) \\ &= (c_t^Q)^T (W_i^{UQ})^T \mathcal{R}_t^T \mathcal{R}_j W_i^{UK} c_j^{KV} \\ &= (c_t^Q)^T (W_i^{UQ})^T \mathcal{R}_{t-j} W_i^{UK} c_j^{KV} \end{aligned}$$

由于 $(W_i^{UQ})^T \mathcal{R}_{t-j} W_i^{UK}$ 随相对位置变化，无法合并为一个固定的投影矩阵，因此不能提前计算以压缩 KV 缓存。

相关文章解读：

1. 缓存与效果的极限拉扯：从MHA、MQA、GQA到MLA

(<https://spaces.ac.cn/archives/10091>)

2. deepseek技术解读(1)-彻底理解MLA (Multi-Head Latent Attention)

(<https://zhuanlan.zhihu.com/p/16730036197>)

作为一种解决方案，DeepSeek-V2 提出解耦 RoPE 策略，即使用额外的多头查询 $q_{t,i}^R \in \mathbb{R}^{d_h^R}$ 和一个共享 key $k_t^R \in \mathbb{R}^{d_h^R}$ 来承载 RoPE，其中 d_h^R 表示解耦后的 query 和 key 的每头维度。采用解耦 RoPE 策略后，MLA 进行以下计算：

$$\begin{aligned}
[q_{t,1}^R; q_{t,2}^R; \dots; q_{t,n_h}^R] &= q_t^R = \text{RoPE}(W^{QR} c_t^Q), \\
k_t^R &= \text{RoPE}(W^{KR} h_t), \\
q_{t,i} &= [q_{t,i}^C; q_{t,i}^R], \\
k_{t,i} &= [k_{t,i}^C; k_t^R], \\
o_{t,i} &= \sum_{j=1}^t \text{Softmax}_j \left(\frac{q_{t,i}^T k_{j,i}}{\sqrt{d_h + d_h^R}} \right) v_{j,i}^C, \\
u_t &= W_O[o_{t,1}; o_{t,2}; \dots; o_{t,n_h}]
\end{aligned}$$

其中, $W_Q^R \in \mathbb{R}^{d_h^R n_h \times d_c'}$ 和 $W_K^R \in \mathbb{R}^{d_h^R \times d}$ 分别是用于生成解耦 query 和 key 的矩阵; $\text{RoPE}(\cdot)$ 表示应用 RoPE 矩阵的操作; $[\cdot; \cdot]$ 表示拼接操作。在推理过程中, 解耦后的 key 也应该被缓存。因此, DeepSeek-V2 总共需要一个包含 $(d_c + d_h^R)l$ 个元素的 KV 缓存。

解耦 RoPE

引入多头查询 $q_{t,i}^R \in \mathbb{R}^{d_h^R}$ 和共享 key $k_t^R \in \mathbb{R}^{d_h^R}$ 后, 第 t 个词元的 query 向量与第 j 个词元的 key 向量之间的计算如下:

$$\begin{aligned}
q_{t,i}^T k_{j,i} &= [q_{t,i}^C; q_{t,i}^R]^T [k_{j,i}^C; k_t^R] \\
&= q_{t,i}^C k_{j,i}^C + q_{t,i}^R k_t^R
\end{aligned}$$

其中, 由于矩阵乘法的结合律, $q_{t,i}^C k_{j,i}^C$ 中, W^{UK} 可被 W^{UQ} 吸收 (同理, W^{UV} 可被 W^O 吸收), 只需缓存 c_t^{KV} 。 $q_{t,i}^R k_t^R$ 可按 MQA 方式计算, 只需缓存共享的 k_t^R 。最终, 避免了在推理过程中重新计算 k_t^C 和 v_t^C 带来的计算开销。

Key-Value 缓存比较

下表中展示了不同注意力机制下每个词元的键值 (KV) 缓存对比情况。多层注意力 (MLA) 仅需少量的 KV 缓存, 与仅有 2.25 组的 GQA 相当, 但 MLA 的性能比 MHA 更强。

Attention Mechanism	KV Cache per Token (# Element)	Capability
Multi-Head Attention (MHA)	$2n_h d_h l$	Strong
Grouped-Query Attention (GQA)	$2n_g d_h l$	Moderate
Multi-Query Attention (MQA)	$2d_h l$	Weak
MLA (Ours)	$(d_c + d_h^R)l \approx \frac{9}{2}d_h l$	Stronger

(/img/post/llm-model-50.png)

其中， n_h 表示注意力头的数量， d_h 表示每个注意力头的维度， l 表示层数， n_g 表示 GQA 中的组数， d_c 和 d_h^R 分别表示 MLA 中的 KV 压缩维度以及解耦 query 和 key 的每个头的维度。KV 缓存量通过元素数量衡量，不考虑存储精度。对于 DeepSeek-V2， d_c 设置为 $4d_h$ ， d_h^R 设置为 $\frac{d_h}{2}$ 。因此，其 KV 缓存量与仅分为 2.25 组的 GQA 相当，但其性能比 MHA 更强。

9.3.1.2、DeepSeekMoE

基本架构

对于前馈神经网络（FFNs），DeepSeek-V2 采用 DeepSeekMoE 架构。DeepSeekMoE 有两个关键理念：

1. 将专家模块划分得更细粒度，以实现更高层次的专家专业化和更精准的知识获取；
2. 分离出一些共享专家，以减少路由专家之间的知识冗余。

在激活的专家参数和专家总参数数量相同的情况下，DeepSeekMoE 的性能大幅优于诸如 GShard 等传统的混合专家（MoE）架构。

设 u_t 为第 t 个词元的前馈神经网络输入，按如下方式计算前馈神经网络的输出 h'_t ：

$$\begin{aligned}
 h'_t &= u_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(u_t) + \sum_{i=1}^{N_r} g_{i,t} \text{FFN}_i^{(r)}(u_t), \\
 g_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r) \\ 0, & \text{otherwise} \end{cases}, \\
 s_{i,t} &= \text{Softmax}_i(u_t^T e_i)
 \end{aligned}$$

其中：

- N_s 和 N_r 分别表示共享专家和路由专家的数量
- $\text{FFN}_i^{(s)}(\cdot)$ 和 $\text{FFN}_i^{(r)}(\cdot)$ 分别表示第 i 个共享专家和第 i 个路由专家
- K_r 表示激活的路由专家数量
- $g_{i,t}$ 是第 i 个专家的门控值
- $s_{i,t}$ 是词元与专家的亲和度
- e_i 是该层第 i 个路由专家的质心
- $\text{Topk}(\cdot, K)$ 表示在为第 t 个词元与所有路由专家计算出的亲和度分数中，由 K 个最高分数组成的集合

设备受限路由

通过设备受限路由机制以控制与混合专家（MoE）相关的通信成本。当采用专家并行时，被路由的专家会分布在多个设备上。对于每个词元，其与 MoE 相关的通信频率与目标专家所覆盖的设备数量成正比。由于 DeepSeekMoE 中专家的细粒度划分，激活的专家数量可能较多，因此如果采用专家并行，与 MoE 相关的通信成本会更高。

对于 DeepSeek-V2，除了对路由专家进行常规的 top-K 选择之外，还额外确保每个词元的目标专家最多分布在 M 个设备上。具体来说，对于每个词元，首先选择 M 个其中专家亲和度分数最高的设备。然后，在这 M 个设备上的专家中进行 top-K 选择。在实践中，当 $M \geq 3$ 时，设备受限路由能够实现与无限制 top-K 路由大致相当的良好性能。

负载均衡的辅助损失

对于自动学习的路由策略，DeepSeek-V2 考虑了负载均衡因素。首先，负载不均衡会增加路由崩溃的风险，导致部分专家无法得到充分训练和利用。其次，在采用专家并行时，负载不均衡会降低计算效率。在 DeepSeek-V2 的训练过程中设计了三种辅助损失，分别用于控制专家级负载均衡、设备级负载均衡和通信均衡。

1、专家级均衡损失

DeepSeek-V2 采用一种专家级均衡损失来降低路由崩溃的风险：

$$\begin{aligned}\mathcal{L}_{\text{ExpBal}} &= \alpha_1 \sum_{i=1}^{N_r} f_i P_i, \\ f_i &= \frac{N_r}{K_r T} \sum_{t=1}^T \mathbb{1}(\text{Token } t \text{ selects Expert } i), \\ P_i &= \frac{1}{T} \sum_{t=1}^T s_{i,t}\end{aligned}$$

其中， α_1 是一个称为专家级均衡因子的超参数； $\mathbb{1}(\cdot)$ 表示指示函数； T 表示序列中的词元数量。

2、设备级均衡损失

除了专家级均衡损失，还额外设计了设备级均衡损失，以确保不同设备间的计算负载均衡。在 DeepSeek-V2 的训练过程中，将所有路由专家划分为 D 组 $E_1, E_2, \dots, E_D$ ，并将每组部署在单个设备上。设备级均衡损失的计算方式如下：

$$\begin{aligned}\mathcal{L}_{\text{DevBal}} &= \alpha_2 \sum_{i=1}^D f'_i P'_i, \\ f'_i &= \frac{1}{|\mathcal{E}_i|} \sum_{j \in \mathcal{E}_i} f_j, \\ P'_i &= \sum_{j \in \mathcal{E}_i} P_j\end{aligned}$$

其中， α_2 是一个称为设备级均衡因子的超参数。

3、通信均衡损失

最后，引入通信均衡损失，以确保每个设备的通信负载均衡。尽管设备受限路由机制保证了每个设备的发送通信量受到限制，但如果某个设备接收的词元数量多于其他设备，实际通信效率仍会受到影响。为缓解这一问题，设计如下通信均衡损失：

$$\mathcal{L}_{\text{CommBal}} = \alpha_3 \sum_{i=1}^D f_i'' P_i'',$$

$$f_i'' = \frac{D}{MT} \sum_{t=1}^T \mathbb{1}(\text{Token } t \text{ is sent to Device } i),$$

$$P_i'' = \sum_{j \in \mathcal{E}_i} P_j$$

其中， α_3 是一个称为通信均衡因子的超参数。设备受限路由机制的运行原则是确保每个设备最多向其他设备传输 MT 个隐藏状态。同时，采用通信均衡损失来促使每个设备从其他设备接收约 MT 个隐藏状态。通信均衡损失保证了设备之间信息的均衡交换，有助于实现高效通信。

词元丢弃策略

尽管均衡损失旨在促进负载均衡，但必须承认，它们无法保证严格的负载均衡。为了进一步减少因负载不均衡导致的计算浪费，DeepSeek-V2 在训练过程中引入设备级词元丢弃策略：首先计算每个设备的平均计算预算，即每个设备的容量因子等同于 1.0。然后，在每个设备上丢弃亲和度分数最低的词元，直至达到计算预算。此外，确保大约 10% 的训练序列中的词元永远不会被丢弃。通过这种方式，可以在推理过程中根据效率要求灵活决定是否丢弃词元，并且始终保证训练和推理之间的一致性。

9.3.2、预训练

9.3.2.1、数据构建

在保持与 DeepSeek 67B 相同的数据处理阶段的同时，DeepSeek-V2 扩充了数据量并提升了数据质量。为了扩大预训练语料库，DeepSeek-V2 探索了互联网数据的潜力并优化了清洗流程，从而恢复了大量被误删除的数据。此外，纳入了更多中文数据，旨在更好地利用中文互联网上可用的语料库。

除了数据量，DeepSeek-V2 还关注数据质量。用来自各种来源的高质量数据丰富预训练语料库，同时改进基于质量的过滤算法。改进后的算法确保大量无用数据被剔除，而有价值的数据大多得以保留。此外，从预训练语料库中过滤掉有争议的内容，以减轻特定地域文化带来的数据偏差。

DeepSeek-V2 采用与 DeepSeek 67B 相同的分词器，该分词器基于字节级字节对编码（BBPE）算法构建，词汇表大小为 10 万个。经过分词的预训练语料库包含 8.1 万亿个词元，其中中文词元比英文词元大约多 12%。

9.3.2.2、超参数

1、模型超参数

DeepSeek-V2 将 Transformer 层数设置为 60，隐藏层维度设置为 5120。所有可学习参数均以标准差 0.006 进行随机初始化。在多层注意力（MLA）中，将注意力头的数量 n_h 设置为 128，每个头的维度 d_h 设置为 128。KV 压缩维度 d_c 设置为 512，query 压缩维度 d'_c 设置为 1536。对于解耦后的 query 和 key，将每个头的维度 d_h^R 设置为 64。

DeepSeek-V2 将除第一层之外的所有前馈神经网络（FFNs）替换为混合专家（MoE）层。每个 MoE 层由 2 个共享专家和 160 个路由专家组成，每个专家的中间隐藏层维度为 1536。在路由专家中，每个词元将激活 6 个专家。此外，低秩压缩和细粒度的专家划分会影响一层的输出规模。因此，在实际操作中，在压缩后的潜在向量之后采用额外的均方根归一化（RMS Norm）层，并在宽度瓶颈处（即压缩后的潜在向量和路由专家的中间隐藏状态）乘以额外的缩放因子，以确保训练的稳定性。在此配置下，DeepSeek-V2 总共有 2360 亿个参数，每个词元激活 210 亿个参数。

2、训练超参数

DeepSeek-V2 采用 AdamW 优化器，超参数设置为 $\beta_1 = 0.9$ ， $\beta_2 = 0.95$ ，权重衰减为 0.1。学习率采用预热与阶梯衰减（warmup-and-step-deca）策略进行调度。最初，在前 2000 步中，学习率从 0 线性增加到最大值。随后，在训练约 60% 的词元后，学习率乘以 0.316，在训练约 90% 的词元后，再次乘以 0.316。最大学习率设置为 2.4×10^{-4} ，梯度裁剪范数设置为 1.0。

DeepSeek-V2 还使用批量大小调度策略，在训练前 2250 亿个词元时，批量大小从 2304 逐渐增加到 9216，在剩余训练过程中保持为 9216。此外，将最大序列长度设置为 4096，并在 8.1 万亿个词元上训练 DeepSeek-V2。

DeepSeek-V2 利用流水线并行技术将模型的不同层部署在不同设备上，对于每一层，路由专家将均匀部署在 8 个设备上（ $D = 8$ ）。对于设备受限路由，每个词元最多发送到 3 个设备（ $M = 3$ ）。对于均衡损失，将 α_1 设置为 0.003， α_2 设置为 0.05， α_3 设置为 0.02。在训练期间采用词元丢弃策略

以加速训练，但在评估时不丢弃任何词元。

9.3.2.3、基础设施

DeepSeek-V2 基于自主开发的高效轻量级训练框架 HAI-LLM 框架进行训练。该框架采用 16 路零气泡流水线并行、8 路专家并行以及 ZeRO-1 数据并行。鉴于 DeepSeek-V2 的激活参数相对较少，且部分操作会重新计算以节省激活内存，因此无需张量并行即可完成训练，从而降低了通信开销。此外，为进一步提高训练效率，将共享专家的计算与专家并行的全对全通信进行重叠处理。

DeepSeek-V2 针对不同专家间的通信、路由算法以及融合线性计算，定制了更快的 CUDA 内核。此外，多层注意力（MLA）也基于改进版的 FlashAttention-2 进行了优化。

所有的实验均在配备 NVIDIA H800 GPU 的集群上进行。H800 集群中的每个节点包含 8 个通过 NVLink 和节点内 NVSwitch 相连的 GPU。跨节点之间，则利用 InfiniBand 互连来实现通信。

9.3.2.4、长上下文扩展

在对 DeepSeek-V2 进行初始预训练后，采用 YaRN 将默认上下文窗口长度从 4K 扩展到 128K。YaRN 专门应用于解耦后的共享键 k_t^R ，因为它负责承载旋转位置嵌入（RoPE）。对于 YaRN，将缩放因子 s 设置为 40， α 设置为 1， β 设置为 32，目标最大上下文长度设置为 160K。在这些设置下，期望模型在 128K 的上下文长度下能有良好表现。由于 DeepSeek-V2 独特的注意力机制，与原始 YaRN 略有不同，DeepSeek-V2 调整了长度缩放因子来调节注意力熵。因子 $\sqrt{t}$ 计算为 $\sqrt{t} = 0.0707 \ln s + 1$ ，目的是使困惑度最小化。

DeepSeek-V2 对模型进行了 1000 步的训练，序列长度设为 32K，批量大小为 576 个序列。尽管训练仅在 32K 的序列长度下进行，但当在 128K 的上下文长度下进行评估时，该模型仍展现出强大的性能。

YaRN

YaRN (Yet another RoPE extensioN method) 是一种用于扩展基于旋转位置嵌入 (RoPE) 的大型语言模型上下文窗口的高效方法, 所需的训练数据和训练步骤分别比之前的方法少10倍和2.5倍。通过对 LLaMA 使用 YaRN 能够有效利用并外推到比其原始预训练所允许的更长的上下文长度, 同时在上下文窗口扩展方面超越了之前的最先进技术。此外, YaRN 具备在微调数据集有限上下文之外进行外推的能力。

1、旋转位置嵌入

YaRN 基于 RoPE, 在一个隐藏层上工作, 其中隐藏神经元的集合由 D 表示。给定一个向量序列 $x_1, \dots, x_L \in \mathbb{R}^{|D|}$, 注意力层首先将这些向量转换为查询向量和键向量:

$$q_m = f_q(x_m, m) \in \mathbb{R}^{|D|}, k_n = f_k(x_n, n) \in \mathbb{R}^{|D|}$$

接下来, 计算注意力权重为

$$\text{softmax} \left(\frac{q_m^T k_n}{\sqrt{|D|}} \right)$$

其中 q_m, k_n 被视为列向量, 因此 $q_m^T k_n$ 简单地是欧几里得内积。在 RoPE 中, 假设 $|D|$ 是偶数, 并将嵌入空间和隐藏状态识别为复向量空间:

$$\mathbb{R}^{|D|} \cong \mathbb{C}^{|D|/2}$$

其中内积 $q^T k$ 变为标准厄米内积的实部 $\text{Re}(q^* k)$ 。更具体的, 同构交错实部和虚部:

$$\begin{aligned} ((x_m)_1, \dots, (x_m)_{|D|}) &\mapsto ((x_m)_1 + i(x_m)_2, \dots, ((x_m)_{|D|-1} + i(x_m)_{|D|})), \\ ((q_m)_1, \dots, (q_m)_{|D|}) &\mapsto ((q_m)_1 + i(q_m)_2, \dots, ((q_m)_{|D|-1} + i(q_m)_{|D|})). \end{aligned}$$

为了将嵌入 x_m, x_n 转换为查询和键向量, 首先需要定义 $\mathbb{R}$ 线性算子:

$$W_q, W_k : \mathbb{R}^{|D|} \rightarrow \mathbb{R}^{|D|}.$$

在复坐标中, 函数 f_q, f_k 由下式给出:

$$f_q(x_m, m) = e^{im\theta} W_q x_m, f_k(x_n, n) = e^{in\theta} W_k x_n,$$

其中 $\theta = \text{diag}(\theta_1, \dots, \theta_{\lfloor D/2 \rfloor})$ 是对角矩阵，且 $\theta_d = b^{-2d/\lfloor D \rfloor}$ ， $b = 10000$ 。这样，RoPE 将每个（复数值）隐藏神经元与一个单独的频率 θ_d 关联起来。这样做的好处是查询（query）向量和键（key）向量之间的点积仅取决于相对距离 $m - n$ ，如下所示：

$$\begin{aligned}\langle f_q(x_m, m), f_k(x_n, n) \rangle_{\mathbb{R}} &= \text{Re}(\langle f_q(x_m, m), f_k(x_n, n) \rangle_{\mathbb{C}}) \\ &= \text{Re}(x_m^* W_q^* W_k x_n e^{i\theta(m-n)}) \\ &= g(x_m, x_n, m - n).\end{aligned}$$

在实坐标中，RoPE 可以用以下函数写成：

$$f_W(x_m, m, \theta_d) = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_l & -\sin m\theta_l \\ 0 & 0 & 0 & 0 & \dots & \sin m\theta_l & \cos m\theta_l \end{pmatrix} \mathbf{v}$$

即：

$$f_q = f_{W_q}, f_k = f_{W_k}.$$

注：

在 RoPE 里，每个维度 d 有对应旋转角度 θ_d ，结合位置索引 m 得到旋转角度 $m\theta_d$ 。从数学实现看，此旋转操作类似在复平面旋转位置向量。类比信号处理，不同的 $m\theta_d$ 使位置向量在各维度以不同“速度”旋转，类似不同频率。大的 θ_d 使向量旋转快，对应高频；小的 θ_d 使向量旋转慢，对应低频。

快速旋转能够捕捉输入序列中的局部细节和短期依赖关系，因为这些变化通常较为频繁且快速。慢速旋转适合捕捉长距离依赖关系，因为这些关系通常变化较慢且需要更长的上下文信息。通过这种设计，模型能够在不同维度上同时捕捉局部细节和长距离依赖。

2、位置插值（PI）

由于语言模型通常在固定的上下文长度下进行预训练，因此很自然会提出这样一个问题：如何通过相对少量的数据微调来扩展上下文长度。

对于使用 RoPE 作为位置嵌入的语言模型，有研究提出位置插值（PI）来扩展超出预训练限制的上下文长度。虽然直接外推在序列 $w_1, \dots, w_L$ （ L 大于预训练极限）中表现不佳，但在少量微调的帮助下，对预训练限制内的位置索引进行插值能取得良好效果。具体来说，给定一个使用 RoPE 的预训练语言模型：

$$f'_W(x_m, m, \theta_d) = f_W\left(x_m, \frac{mL}{L'}, \theta_d\right), \quad (1)$$

其中 $L' > L$ 是超出预训练限制的新上下文窗口。通过原始预训练模型加上修改后的 RoPE 公式，在数量少几个数量级的词元上进一步微调了语言模型，并成功实现了上下文窗口的扩展。

3、额外的符号说明

扩展的上下文长度与原始上下文长度之间的比率特别重要，引入符号 s （比例因子）定义为：

$$s = \frac{L'}{L},$$

重写并简化方程 (1) 为以下一般形式：

$$f'_W(x_m, m, \theta_d) = f_W(x_m, g(m), h(\theta_d)), \quad (2)$$

其中 $g(m), h(\theta_d)$ 是方法相关的函数。对于 PI，有 $g(m) = m/s, h(\theta_d) = \theta_d$ 。

此外，定义 λ_d 为 RoPE 嵌入在第 d 个隐藏维度上的波长：

$$\lambda_d = \frac{2\pi}{\theta_d} = 2\pi b^{\frac{2d}{|D|}}. \quad (3)$$

波长描述了 RoPE 嵌入在第 d 个维度上执行一次完整旋转 (2π) 所需的词元长度。

鉴于一些插值方法（例如 PI）不关心维度的波长，将这些方法称为“盲目”的插值方法，而其他方法（例如 YaRN）则将其分类为“目标”插值方法。

4、高频信息损失 - “NTK感知”插值法

PI 尝试通过重新定义 $g(m)$ 将位置索引拉伸到预训练窗口内，PI 所描述的理论插值界限不足以预测 RoPE 与大语言模型 (LLM) 内部嵌入之间的复杂动态关系。

如果仅从信息编码的角度来看 RoPE，利用神经正切核 (NTK) 理论表明，若输入维度较低且相应的嵌入缺乏高频成分，深度神经网络在学习高频信息时会遇到困难。一个词元的位置信息是一维的，而 RoPE 将其扩展为一个 n 维的复数向量嵌入。

在许多方面，RoPE 与傅里叶特征极为相似，因为可以将 RoPE 定义为傅里叶特征的一种特殊一维情况。不加区分地拉伸 RoPE 嵌入会导致重要高频细节的丢失，而网络需要这些细节来分辨非常相似且位置相近的词元（描述最小距离的旋转不能太小，以便网络能够检测到）。

假设在 PI 中，在较大上下文规模上微调后，较短上下文规模的困惑度略有增加可能与该问题有关。在理想情况下，在较大上下文规模上进行微调不应降低较小上下文规模的性能。

为解决在对 RoPE 嵌入进行插值时丢失高频信息的问题，开发了“NTK感知”插值法。与对 RoPE 的每个维度同等按因子 s 进行缩放不同，通过减少高频的缩放幅度并增加低频的缩放幅度，将插值压力分散到多个维度上。可以通过多种方式获得这种变换，但最简单的方法是对 θ 的值进行底数变换。

更准确地说，按照第 3 节中设定的符号，将“NTK感知”插值方案定义如下：

定义1 “NTK感知”插值是对 RoPE 的一种修改，使用公式 (2) 以及以下函数：

$$g(m) = m$$
$$h(\theta_d) = b'^{-2d/|D|}$$

其中，

$$b' = b \cdot s^{\frac{|D|}{|D|-2}}$$

与 PI 相比，该方法在扩展非微调模型的上下文规模方面表现要好得多。然而，该方法的一个主要缺点是，由于它不仅仅是一种插值方案，一些维度会稍微外推到“越界”值，因此使用“NTK感知”插值法进行微调得到的结果不如 PI。此外，由于存在“越界”值，理论缩放因子 s 不能准确描述真实的上下文扩展比例。在实践中，对于给定的上下文长度扩展，缩放值 s 必须设置得高于预期比例。

5、相对局部距离的损失 - “逐部分NTK”插值法

PI和“NTK感知”插值这样的盲目插值方法中对 RoPE 的所有隐藏维度一视同仁（即认为它们对网络具有相同的影响）。然而，有明显线索表明需要有针对性的插值方法。

在本节中，主要从 RoPE 公式（公式 3）中定义的波长 λ_d 的角度来考虑问题。为简化起见，省略 λ_d 中的下标 d ，建议将 λ 视为任意周期函数的波长。

对 RoPE 嵌入的一个有趣观察是，对于给定的上下文大小 L ，在某些维度 d 上，波长比预训练期间看到的最大上下文长度更长 ($\lambda > L$)，这表明某些维度的嵌入在旋转域中可能分布不均匀。在这种情况下，假定拥有所有独特的位置意味着绝对位置信息保持完整。相反，当波长较短时，网络只能获取相对位置信息。

此外，当通过缩放因子 s 或使用底数变换 b' 对所有 RoPE 维度进行拉伸时，所有词元彼此之间变得更加接近，因为旋转量较小的两个向量的点积更大。这种缩放严重削弱了大语言模型理解其内部嵌入之间细微局部关系的能力。假设这种压缩会导致模型对相邻词元的位置顺序感到困惑，从而损害模型的能力。

为解决这个问题，选择完全不对高频维度进行插值，而始终对低频维度进行插值。具体来说：

- 如果波长 λ 远小于上下文大小 L ，不进行插值
- 如果波长 λ 等于或大于上下文大小 L ，只希望仅进行插值并避免任何外推（不同于之前的“NTK感知型”方法）
- 介于两者之间的维度可以两者兼具，类似于“NTK感知”插值法

因此，引入原始上下文大小 L 与波长 λ 之间的比率 $r = \frac{L}{\lambda}$ 会更方便。在第 d 个隐藏状态中，比率 r 与 d 的关系如下：

$$r(d) = \frac{L}{\lambda_d} = \frac{L}{2\pi b' \frac{2d}{|D|}}$$

为了如上述定义不同插值策略的边界，引入两个额外参数 α 、 β 。所有满足 $r(d) < \alpha$ 的隐藏维度 d ，按照缩放因子 s 进行线性插值（与 PI 完全一样，避免任何外推），而满足 $r(d) > \beta$ 的维度 d 则完全不进行插值。定义斜坡函数 γ 如下：

$$\gamma(r) = \begin{cases} 0, & \text{if } r < \alpha \\ 1, & \text{if } r > \beta \\ \frac{r-\alpha}{\beta-\alpha}, & \text{otherwise} \end{cases}$$

借助斜坡函数，“逐部分NTK”方法可以描述如下。

定义2 “逐部分NTK”插值是对 RoPE 的一种修改，使用公式（2）以及以下函数（对 h 进行线性斜坡插值可能有其他替代方法，例如对 θ_d/s 和从波长线性插值转换而来的 θ_d 取调和平均数。这里选择 h 是为了实现简单）：

$$g(m) = m$$

$$h(\theta_d) = (1 - \gamma(r(d))) \frac{\theta_d}{s} + \gamma(r(d)) \theta_d$$

α 和 β 的值应根据具体情况进行调整。例如，通过实验发现，对于 LLaMa 系列模型， $\alpha = 1$ 和 $\beta = 32$ 是较好的值。

使用本节所述的技术，由此产生的一种改进方法以“逐部分NTK”插值法的名称发布。无论是对于非微调模型还是微调模型，这种改进方法的表现都优于之前的 PI 和“NTK感知”（第 4 节）插值方法。

6、动态缩放 - “动态NTK”插值法

在许多应用场景中，会对长度从 1 到最大上下文大小不等的序列进行多次前向传播。一个典型的例子是自回归生成，其中序列长度在每一步后增加 1。对于使用缩放因子 s 的插值方法（包括 PI、“NTK感知”和“逐部分NTK”），有两种应用方式：

1. 在整个推理周期中，嵌入层保持固定，包括缩放因子 $s = L'/L$ ，其中 L' 是扩展上下文大小的固定值
2. 在每次前向传播中，位置嵌入更新缩放因子 $s = \max(1, l'/L)$ ，其中 l' 是当前序列的长度

方式 (1) 的问题在于，模型在长度小于 L 时可能会出现性能折扣，而当序列长度大于 L' 时性能会急剧下降。但是，通过像方式 (2) 那样进行动态缩放，它允许模型在达到训练上下文限制 L' 时性能逐渐下降，而不是立即失效。将这种推理时的方法称为动态缩放方法。当它与“NTK感知”插值法结合时，称之为“动态NTK”插值法。“动态NTK”插值法在未经任何微调、在长度 L 上预训练的模型 ($L' = L$) 上表现异常出色。

在重复的前向传播中，通常会应用 KV 缓存，以便可以重用先前的键值向量并提高整体效率。在某些实现中，当缓存 RoPE 嵌入时，为了在使用键值缓存的情况下对其进行动态缩放修改，正确的实现应该在应用 RoPE 之前缓存键值嵌入，因为当 s 变化时，每个词元的 RoPE 嵌入都会改变。

7、YaRN

除了之前的插值技术，在注意力 softmax 操作之前的对数几率 (logits) 上引入一个温度参数 t ，无论数据样本和扩展上下文窗口内的词元位置如何，都会对困惑度产生一致的影响。更确切的说，将注意力权重的计算修改为：

$$\text{softmax} \left(\frac{\mathbf{q}_m^T \mathbf{k}_n}{t \sqrt{|\mathcal{D}|}} \right) \quad (4)$$

将 RoPE 重新参数化为一组二维矩阵，对于实现这种注意力缩放有明显的好处：可以采用“长度缩放”技巧，只需将复数形式的 RoPE 嵌入按相同比例缩放一个常数因子 $\sqrt{1/t}$ ，就可以对查询向量 $\mathbf{q}_m$ 和键向量 $\mathbf{k}_n$ 进行同等缩放。这样，YaRN 可以在不修改代码的情况下有效改变注意力机制。此外，由于 RoPE 嵌入是预先生成的，并且在所有前向传播过程中都会被重复使用，因此该方法在推理和训练过程中均无额外开销。将其与“逐部分NTK”插值法相结合就得到了 YaRN 方法。

定义3 “YaRN方法”指的是公式 (4) 中的注意力缩放方法与第 5 节中介绍的“逐部分NTK”插值法的结合。

对于 LLaMA 模型推荐使用以下参数值：

$$\sqrt{\frac{1}{t}} = 0.1 \ln(s) + 1$$

9.3.2.5、训练和推理效率

1、训练成本

由于 DeepSeek-V2 相较于 DeepSeek 67B，每个词元激活的参数更少，所需的浮点运算次数（FLOPs）也更少，从理论上讲，训练 DeepSeek-V2 比训练 DeepSeek 67B 更经济。尽管训练混合专家（MoE）模型会引入额外的通信开销，但通过对算子和通信的优化，DeepSeek-V2 的训练能够实现较高的模型浮点运算利用率（MFU）。在基于 H800 集群的实际训练中，每训练一万亿个词元，DeepSeek 67B 需要 30.06 万个 GPU 小时，而 DeepSeek-V2 仅需 17.28 万个 GPU 小时，即稀疏的 DeepSeek-V2 与密集的 DeepSeek 67B 相比，可节省 42.5% 的训练成本。

2、推理效率

为了高效地将 DeepSeek-V2 部署用于服务，首先将其参数转换为 FP8 精度。此外，对 DeepSeek-V2 执行键值（KV）缓存量化以进一步将其 KV 缓存中的每个元素平均压缩至 6 比特。得益于多层注意力（MLA）以及这些优化措施，实际部署的 DeepSeek-V2 相较于 DeepSeek 67B 所需的 KV 缓存显著减少，因此能够处理更大的批量。基于实际部署的 DeepSeek 67B 服务中的 prompt 和生成长度分布，在配备 8 个 H800 GPU 的单个节点上，DeepSeek-V2 的生成吞吐量超过每秒 5 万个词元，是 DeepSeek 67B 最大生成吞吐量的 5.76 倍。此外，DeepSeek-V2 的 prompt 输入吞吐量超过每秒 10 万个词元。

9.3.3、对齐

9.3.3.1、监督式微调

指令调优数据集包含 150 万个实例，包括 120 万个用于提升模型有用性的实例和 30 万个用于增强安全性的实例。与初始版本相比提高了数据质量，以减少模型产生幻觉式回复的情况，并提升其语言表达能力。DeepSeek-V2 进行 2 轮微调，学习率设置为 5×10^{-6} 。对于 DeepSeek-V2 Chat（SFT）的评估，除了几个有代表性的多项选择题任务（如 MMLU 和 ARC），主要采用基于生成的基准测试。

9.3.3.2、强化学习

DeepSeek-V2 通过强化学习（RL）进一步释放 DeepSeek-V2 的潜力并使其与人类偏好对齐。

1、强化学习算法

为了节省 RL 的训练成本，DeepSeek-V2 采用群组相对策略优化（Group Relative Policy Optimization, GRPO），该方法摒弃了通常与策略模型规模相同的评论家模型，而是从组分数中估计基线。具体来说，对于每个问题 q ，GRPO 从旧策略 $\pi_{\theta_{old}}$ 中采样一组输出 $\{o_1, o_2, \dots, o_G\}$ ，然后通过最大化以下目标来优化策略模型 π_θ ：

$$\begin{aligned} \mathcal{J}_{GRPO}(\theta) = & \mathbb{E} \left[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O | q) \right] \\ & \frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_\theta(o_i | q)}{\pi_{\theta_{old}}(o_i | q)} A_i, \text{clip} \left(\frac{\pi_\theta(o_i | q)}{\pi_{\theta_{old}}(o_i | q)}, 1 - \varepsilon, 1 + \varepsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_\theta || \pi_{ref}) \right) \end{aligned}$$

$$\mathbb{D}_{KL}(\pi_\theta || \pi_{ref}) = \frac{\pi_{ref}(o_i | q)}{\pi_\theta(o_i | q)} - \log \frac{\pi_{ref}(o_i | q)}{\pi_\theta(o_i | q)} - 1,$$

其中 ε 和 β 是超参数； A_i 是根据每组输出对应的奖励 $\{r_1, r_2, \dots, r_G\}$ 计算的收益：

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}.$$

2、训练策略

针对推理数据（如代码和数学提示）的强化学习训练展现出与一般数据训练不同的独特特征。例如，模型的数学和编码能力在较长的训练步数内能够持续提升。因此，DeepSeek-V2 采用两阶段强化学习训练策略，首先进行推理对齐，然后进行人类偏好对齐。

$$r_i = RM_{\text{reasoning}}(o_i).$$

在第二个人类偏好对齐阶段采用了一个多奖励框架，从有帮助的奖励模型 RM_{helpful} 、安全奖励模型 RM_{safety} 和基于规则的奖励模型 RM_{rule} 获取奖励。响应 o_i 的最终奖励为：

$$r_i = c_1 \cdot RM_{\text{helpful}}(o_i) + c_2 \cdot RM_{\text{safety}}(o_i) + c_3 \cdot RM_{\text{rule}}(o_i),$$

其中 c_1 、 c_2 和 c_3 是相应的系数。

为了获得在 RL 训练中起关键作用的可靠奖励模型，DeepSeek-V2 仔细收集了偏好数据，并精心进行了质量过滤和比例调整。DeepSeek-V2 基于编译器反馈获得代码偏好数据，基于真实标签获得数学偏好数据。对于奖励模型的训练，使用 DeepSeek-V2 Chat（SFT）初始化奖励模型，并使用 point-

wise 或 pair-wise 损失对其进行训练。

3、训练效率优化

对大型模型进行 RL 训练对训练框架提出了很高的要求，需要精心的工程优化来管理 GPU 内存和 RAM 压力，同时保持快速的训练速度。为此，DeepSeek-V2 实施了以下工程优化：

1. DeepSeek-V2 提出了一种混合引擎，分别采用不同的并行策略进行训练和推理，以实现更高的 GPU 利用率
2. 利用 vLLM 和设置较大的批量大小作为推理后端，以加速推理速度
3. 设计了一种将模型卸载到 CPU 并重新加载回 GPU 的调度策略，在训练速度和内存消耗之间实现了近乎最优的平衡

9.3.3.3、讨论

1、监督微调（SFT）数据量

围绕大规模 SFT 语料库必要性的讨论一直是激烈争论的话题。先前的研究认为，少于 10,000 个 SFT 数据实例就足以产生令人满意的结果。然而，在 DeepSeek-V2 的实验中，如果使用少于 10,000 个实例，在 IFEval 基准测试中性能显著下降。一种可能的解释是，语言模型需要一定量的数据来培养特定技能。尽管随着模型规模的增加，所需的数据量可能会减少，但它不可能完全消除。DeepSeek-V2 的观察强调了为大语言模型（LLM）配备所需能力时，充足数据的关键必要性。此外，SFT 数据的质量也至关重要，特别是对于涉及写作或开放式问题的任务。

2、强化学习的对齐代价

在人类偏好对齐过程中，DeepSeek-V2 观察到在开放式生成基准测试中，无论是人工智能还是人类评估者给出的分数，都显示出性能有显著提升。然而，DeepSeek-V2 也注意到一种“对齐代价（alignment tax）”现象，即对齐过程可能会对某些标准基准测试（如 BBH）的性能产生负面影响。为了减轻这种对齐代价，在强化学习阶段，DeepSeek-V2 在数据处理和改进训练策略方面做出了巨大努力，最终在标准基准测试和开放式基准测试的性能之间实现了可接受的权衡。

3、在线强化学习

在 DeepSeek-V2 的偏好对齐实验中，在线方法显著优于离线方法。因此，DeepSeek-V2 投入了大量精力来实现一个用于对齐 DeepSeek-V2 的在线强化学习框架。

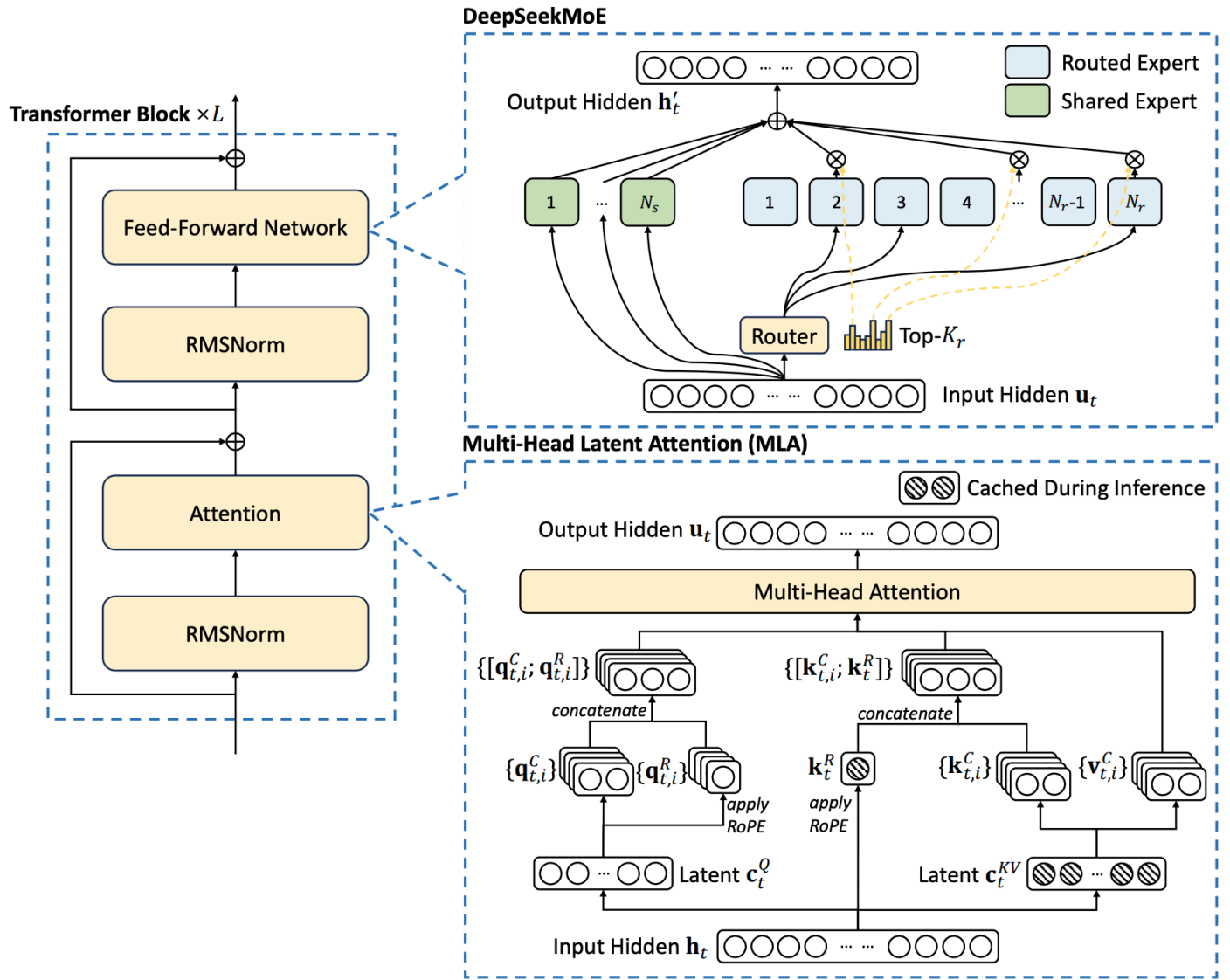
9.4、DeepSeek-V3

DeepSeek-V3 发布于 2024 年 12 月，采用了 DeepSeek-V2 中的多头潜在注意力（MLA）和 DeepSeekMoE 架构，此外，DeepSeek-V3 开创了一种无辅助损失的负载均衡策略，并设定了多词元预测训练目标以获得更强的性能。

DeepSeek-V3 总参数量达 6710 亿个，每个词元激活 370 亿个参数，DeepSeek-V3 在 14.8 万亿个多样化且高质量标记上对 DeepSeek-V3 进行了预训练，随后通过监督微调和强化学习阶段来充分发挥其能力。

9.4.1、架构

DeepSeek-V3 的基本架构仍在 Transformer 框架内，为了实现高效的推理和经济的训练，DeepSeek-V3 采用了 MLA（多级注意力）和 DeepSeekMoE（深度可微分的多任务学习），并引入了一个无辅助损失的负载均衡机制。



(/img/post/llm-model-51.png)

9.4.1.1、多头潜在注意力

DeepSeek-V3 采用 MLA（Multi-Head Latent Attention）架构。设 d 表示嵌入维度， n_h 表示注意力头的数量， d_h 表示每个头的维度， $h_t \in R^d$ 表示在给定注意力层的第 t 个词元的注意力输入。MLA 的核心是对注意力键（key）和值（value）进行低秩联合压缩以减少推理过程中的键值（KV）缓存：

$$\begin{aligned}
 c_t^{KV} &= W^{DKV} h_t, \\
 [k_{t,1}^C; k_{t,2}^C; \dots; k_{t,n_h}^C] &= k_t^C = W^{UK} c_t^{KV}, \\
 k_t^R &= \text{RoPE}(W^{KR} h_t), \\
 k_{t,i} &= [k_{t,i}^C; k_t^R], \\
 [v_{t,1}^C; v_{t,2}^C; \dots; v_{t,n_h}^C] &= v_t^C = W^{UV} c_t^{KV},
 \end{aligned}$$

其中：

- $c_t^{KV} \in \mathbb{R}^{d_c}$ 是用于 key 和 value 的压缩潜在向量， $d_c (\ll d_h n_h)$ 表示 KV 压缩维度；
- $W^{DKV} \in \mathbb{R}^{d_c \times d}$ 表示下投影矩阵；
- $W^{UK}, W^{UV} \in \mathbb{R}^{d_h n_h \times d_c}$ 分别是用于 key 和 value 的上升投影矩阵；
- $W^{KR} \in \mathbb{R}^{d_h^R \times d}$ 是用于生成携带旋转位置嵌入（RoPE）的解耦键的矩阵；
- $\text{RoPE}(\cdot)$ 表示应用 RoPE 矩阵的操作；
- $[\cdot; \cdot]$ 表示连接。

对于 MLA，只有 c_t^{KV} 和 k_t^R 向量需要在生成过程中缓存，如此显著减少了 KV 缓存，同时保持与标准多头注意力（MHA）相当的性能。

对于注意力 query 执行低秩压缩以减少训练期间的激活内存：

$$\begin{aligned} c_t^Q &= W^{DQ} h_t, \\ [q_{t,1}^C; q_{t,2}^C; \dots; q_{t,n_h}^C] &= q_t^C = W^{UQ} c_t^Q, \\ [q_{t,1}^R; q_{t,2}^R; \dots; q_{t,n_h}^R] &= q_t^R = \text{RoPE}(W^{QR} c_t^Q), \\ q_{t,i} &= [q_{t,i}^C; q_{t,i}^R] \end{aligned}$$

其中：

- $c_t^Q \in \mathbb{R}^{d'_c}$ 是用于 query 的压缩潜在向量；
- $d'_c (\ll d_h n_h)$ 表示 query 压缩维度；
- $W^{DQ} \in \mathbb{R}^{d'_c \times d}, W^{UQ} \in \mathbb{R}^{d_h n_h \times d'_c}$ 分别是用于 query 的下投影和上升投影矩阵；
- $W^{QR} \in \mathbb{R}^{d_h^R n_h \times d'_c}$ 是用于生成携带 RoPE 的解耦 query 的矩阵。

最终，将注意力 query ($q_{t,i}$)、key ($k_{j,i}$) 和 value ($v_{j,i}^C$) 结合起来，得到最终的注意力输出 u_t ：

$$\begin{aligned} o_{t,i} &= \sum_{j=1}^t \text{Softmax}_j \left(\frac{q_{t,i}^T k_{j,i}}{\sqrt{d_h + d_h^R}} \right) v_{j,i}^C \\ u_t &= W^O [o_{t,1}; o_{t,2}; \dots; o_{t,n_h}], \end{aligned}$$

其中 $W^O \in R^{d \times d_h n_h}$ 表示输出投影矩阵。

9.4.1.2、采用辅助无损失负载均衡的 DeepSeekMoE

1、DeepSeekMoE 基本架构

对于前馈网络（FFN），DeepSeek-V3 采用了 DeepSeekMoE 架构。与 GShard 等传统的混合专家（MoE）架构相比，DeepSeekMoE 使用了更细粒度的专家，并将一些专家隔离为共享专家。

设 $\mathbf{u}_t$ 表示第 t 个 token 的 FFN 输入，按如下方式计算 FFN 输出 $\mathbf{h}'_t$ ：

$$\begin{aligned}\mathbf{h}'_t &= \mathbf{u}_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t} \text{FFN}_i^{(r)}(\mathbf{u}_t), \\ g_{i,t} &= \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{j,t}}, \\ g'_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r) \\ 0, & \text{otherwise} \end{cases}, \\ s_{i,t} &= \text{Sigmoid}(\mathbf{u}_t^T \mathbf{e}_i),\end{aligned}$$

其中：

- N_s 和 N_r 分别表示共享专家和路由专家的数量；
- $\text{FFN}_i^{(s)}(\cdot)$ 和 $\text{FFN}_i^{(r)}(\cdot)$ 分别表示第 i 个共享专家和第 i 个路由专家；
- K_r 表示被激活的路由专家的数量； $g_{i,t}$ 是第 i 个专家的门控值； $s_{i,t}$ 是标记与专家的亲和度； $\mathbf{e}_i$ 是第 i 个路由专家的质心向量；
- $\text{Topk}(\cdot, K)$ 表示在为第 t 个标记和所有路由专家计算的亲和度得分中，由 K 个最高分组成的集合

与 DeepSeek-V2 略有不同，DeepSeek-V3 使用 Sigmoid 函数来计算亲和度得分，并在所有选定的亲和度得分上应用归一化来生成门控值。

2、无辅助损失的负载均衡

对于混合专家（MoE）模型而言，专家负载不均衡会导致路由崩溃，并且在专家并行的场景下会降低计算效率。传统的解决方案通常依靠辅助损失来避免负载不均衡。然而，过大的辅助损失会损害模型性能。为了在负载均衡和模型性能之间实现更好的权衡，DeepSeek-V3 首创了一种无辅助损失的负载均衡策略来确保负载均衡。具体来说，DeepSeek-V3 为每个专家引入一个偏置项 b_i ，并将其添加到相应的亲和度得分 $s_{i,t}$ 中，以确定前 K 路由：

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Top}k(\{s_{j,t} + b_j | 1 \leq j \leq N_r\}, K_r) \\ 0, & \text{otherwise} \end{cases}$$

需要注意的是，偏置项仅用于路由。将与前馈网络（FFN）输出相乘的门控值仍然由原始的亲和度得分 $s_{i,t}$ 得出。在训练过程中每个步骤结束时，如果某个专家的负载过重，将其对应的偏置项减小 γ ；如果某个专家的负载过轻，将其对应的偏置项增加 γ ，其中 γ 是一个被称为偏置更新速度的超参数。通过这种动态调整，DeepSeek-V3 在训练过程中保持了专家负载的均衡，并且比单纯依靠辅助损失来促进负载均衡的模型取得了更好的性能。

3、互补的序列级辅助损失

尽管 DeepSeek-V3 主要依靠无辅助损失策略来实现负载均衡，但为了防止任何单个序列内出现极端的不均衡情况，还采用了一种互补的序列级均衡损失：

$$\begin{aligned} \mathcal{L}_{\text{Bal}} &= \alpha \sum_{i=1}^{N_r} f_i P_i, \\ f_i &= \frac{N_r}{K_r T} \sum_{t=1}^T \mathbb{1}(s_{i,t} \in \text{Top}k(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r)), \\ s'_{i,t} &= \frac{s_{i,t}}{\sum_{j=1}^{N_r} s_{j,t}}, \\ P_i &= \frac{1}{T} \sum_{t=1}^T s'_{i,t}, \end{aligned}$$

其中，均衡因子 α 是一个超参数，在 DeepSeek-V3 中会被赋予一个极小的值； $\mathbb{1}(\cdot)$ 表示指示函数； T 表示一个序列中的 token 数量。这种序列级均衡损失促使每个序列上的专家负载达到均衡。

4、节点受限路由

与 DeepSeek-V2 采用的设备受限路由类似，DeepSeek-V3 同样使用了一种受限路由机制，以限制训练过程中的通信成本。简而言之，确保每个 token 最多被发送到 M 个节点，这些节点根据分布在每个节点上的专家的最高 K_r^M 个亲和度得分之和来选择的。在这一约束条件下，混合专家（MoE）训练框架几乎可以实现计算与通信的完全重叠。

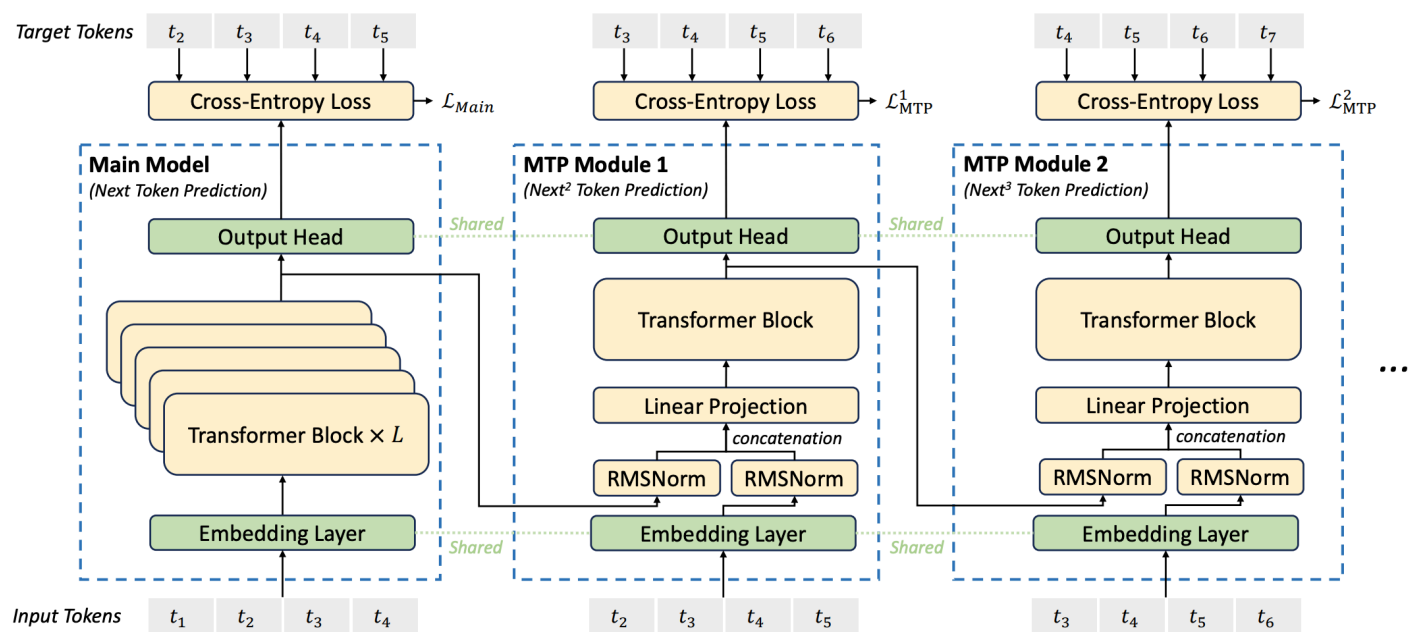
5、无词元丢弃

由于采用了有效的负载均衡策略，DeepSeek-V3 在整个训练过程中都能保持良好的负载均衡。因此，DeepSeek-V3 在训练期间不会丢弃任何 token。此外，DeepSeek-V3 还实施了特定的部署策略，以确保推理阶段的负载均衡，所以在推理过程中同样不会丢弃 token。

9.4.1.3、多词元预测

DeepSeek-V3 设定了多词元预测（Multi-Token Prediction, MTP）目标，该目标将每个位置的预测范围扩展到多个未来 token。一方面，MTP 目标使训练信号更加密集，从而提高数据效率。另一方面，MTP 使模型能够预先规划其表示，以便更好地预测未来 token。与其他使用独立输出头并行预测 D 个额外 token 不同，DeepSeek-V3 按顺序预测额外标记，并在每个预测深度保持完整的因果链。

1、MTP 模块



MTP 实现使用 D 个顺序模块来预测 D 个额外 token。第 k 个 MTP 模块由一个共享嵌入层 $\text{Emb}(\cdot)$ 、一个共享输出头 $\text{OutHead}(\cdot)$ 、一个 Transformer 块 $\text{TRM}_k(\cdot)$ 和一个投影矩阵 $M_k \in \mathbb{R}^{d \times 2d}$ 组成。

对于第 i 个输入 token t_i ，在第 k 个预测深度首先通过线性投影将第 $(k-1)$ 层深度下第 i 个 token 的表示 $\mathbf{h}_i^{k-1} \in \mathbb{R}^d$ 与第 $(i+k)$ 个 token 的嵌入 $\text{Emb}(t_{i+k}) \in \mathbb{R}^d$ 相结合：

$$\mathbf{h}_i^{'k} = M_k[\text{RMSNorm}(\mathbf{h}_i^{k-1}); \text{RMSNorm}(\text{Emb}(t_{i+k}))]$$

其中， $[\cdot; \cdot]$ 表示拼接操作。特别地，当 $k=1$ 时， $\mathbf{h}_i^{k-1}$ 指的是主模型给出的表示。

需要注意的是，对于每个 MTP 模块，其嵌入层与主模型共享。组合后的 $\mathbf{h}_i^{'k}$ 作为第 k 层深度的 Transformer 块的输入，以生成当前深度的输出表示 $\mathbf{h}_i^k$ ：

$$\mathbf{h}_{1:T-k}^k = \text{TRM}_k(\mathbf{h}_{1:T-k}^{'k})$$

其中， T 表示输入序列的长度， $i:j$ 表示切片操作（包含左右边界）。最后，以 $\mathbf{h}_i^k$ 作为输入，共享输出头将计算第 k 个额外预测标记 $\mathbf{P}_{i+1+k}^k \in \mathbb{R}^V$ 的概率分布，其中 V 是词汇表大小：

$$\mathbf{P}_{i+k+1}^k = \text{OutHead}(\mathbf{h}_i^k)$$

输出头 $\text{OutHead}(\cdot)$ 将表示线性映射为对数几率，然后应用 $\text{Softmax}(\cdot)$ 函数来计算第 k 个额外标记的预测概率。同样，对于每个 MTP 模块，其输出头与主模型共享。保持预测因果链的原则与 EAGLE 类似，但其主要目标是推测性解码，而 DeepSeek-V3 利用 MTP 来改进训练。

9.4.3、训练框架

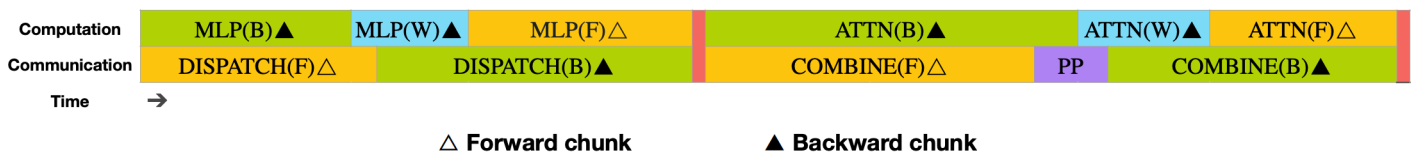
DeepSeek-V3 的训练由 HAI-LLM 框架提供支持，采用了 16 路流水线并行（PP）、跨越 8 个节点的 64 路专家并行（EP）以及 ZeRO-1 数据并行（DP）。

9.4.3.1、双管道与计算-通信重叠

DeepSeek-V3 的跨节点专家并行引入的通信开销导致了约 1:1 的低效计算与通信比率。为此设计了一种创新的管道并行算法，称为双管道（DualPipe）。该算法不仅通过有效重叠前向和后向计算-通信阶段来加速模型训练，还能减少管道气泡。

双管道的核心思想是在一对单独的前向和后向块内重叠计算与通信。具体而言，将每个块划分为四个组件：注意力（attention）、全对全分发（all-to-all dispatch）、多层感知器（MLP）以及全对全合并（all-to-all combine）。特别地，对于后向块，注意力和多层感知器进一步细分为输入的反向计算和权重的反向计算两部分。

此外，DeepSeek-V3 还有一个管道并行（PP）通信组件。如下图所示，对于一对前向和后向块，重新排列这些组件，并手动调整专用于通信与计算的 GPU 流式多处理器（SM）的比例。在这种重叠策略下，可以确保在执行过程中全对全通信和管道并行通信都能被完全隐藏。



(/img/post/llm-model-53.png)

一对单独的前向和后向块的重叠策略（Transformer模块的边界未对齐）。橙色表示前向，绿色表示“输入的反向”，蓝色表示“权重的反向”，紫色表示管道并行（PP）通信，红色表示屏障。全对全通信和管道并行通信都可以完全被隐藏

鉴于这种高效的重叠策略，完整的双管道调度如下图所示。其采用双向管道调度，从管道的两端同时输入微批次数据，并且很大一部分通信可以完全重叠。这种重叠还确保了，随着模型进一步扩展，只要保持恒定的计算与通信比率，仍然可以在跨节点使用细粒度的专家，同时实现近乎零的全对全通信开销。



(/img/post/llm-model-54.png)

针对8个管道并行（PP）等级和20个微批次在两个方向上的双管道（DualPipe）调度示例。反向的微批次与正向的微批次对称，为简化说明省略了反向微批次的批次ID。被共享黑色边框包围的两个单元

格，其计算和通信相互重叠

9.4.3.2、跨节点全对全通信的高效实现

为确保双管道（DualPipe）有足够的计算性能，DeepSeek-V3 定制了高效的跨节点全对全通信内核（包括分发和合并），以节省用于通信的流式多处理器（SM）数量。这些内核的实现与混合专家（MoE）选通算法以及集群的网络拓扑共同设计。

在集群中，跨节点的图形处理器（GPU）通过无限带宽（IB）网络完全互连，而节点内的通信则通过英伟达高速互连（NVLink）处理。NVLink 提供 160GB/s 的带宽，约为 IB 带宽（50GB/s）的 3.2 倍。为有效利用 IB 和 NVLink 不同的带宽，将每个 token 最多分发到 4 个节点，从而减少 IB 流量。对于每个 token，在做出路由决策后，首先会通过 IB 传输到目标节点上具有相同节点内索引的 GPU。一旦到达目标节点，会确保它能通过 NVLink 立即转发到承载目标专家的特定 GPU，而不会被随后到达的 token 阻塞。通过这种方式，IB 和 NVLink 的通信得以完全重叠，并且每个 token 在每个节点上平均能高效选择 3.2 个专家，而不会产生 NVLink 带来的额外开销。这意味着，尽管 DeepSeek-V3 在实际中仅选择 8 个路由专家，但在保持相同通信成本的情况下，可将该数量最多扩展到 13 个专家（4 个节点 × 每个节点 3.2 个专家）。总体而言，在这样的通信策略下，仅需 20 个 SM 就足以充分利用 IB 和 NVLink 的带宽。

详细来说，DeepSeek-V3 采用线程束专用化技术，并将 20 个 SM 划分为 10 个通信通道。在分发过程中，（1）IB 发送、（2）从 IB 到 NVLink 的转发以及（3）NVLink 接收分别由各自的线程束处理。分配给每个通信任务的线程束数量会根据所有 SM 的实际工作负载动态调整。类似地，在合并过程中，（1）NVLink 发送、（2）从 NVLink 到 IB 的转发与累加以及（3）IB 接收与累加也由动态调整的线程束处理。

此外，分发和合并内核都与计算流重叠，因此也考虑了它们对其他 SM 计算内核的影响。具体来说，DeepSeek-V3 采用定制的并行线程执行（PTX）指令，并自动调整通信块大小，这显著减少了二级缓存（L2 cache）的使用以及对其他 SM 的干扰。

9.4.3.3、以极小开销实现极致内存节省

为减少训练过程中的内存占用，DeepSeek-V3 采用了以下技术：

- RMSNorm 和 MLA 上采样投影的重计算

- 在反向传播过程中，对所有 RMSNorm 操作和 MLA 上采样投影进行重计算，从而无需持续存储它们的输出激活值。通过这种略微增加开销的策略，显著降低了存储激活值所需的内存
- 在 CPU 中进行指数移动平均
 - 在训练期间保存模型参数的指数移动平均（EMA），以便在学习率衰减后尽早评估模型性能。EMA 参数存储在 CPU 内存中，并在每个训练步骤后异步更新。这种方法能够维护 EMA 参数，而不会产生额外的内存或时间开销
- 多令牌预测的共享嵌入层和输出头
 - 借助双管道（DualPipe）策略，将模型的最浅层（包括嵌入层）和最深层（包括输出头）部署在相同的管道并行（PP）等级上。这种安排使得多令牌预测（MTP）模块与主模型之间能够在物理上共享嵌入层和输出头的参数与梯度。这种物理共享机制进一步提高了内存使用效率

9.4.4、FP8训练

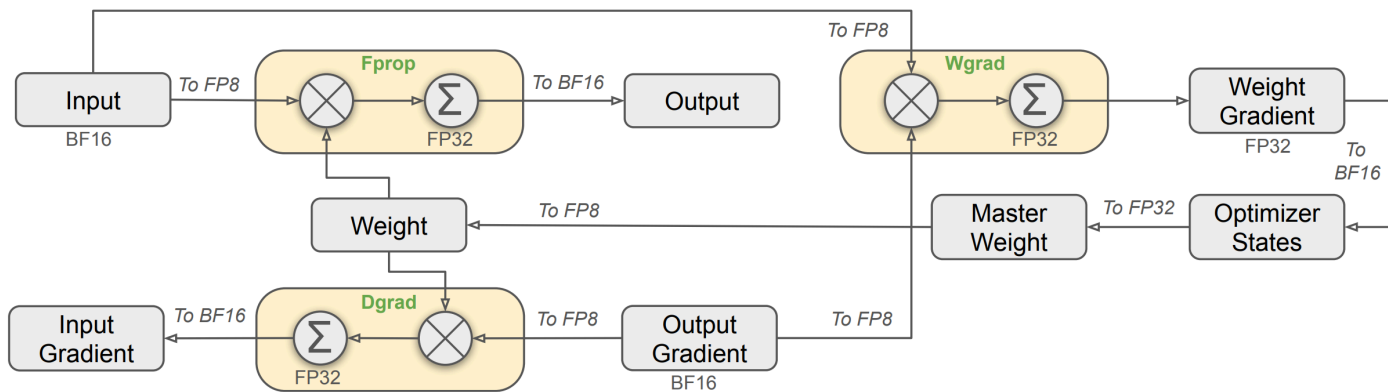
DeepSeek-V3 提出了一种细粒度混合精度框架，采用 FP8 数据格式来训练 DeepSeek-V3。尽管低精度训练前景广阔，但它常常受到激活值、权重和梯度中异常值的限制。虽然推理量化方面已取得显著进展，但相对较少有研究表明低精度技术能成功应用于大规模语言模型的预训练。

为应对这一挑战并有效扩展 FP8 格式的动态范围，DeepSeek-V3 引入了一种细粒度量化的策略：按 $1 \times N_c$ 元素的瓦片式分组（tile-wise grouping），或按 $N_c \times N_c$ 元素的块式分组（block-wise grouping）。在提高精度的累加过程中，相关的反量化开销得到了大幅缓解，这是实现精确的 FP8 通用矩阵乘法（GEMM）的关键因素。

此外，为进一步降低混合专家（MoE）训练中的内存和通信开销，DeepSeek-V3 以 FP8 格式缓存和分发激活值，同时以 BF16 格式存储低精度优化器状态。在与 DeepSeek-V2-Lite 和 DeepSeek-V2 规模相近的两种模型上验证了所提出的 FP8 混合精度框架，训练数据量约为 1 万亿 个词元。值得注意的是，与 BF16 基准相比，采用 FP8 训练的模型相对损失误差始终保持在 0.25% 以下，这一水平完全处于训练随机性可接受的范围内。

9.4.4.1、混合精度框架

DeepSeek-V3 提出了一种用于 FP8 训练的混合精度框架。在这个框架中，大多数计算密集型操作以 FP8 进行，而一些关键操作则策略性地保持其原始数据格式，以平衡训练效率和数值稳定性。



(/img/post/llm-model-55.png)

采用 FP8 数据格式的整体混合精度框架。为清晰起见，仅展示了线性算子

首先，为了加速模型训练，大多数核心计算内核，即通用矩阵乘法（GEMM）操作，以 FP8 精度实现。这些 GEMM 操作接受 FP8 张量作为输入，并生成 BF16 或 FP32 格式的输出。如上图所示，与线性算子相关的所有三个 GEMM 操作，即前向传播（Fprop）、激活反向传播（Dgrad）和权重反向传播（Wgrad），均以 FP8 执行。与原始的 BF16 方法相比，这种设计在理论上可将计算速度提高一倍。此外，FP8 格式的权重反向传播（Wgrad）GEMM 操作允许激活值以 FP8 格式存储，以便在反向传播中使用。这显著降低了内存消耗。

尽管 FP8 格式具有效率优势，但某些算子由于对低精度计算较为敏感，仍需要更高的精度。此外，一些低成本算子也可以使用更高的精度，且对整体训练成本的影响可忽略不计。基于这些原因，对以下组件保持原始精度（如 BF16 或 FP32）：嵌入模块、输出头、混合专家（MoE）选通模块、归一化算子和注意力算子。这些有针对性地保留高精度的做法，确保了 DeepSeek-V3 训练过程的稳定性。

为进一步保证数值稳定性，DeepSeek-V3 以更高精度存储主权重、权重梯度和优化器状态。虽然这些高精度组件会带来一些内存开销，但通过在其分布式训练系统中跨多个数据并行（DP）等级进行高效分片，其影响可降至最低。

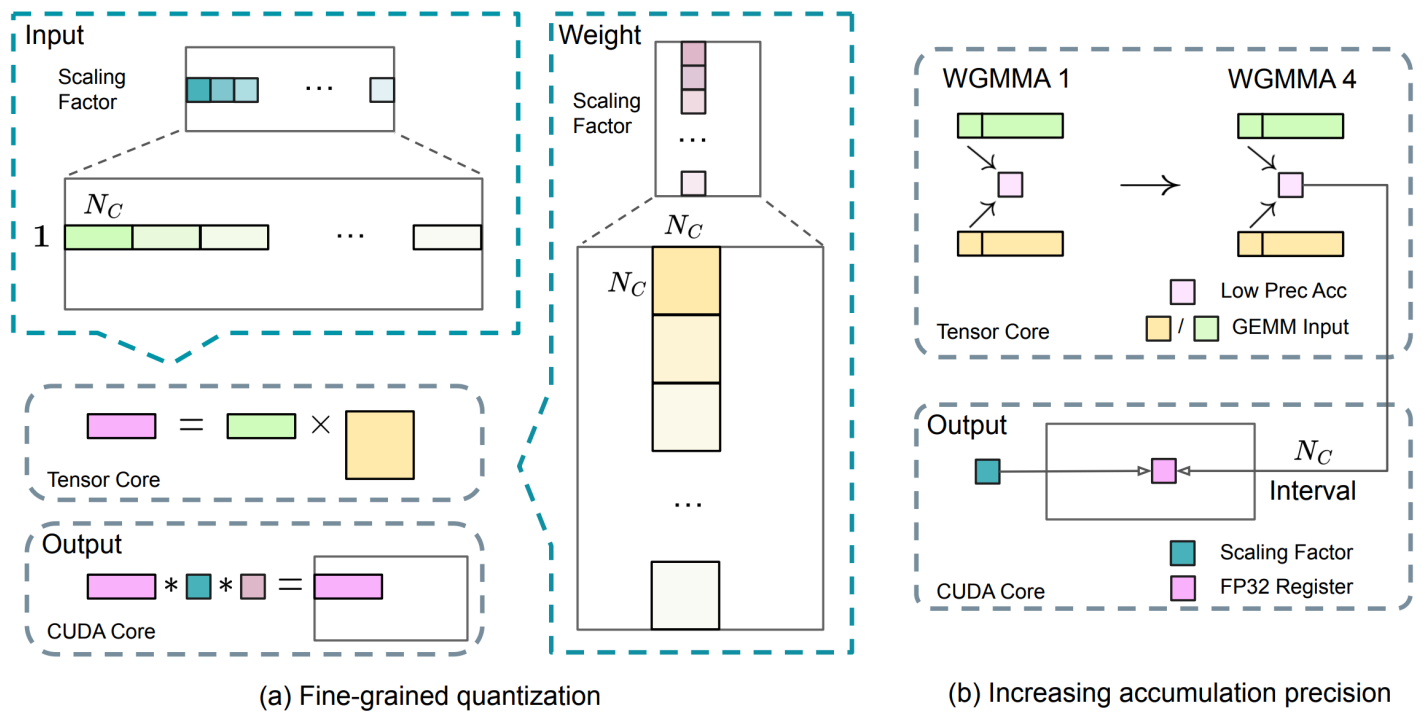
9.4.4.2、通过量化和乘法提高精度

基于混合精度 FP8 框架，DeepSeek-V3 引入了几种策略来提高低精度训练的准确性，主要聚焦于量化方法和乘法过程：

1、细粒度量化

在低精度训练框架中，由于 FP8 格式的动态范围有限（受其减少的指数位限制），上溢和下溢是常见问题。作为一种标准做法，通过将输入张量的最大绝对值缩放到 FP8 的最大可表示值，使输入分布与 FP8 格式的可表示范围对齐。这种方法使得低精度训练对激活异常值高度敏感，严重降低量化精度。

为解决此问题，DeepSeek-V3 提出一种细粒度量化方法，在更细的粒度上应用缩放。如下图中 (a) 所示，(1) 对于激活值，以 1×128 的瓦片为基础对元素进行分组和缩放（即每个词元每 128 个通道）；(2) 对于权重，以 128×128 的块为基础对元素进行分组和缩放（即每 128 个输入通道对应每 128 个输出通道）。这种方法确保量化过程能够通过根据较小的元素组调整缩放比例，更好地处理异常值。



(/img/post/llm-model-56.png)

(a) DeepSeek-V3 提出一种细粒度量化方法，以减轻由特征异常值导致的量化误差；为简化说明，仅展示了前向传播 (Fprop) 的情况。(b) 结合 DeepSeek-V3 的量化策略，通过每隔 $N_C = 128$ 个元素的矩阵乘加 (MMA) 操作将计算提升至 CUDA 核心，来提高 FP8 通用矩阵乘法 (GEMM) 的精度，以实现高精度累加

DeepSeek-V3 方法的一个关键改进是在 GEMM 操作的内维度引入每组缩放因子。标准的 FP8 GEMM 并不直接支持此功能。然而，结合 DeepSeek-V3 精确的 FP32 累加策略，则可以高效实现。

值得注意的是，DeepSeek-V3 的细粒度量化策略与微缩放格式的理念高度一致，而英伟达下一代 GPU (Blackwell 系列) 的张量核心已宣布支持具有更细量化粒度的微缩放格式。

2、提高累加精度

低精度 GEMM 操作经常面临下溢问题，其准确性在很大程度上依赖于高精度累加，通常这一过程以 FP32 精度执行。然而，在英伟达 H800 GPU 上，FP8 GEMM 的累加精度仅限于保留约 14 位，这明显低于 FP32 的累加精度。当内维度 K 较大时，这个问题会更加突出，这是大规模模型训练中增加批量大小和模型宽度时的典型场景。以两个随机矩阵的 GEMM 操作 ($K = 4096$) 为例，在初步测试中，张量核心有限的累加精度导致最大相对误差接近 2%。尽管存在这些问题，有限的累加精度在一些 FP8 框架中仍是默认选项，严重限制了训练精度。

为解决此问题，DeepSeek-V3 采用提升到 CUDA 核心以获得更高精度的策略。过程如上图 7 (b) 所示。具体来说，在张量核心执行矩阵乘加 (MMA) 操作时，中间结果使用有限的位宽进行累加。一旦达到间隔 NC ，这些部分结果将被复制到 CUDA 核心的 FP32 寄存器中，在那里进行全精度的 FP32 累加。如前所述，DeepSeek-V3 的细粒度量化沿内维度 K 应用每组缩放因子。这些缩放因子可以在 CUDA 核心上作为反量化过程高效相乘，且额外计算成本最小。

值得注意的是，此修改降低了单个线程束组的线程束组级矩阵乘加 (WGMMMA) 指令发布率。然而，在 H800 架构中，典型情况是两个 WGMMMA 可以同时持续执行：当一个线程束组执行提升操作时，另一个能够执行 MMA 操作。这种设计使两个操作可以重叠，保持张量核心的高利用率。基于实验，设置 $NC = 128$ 个元素，相当于 4 个 WGMMMA，是能够显著提高精度且不引入大量开销的最小累加间隔。

3、尾数优先于指数

与先前工作采用的混合 FP8 格式不同，他们在前向传播 (Fprop) 中使用 E4M3 (4 位指数和 3 位尾数)，在激活反向传播 (Dgrad) 和权重反向传播 (Wgrad) 中使用 E5M2 (5 位指数和 2 位尾数)，DeepSeek-V3 对所有张量采用 E4M3 格式以获得更高精度。将这种方法的可行性归因于 DeepSeek-V3 的细粒度量化策略，即瓦片式和块式缩放。通过对较小的元素组进行操作，DeepSeek-V3 的方法有效地在这些分组元素之间共享指数位，减轻了有限动态范围的影响。

4、在线量化

张量级量化框架采用延迟量化，它维护先前迭代中的最大绝对值历史以推断当前值。为确保准确的缩放并简化框架，DeepSeek-V3 为每个 1×128 的激活瓦片或 128×128 的权重块在线计算最大绝对值。基于此导出缩放因子，然后将激活值或权重在线量化为 FP8 格式。

9.4.4.3、低精度存储和通信

结合 FP8 训练框架，DeepSeek-V3 通过将缓存的激活值和优化器状态压缩为低精度格式，进一步降低内存消耗和通信开销。

1、低精度优化器状态

DeepSeek-V3 采用 BF16 数据格式而非 FP32 来跟踪 AdamW 优化器中的一阶矩和二阶矩，且不会导致可察觉的性能下降。不过，主权重（由优化器存储）和梯度（用于批量大小累积）仍保留为 FP32 格式，以确保整个训练过程中的数值稳定性。

2、低精度激活

Wgrad 操作以 FP8 格式执行，为降低内存消耗，在 Linear 算子的反向传播中，将激活值缓存为 FP8 格式是很自然的选择。然而，对于一些算子，为实现低成本的高精度训练，需进行特殊考量：

1. **注意力算子之后的Linear的输入**：这些激活值也用于注意力算子的反向传播，这使得它们对精度很敏感。DeepSeek-V3 专门为这些激活值采用定制的 E5M6 数据格式。此外，在反向传播中，这些激活值将从 1×128 的量化块转换为 128×1 的量化块。为避免引入额外的量化误差，所有缩放因子均进行舍入缩放，即 2 的整数次幂
2. **混合专家（MoE）中SwiGLU算子的输入**：为进一步降低内存成本，缓存 SwiGLU 算子的输入，并在反向传播中重新计算其输出。这些激活值也使用 DeepSeek-V3 的细粒度量化方法存储为 FP8 格式，在内存效率和计算精度之间取得平衡

3、低精度通信

通信带宽是 MoE 模型训练中的关键瓶颈。为应对这一挑战，在 MoE 上投影之前将激活值量化为 FP8，然后应用调度组件，这与 MoE 上投影中的 FP8 正向传播兼容。与注意力算子之后的 Linear 的输入类似，此激活值的缩放因子为 2 的整数次幂。类似的策略也应用于 MoE 下投影之前的激活梯度。对于前向和反向组合组件，将它们保留为 BF16 格式，以在训练流程的关键部分保持训练精度。

9.4.4、推理和部署

DeepSeek-V3 部署在 H800 集群上，集群内每个节点中的 GPU 通过 NVLink 互连，而整个集群中的所有 GPU 通过 IB 实现全互连。为了同时确保在线服务的服务水平目标（SLO）和高吞吐量，采用以下将预填充和解码阶段分开的部署策略。

9.4.4.1、预填充

预填充阶段的最小部署单元由 4 个节点共 32 个 GPU 组成。注意力部分采用结合序列并行（SP）的 4 路张量并行（TP4），以及 8 路数据并行（DP8）。其较小的 TP 规模（大小为 4）限制了张量并行通信的开销。对于混合专家（MoE）部分，DeepSeek-V3 使用 32 路专家并行（EP32），这确保每个专家处理足够大的批量大小，从而提高计算效率。

对于 MoE 的全对全通信，DeepSeek-V3 使用与训练中相同的方法：首先通过 IB 在节点间传输令牌，然后通过 NVLink 在节点内的 GPU 之间转发。特别地，对于浅层的密集多层感知器（MLP），使用 1 路张量并行以节省张量并行通信开销。

为了在 MoE 部分的不同专家之间实现负载均衡，需要确保每个 GPU 处理的 token 数量大致相同。为此引入冗余专家的部署策略，即复制高负载专家并进行冗余部署。高负载专家是根据在线部署期间收集的统计信息检测出来的，并定期进行调整（例如每 10 分钟一次）。在确定冗余专家集合后，根据观察到的负载在节点内的 GPU 之间仔细重新安排专家，力求在不增加跨节点全对全通信开销的情况下，尽可能平衡各 GPU 之间的负载。DeepSeek-V3 在预填充阶段设置 32 个冗余专家。对于每个 GPU，除了其原本承载的 8 个专家外，还将额外承载一个冗余专家。

此外，在预填充阶段，为了提高吞吐量并隐藏全对全通信和张量并行通信的开销，同时处理两个计算工作量相似的微批次，将一个微批次的注意力和 MoE 操作与另一个微批次的分发和合并操作重叠。

最后，DeepSeek-V3 正在探索一种专家动态冗余策略，即每个 GPU 承载更多的专家（例如 16 个专家），但在每次推理步骤中仅激活 9 个。在每一层的全对全操作开始之前，DeepSeek-V3 实时计算全局最优路由方案。鉴于预填充阶段涉及大量计算，计算此路由方案的开销几乎可以忽略不计。

9.4.4.2、解码

在解码过程中，将共享专家视为被路由的专家。从这个角度看，每个令牌在路由过程中会选择 9 个专家，其中共享专家被视为高负载专家，总会被选中。解码阶段的最小部署单元由 40 个节点共 320 个 GPU 组成。注意力部分采用结合序列并行（SP）的 4 路张量并行（TP4），以及 80 路数据并行（DP80），而混合专家（MoE）部分则使用 320 路专家并行（EP320）。在 MoE 部分，每个 GPU 仅承载一个专家，64 个 GPU 负责承载冗余专家和共享专家。分发和合并部分的全对全通信通过 IB 上的直接点对点传输来实现，以达到低延迟。此外，DeepSeek-V3 利用 IBGDA 技术进一步降低延迟并提高通信效率。

与预填充类似，根据在线服务中专家负载的统计数据，在特定间隔内定期确定冗余专家集合。不过，由于每个 GPU 仅承载一个专家，因此无需重新安排专家。此外，为了提高吞吐量并隐藏全对全通信的开销，DeepSeek-V3 还在探索在解码阶段同时处理两个计算工作量相似的微批次。与预填充不同，在解码阶段注意力部分消耗的时间占比更大。因此，将一个微批次的注意力操作与另一个微批次的分发 + MoE + 合并操作重叠。在解码阶段，每个专家的批次大小相对较小（通常在 256 个令牌以内），瓶颈在于内存访问而非计算。由于 MoE 部分只需要加载一个专家的参数，内存访问开销极小，所以使用较少的流式多处理器（SM）不会显著影响整体性能。因此，为避免影响注意力部分的计算速度，可以仅分配一小部分 SM 给分发 + MoE + 合并操作。

9.4.5、预训练

9.4.5.1、数据构建

与 DeepSeek-V2 相比，DeepSeek-V3 通过提高数学和编程样本的比例对预训练语料库进行了优化，同时将多语言覆盖范围扩展到英语和中文之外。此外，DeepSeek-V3 对数据处理流程进行了优化，在保持语料库多样性的同时尽量减少冗余。DeepSeek-V3 采用文档打包方法以保证数据完整性，但在训练过程中不采用跨样本注意力掩码。最终，DeepSeek-V3 的训练语料库在分词器中有 14.8T 高质量且多样的词元。

在 DeepSeekCoder-V2 的训练过程中，“填中间”（Fill-in-Middle, FIM）策略在不损害下一词元预测能力的同时，能使模型基于上下文线索准确预测中间文本。与 DeepSeekCoder-V2 一致，在 DeepSeek-V3 的预训练中也采用了 FIM 策略。具体来说，使用“前缀-后缀-中间”（Prefix-Suffix-Middle, PSM）框架按如下方式构建数据：

$$\langle \text{fim_begin} \rangle f_{\text{pre}} \langle \text{fim_hole} \rangle f_{\text{suf}} \langle \text{fim_end} \rangle f_{\text{middle}} \langle \text{eos_token} \rangle$$

这种结构在文档层面作为预打包过程的一部分应用。FIM 策略以 0.1 的比例应用，与 PSM 框架一致。

DeepSeek-V3 的分词器采用字节级字节对编码 (Byte-level BPE)，词汇表扩展到 12.8 万个词元。DeepSeek-V3 对分词器的预分词器和训练数据进行了修改，以优化多语言压缩效率。此外，与 DeepSeek-V2 相比，新的预分词器引入了将标点符号和换行符组合在一起的词元。然而，当模型处理无末尾换行符的多行提示时，尤其是在少样本评估提示中，这种方法可能会引入词元边界偏差。为解决这一问题，DeepSeek-V3 在训练过程中随机拆分一定比例的此类组合词元，这使模型接触到更广泛的特殊情况，从而减轻这种偏差。

9.4.5.2、超参数

1、模型超参数

DeepSeek-V3 将 Transformer 层数设置为 61 层，隐藏维度设置为 7168。所有可学习参数均以 0.006 的标准差进行随机初始化。在多尺度局部注意力 (MLA) 中，DeepSeek-V3 将注意力头的数量 n_h 设置为 128，每个头的维度 d_h 设置为 128。键值 (KV) 压缩维度 d_c 设置为 512，查询压缩维度 d'_c 设置为 1536。对于解耦的 query 和 key，将每个头的维度 d_h^R 设置为 64。除了前三层，DeepSeek-V3 将所有前馈神经网络 (FFN) 替换为混合专家 (MoE) 层。每个 MoE 层由 1 个共享专家和 256 个路由专家组成，每个专家的中间隐藏维度为 2048。在路由专家中，每个词元会激活 8 个专家，并且确保每个词元最多被发送到 4 个节点。多词元预测深度 D 设置为 1，即除了预测紧邻的下一个词元外，每个词元还会额外预测一个词元。与 DeepSeek-V2 一样，DeepSeek-V3 也在压缩后的潜在向量之后采用额外的 RMSNorm 层，并在宽度瓶颈处乘以额外的缩放因子。在此配置下，DeepSeek-V3 总共有 6710 亿个参数，每个词元会激活 37 亿个参数。

2、训练超参数

DeepSeek-V3 采用 AdamW 优化器，超参数设置为 $\beta_1 = 0.9$ ， $\beta_2 = 0.95$ ，权重衰减为 0.1。在预训练期间，将最大序列长度设置为 4K，并在 14.8T 词元上对 DeepSeek-V3 进行预训练。关于学习率调度，在前 2000 步中，先将学习率从 0 线性增加到 2.2×10^{-4} 。然后，在模型处理 10T 训练词元之前，保持 2.2×10^{-4} 的恒定学习率。随后，在接下来的 4.3T 词元训练过程中，按照余弦退火曲线将学习率逐渐衰减至 2.2×10^{-5} 。在最后 5000 亿词元的训练中，前 3330 亿词元保持 2.2×10^{-5} 的恒定学习率，在剩余的 1670 亿词元中切换到 7.3×10^{-6} 的恒定学习率。梯度裁剪范数设置为 1.0。DeepSeek-V3 采用批量大小调度策略，在训练前 4690 亿词元时，批量大小从 3072 逐渐增加到

15360，在剩余的训练过程中保持 15360。DeepSeek-V3 利用流水线并行技术将模型的不同层部署在不同的 GPU 上，对于每一层，路由专家将均匀部署在属于 8 个节点的 64 个 GPU 上。对于节点限制路由，每个词元最多被发送到 4 个节点（即 $M = 4$ ）。为实现无辅助损失的负载均衡，在前 14.3T 词元的训练中，将偏差更新速度 γ 设置为 0.001，在剩余的 5000 亿词元训练中设置为 0.0。对于平衡损失，将 α 设置为 0.0001，只是为了避免任何单个序列内出现极端不平衡。多词元预测（MTP）损失权重 λ 在前 10T 词元设置为 0.3，在剩余的 4.8T 词元设置为 0.1。

9.4.5.3、长上下文扩展

DeepSeek-V3 采用了与 DeepSeek-V2 类似的方法来实现长上下文处理能力。在预训练阶段之后，应用 YaRN 进行上下文扩展，并执行两个额外的训练阶段，每个阶段包含 1000 步，以逐步将上下文窗口从 4K 扩展到 32K，然后再扩展到 128K。YaRN 的配置与 DeepSeek-V2 中使用的配置一致，仅应用于解耦的共享键 k_t^R 。两个阶段的超参数保持相同，缩放比例 $s = 40$ ， $\alpha = 1$ ， $\beta = 32$ ，缩放因子 $\sqrt{t} = 0.1 \ln s + 1$ 。在第一阶段，序列长度设置为 32K，批量大小为 1920。在第二阶段，序列长度增加到 128K，批量大小减小到 480。两个阶段的学习率均设置为 7.3×10^{-6} ，与预训练阶段的最终学习率相匹配。

通过这两个阶段的扩展训练，DeepSeek-V3 能够处理长度高达 128K 的输入，同时保持强大的性能。

9.4.6、后训练

9.4.6.1、监督式微调

DeepSeek-V3 指令调整数据集包含 150 万个跨越多个领域的实例，每个领域都根据其特定需求采用不同的数据创建方法。

1、推理数据

对于与推理相关的数据集，包括专注于数学、代码竞赛问题和逻辑谜题的数据集，借助 DeepSeek-R1 模型生成数据。具体而言，虽然 R1 生成的数据准确率很高，但存在诸如过度思考、格式不佳以及篇幅过长等问题。DeepSeek-V3 目标是在 R1 生成的推理数据的高准确率与常规格式推理数据的清晰简洁之间找到平衡。

DeepSeek-V3 首先针对特定领域（如代码、数学或通用推理），使用监督微调（SFT）和强化学习（RL）相结合的训练流程开发一个专家模型。这个专家模型将作为最终模型的数据生成器。训练过程涉及为每个实例生成两种不同类型的 SFT 样本：第一种是以 <问题，原始回答> 的格式将问题与其原始回答配对，而第二种则是在问题和 R1 回答的基础上加入系统提示，格式为 <系统提示，问题，R1回答>。

系统提示经过精心设计，包含引导模型生成富含反思与验证机制的回答的指令。在强化学习阶段，即使没有明确的系统提示，模型也会利用高温采样生成融合了 R1 生成数据和原始数据模式的回答。经过数百步强化学习后，中间的强化学习模型学会融入 R1 模式，从而从策略上提升整体性能。

在完成强化学习训练阶段后，DeepSeek-V3 采用拒绝采样为最终模型精心挑选高质量的 SFT 数据，此时专家模型被用作数据生成源。这种方法确保最终的训练数据既保留了 DeepSeek-R1 的优势，又能生成简洁有效的回答。

2、非推理数据

对于非推理数据，如创意写作、角色扮演和简单问答，DeepSeek-V3 利用 DeepSeek-V2.5 生成回答，并聘请人工标注员来验证数据的准确性和正确性。

3、SFT设置

DeepSeek-V3 使用 SFT 数据集对 DeepSeek-V3-Base 进行两个轮次的微调，采用余弦退火学习率调度，起始学习率为 5×10^{-6} ，并逐渐降至 1×10^{-6} 。在训练过程中，每个单一序列由多个样本打包而成。DeepSeek-V3 采用样本掩码策略，以确保这些示例相互隔离且不可见。

9.4.6.2、强化学习

1、奖励模型

在强化学习（RL）过程中，DeepSeek-V3 采用了基于规则的奖励模型（RM）和基于模型的奖励模型。

a、基于规则的奖励模型

对于那些可以用特定规则验证的问题，DeepSeek-V3 采用基于规则的奖励系统来确定反馈。例如，某些数学问题有确切的答案，要求模型按照指定格式（如放在方框内）给出最终答案，以便运用规则来验证其正确性。同样，对于力扣（LeetCode）上的题目，可以利用编译器根据测试用例生成反馈。只要有可能，DeepSeek-V3 就利用基于规则的验证，以确保更高的可靠性，因为这种方法不易被操纵或利用。

b、基于模型的奖励模型

对于那些参考答案形式自由的问题，DeepSeek-V3 依靠奖励模型来判断回答是否符合预期的参考答案。相反，对于没有明确参考答案的问题，比如创意写作类的问题，奖励模型则根据问题及相应的回答作为输入来提供反馈。该奖励模型是从 DeepSeek-V3 的有监督微调（SFT）检查点训练得到的。为了提高其可靠性，DeepSeek-V3 构建了偏好数据，这些数据不仅给出最终奖励，还包含得出该奖励的推理过程。这种方法有助于降低在特定任务中出现奖励操控行为的风险。

2、群组相对策略优化

与 DeepSeek-V2 类似，DeepSeek-V3 采用群组相对策略优化（Group Relative Policy Optimization, GRPO），该方法摒弃了通常与策略模型规模相同的价值评判模型，而是从组分数中估计基线。具体而言，对于每个问题 q ，GRPO 从旧策略模型 $\pi_{\theta_{old}}$ 中采样一组输出 $\{o_1, o_2, \dots, o_G\}$ ，然后通过最大化以下目标来优化策略模型 π_{θ} ：

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E} \left[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O | q) \right]$$

$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i | q)}{\pi_{\theta_{old}}(o_i | q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i | q)}{\pi_{\theta_{old}}(o_i | q)}, 1 - \varepsilon, 1 + \varepsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right)$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i | q)}{\pi_{\theta}(o_i | q)} - \log \frac{\pi_{ref}(o_i | q)}{\pi_{\theta}(o_i | q)} - 1$$

其中， ε 和 β 是超参数； π_{ref} 是参考模型； A_i 是优势值，由每组输出对应的奖励 $\{r_1, r_2, \dots, r_G\}$ 得出：

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}$$

在强化学习过程中纳入了来自不同领域的提示，如编码、数学、写作、角色扮演和问答等。这种方法不仅使模型更符合人类偏好，还提升了在基准测试中的性能，特别是在可用的有监督微调（SFT）数据有限的场景下。

9.5、DeepSeek-R1

DeepSeek-R1-Zero 和 DeepSeek-R1 发布于 2025 年 1 月。DeepSeek-R1-Zero 通过大规模强化学习训练而成，该模型在不使用任何 SFT 数据的情况下，直接将强化学习应用于基础模型。DeepSeek-R1 则在强化学习之前结合了多阶段训练和冷启动数据。

DeepSeek-R1-Zero 直接将强化学习应用于基础模型，而不依赖于监督微调作为初步步骤。其展现了自我验证、反思以及生成长思维链等能力，证实了大型语言模型（LLM）通过大规模强化学习（RL）可以显著提高推理能力，甚至无需以监督微调（SFT）作为冷启动方式。此外，加入少量冷启动数据，性能还能进一步提升。

此外，DeepSeek-R1 发现较大模型的推理模式可以被蒸馏到较小模型中，从而在推理性能上超越通过在小模型上进行强化学习发现的推理模式。

9.5.1、DeepSeek-R1-Zero：在基础模型上进行强化学习

9.5.1.1、强化学习算法

群组相对策略优化

为降低强化学习的训练成本，DeepSeek-R1-Zero 采用群组相对策略优化（Group Relative Policy Optimization, GRPO）。该方法舍弃了通常与策略模型规模相同的价值评估模型，而是从组得分中估计基线。具体而言，对于每个问题 q ，GRPO 从旧策略 $\pi_{\theta_{old}}$ 中采样一组输出 $\{o_1, o_2, \dots, o_G\}$ ，然后通过最大化以下目标来优化策略模型 π_{θ} ：

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E} \left[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O | q) \right]$$

$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i | q)}{\pi_{\theta_{old}}(o_i | q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i | q)}{\pi_{\theta_{old}}(o_i | q)}, 1 - \varepsilon, 1 + \varepsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{\theta_{old}}) \right)$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{\theta_{old}}) = \frac{\pi_{\theta}(o_i | q)}{\pi_{\theta_{old}}(o_i | q)} - \log \frac{\pi_{\theta}(o_i | q)}{\pi_{\theta_{old}}(o_i | q)} - 1$$

其中， ε 和 β 是超参数， A_i 是优势值，使用与每组输出对应的一组奖励 $\{r_1, r_2, \dots, r_G\}$ 计算得出：

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}$$

9.5.1.2、奖励建模

奖励是训练信号的来源，决定了强化学习的优化方向。DeepSeek-R1-Zero 采用了一种基于规则的奖励系统，该系统主要包含两种奖励类型：

- **准确率奖励**：准确率奖励模型用于评估回复是否正确。例如，对于有确定答案的数学问题，要求模型以指定格式（例如，在框内）给出最终答案，以便基于规则可靠地验证其正确性。同样，对于力扣（LeetCode）问题，可以使用编译器根据预定义的测试用例生成反馈
- **格式奖励**：除了准确率奖励模型，还采用了格式奖励模型，该模型强制模型将其思考过程置于 `<think>` 和 `</think>` 标签之间。

在开发 DeepSeek-R1-Zero 时，没有应用结果或过程神经奖励模型，因为神经奖励模型在大规模强化学习过程中可能会遭遇奖励作弊问题，而且重新训练奖励模型需要额外的训练资源，这会使整个训练流程变得复杂。

9.5.1.3、训练模板

为了训练 DeepSeek-R1-Zero，首先设计一个简单的模板，引导基础模型遵循指定的指令。如表所示，该模板要求 DeepSeek-R1-Zero 首先生成一个推理过程，然后给出最终答案。DeepSeek-R1-Zero 特意将约束限定在这种结构格式上，避免任何特定于内容的偏向，比如强制进行反思性推理或推崇特定的解题策略，以确保能够准确观察模型在强化学习过程中的自然进展。

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within <think> </think> and <answer> </answer> tags, respectively, i.e., <think> reasoning process here </think> <answer> answer here </answer>. User: **prompt**. Assistant:

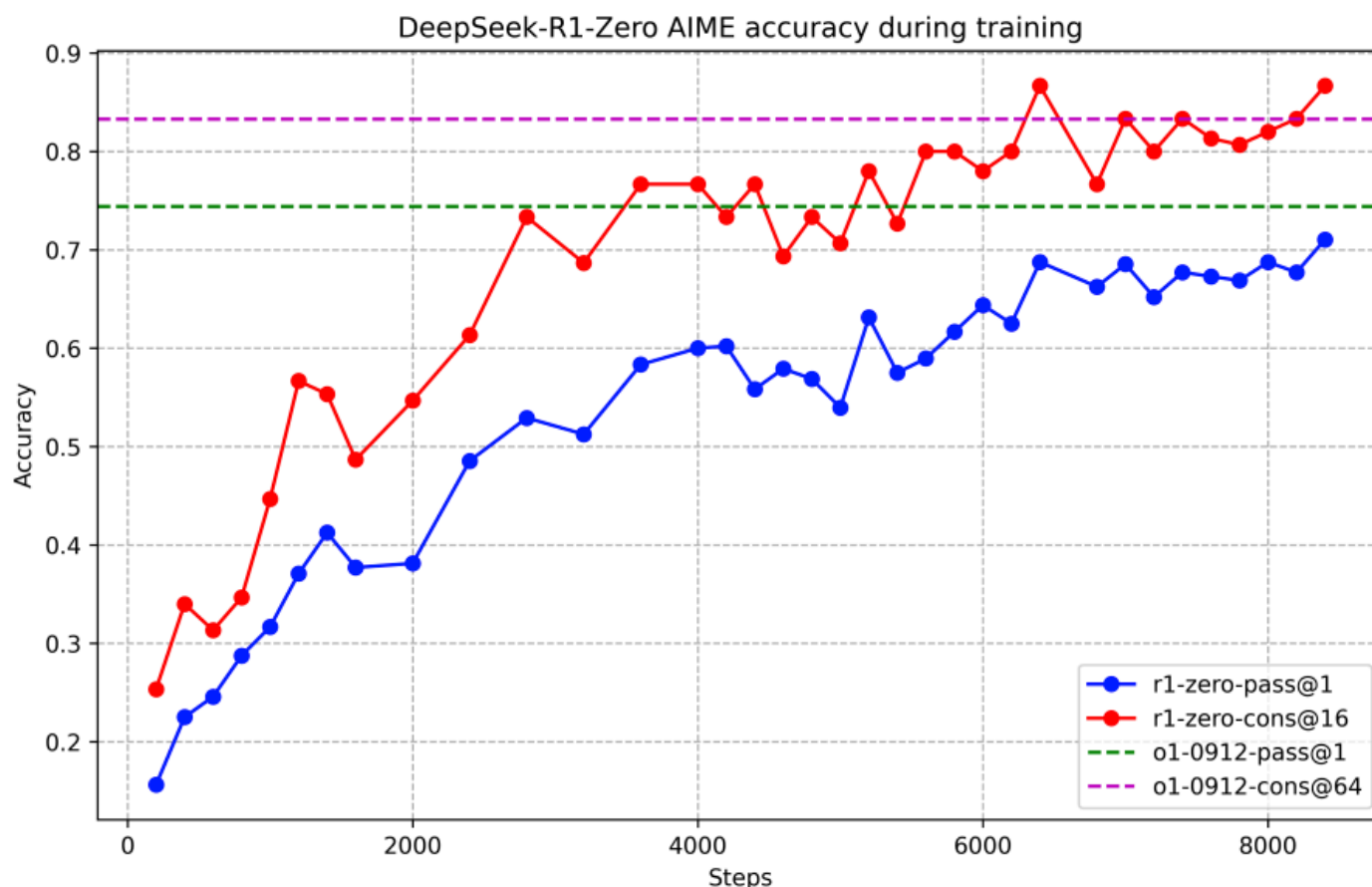
(/img/post/llm-model-57.png)

DeepSeek-R1-Zero 的模板。在训练过程中，“prompt”（提示）将被具体的推理问题所取代

9.5.1.4、DeepSeek-R1-Zero 的性能、自我进化过程以及顿悟时刻

1、DeepSeek-R1-Zero 的性能

如图展示了 DeepSeek-R1-Zero 在 2024 年美国数学邀请赛（AIME）基准测试中，整个强化学习训练过程的性能轨迹。随着强化学习训练的推进，DeepSeek-R1-Zero 的性能呈现出稳定且持续提升。



(/img/post/llm-model-58.png)

训练过程中 DeepSeek-R1-Zero 在 AIME（美国数学邀请赛）上的准确率。对于每个问题采样 16 个回答，并计算总体平均准确率，以确保评估的稳定性

下表对 DeepSeek-R1-Zero 和 OpenAI 的 o1-0912 模型在各种推理相关基准测试上进行了对比分析。结果表明，强化学习使 DeepSeek-R1-Zero 无需任何监督微调数据就能获得强大的推理能力。此外，通过应用多数投票法，DeepSeek-R1-Zero 的性能还能进一步提升。例如，在 AIME 基准测试中采用多数投票法时，DeepSeek-R1-Zero 的性能从 71.0% 提升至 86.7%，从而超过了 OpenAI-o1-0912 的性能。

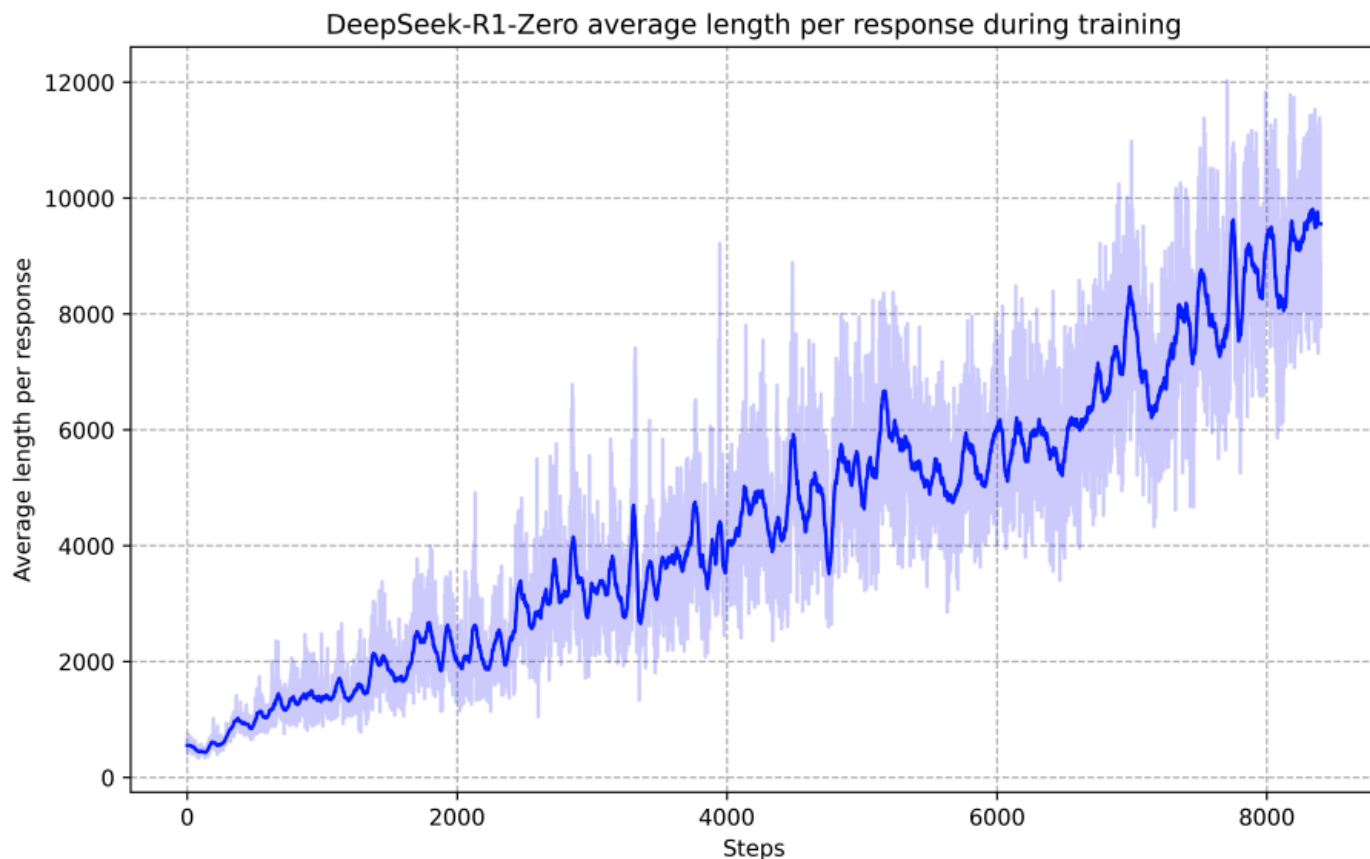
Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
OpenAI-o1-0912	74.4	83.3	94.8	77.3	63.4	1843
DeepSeek-R1-Zero	71.0	86.7	95.9	73.3	50.0	1444

(/img/post/llm-model-59.png)

2、DeepSeek-R1-Zero 的自我进化过程

DeepSeek-R1-Zero 的自我进化过程生动地展示了强化学习如何驱动模型自主提升其推理能力。通过直接从基础模型启动强化学习，能够在不受监督微调阶段影响的情况下，密切监测模型的进展。这种方法清晰地呈现了模型随时间的演变，尤其是在处理复杂推理任务的能力方面。

如图所示，DeepSeek-R1-Zero 的思考时间在整个训练过程中持续改善。这种改善并非外部调整的结果，而是模型内部的固有发展。DeepSeek-R1-Zero 通过利用更长的测试时间计算，自然地获得了解决日益复杂推理任务的能力。这种计算范围从生成数百到数千个推理标记不等，使模型能够更深入地探索和完善其思考过程。



(/img/post/llm-model-60.png)

在强化学习（RL）过程中，DeepSeek-R1-Zero 在训练集上的平均回复长度。DeepSeek-R1-Zero 自然而然地学会了花费更多思考时间来解决推理任务

自我演化的最显著特点之一是随着测试时计算的增加，出现了复杂的行为。比如反思行为——模型回顾并重新评估其先前步骤——以及探索解决问题的替代方法，这些行为都自发产生。这些行为并非明确编程设定，而是模型与强化学习环境相互作用的结果。这种自发发展显著增强了 DeepSeek-R1-Zero 的推理能力，使其能够以更高的效率和准确性应对更具挑战性的任务。

3、DeepSeek-R1-Zero 的顿悟时刻

在 DeepSeek-R1-Zero 的训练过程中观察到一个特别有趣的现象，即“顿悟时刻（aha moment）”的出现。如表所示，这一时刻出现在模型的一个中间版本中。在此阶段，DeepSeek-R1-Zero 学会了通过重新评估其初始方法，为一个问题分配更多思考时间。这种行为不仅证明了模型推理能力的不断增强，也是强化学习如何能带来意想不到且复杂结果的一个引人入胜的例子。

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both ...

$$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a + x}} = x$$

First, let's square both sides:

$$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$$

Next, I could square both sides again, treating the equation: ...

...

(/img/post/llm-model-61.png)

DeepSeek-R1-Zero 一个中间版本出现的有趣“顿悟时刻”。该模型学会了以拟人化的口吻重新思考。对我们来说，这同样是一个顿悟时刻，让我们见证了强化学习的力量与美妙

这个时刻不仅对模型来说是一个“顿悟时刻”，对观察其行为的研究人员来说也是如此。它突显了强化学习的力量与美妙之处：我们并非明确教导模型如何解决问题，而只是为其提供正确的激励，它就能自主开发出先进的问题解决策略。“顿悟时刻”有力地提醒我们，强化学习有潜力在人工系统中解锁新的智能水平，为未来更自主、自适应的模型发展铺平道路。

4、DeepSeek-R1-Zero 的缺点

尽管 DeepSeek-R1-Zero 展现出强大的推理能力，并自主发展出意想不到且强大的推理行为，但它面临一些问题。例如，DeepSeek-R1-Zero 在可读性差和语言混用等挑战上存在困难。为了使推理过程更具可读性并与开放社区分享，从而探索了 DeepSeek-R1，这是一种结合人类友好型冷启动数据使用强化学习的方法。

9.5.2、DeepSeek-R1：带有冷启动的强化学习

受 DeepSeek-R1-Zero 取得的出色成果启发，两个很自然的问题出现了：

1. 通过引入少量高质量数据作为冷启动，能否进一步提升推理性能或加快收敛速度？
2. 如何训练一个既能够生成清晰连贯的思维链（CoT），又具备强大通用能力的用户友好型模型？

为解决这些问题，设计一个用于训练 DeepSeek-R1 的流程。该流程包含四个阶段，具体如下。

9.5.2.1、冷启动

与 DeepSeek-R1-Zero 不同，为避免从基础模型进行强化学习训练时早期冷启动阶段的不稳定，对于 DeepSeek-R1 构建并收集了少量长思维链（CoT）数据，对模型进行微调，以此作为初始的强化学习行动者。为收集此类数据探索了多种方法：以长思维链作为示例进行少样本提示，直接促使模型生成带有反思与验证的详细答案，收集格式可读的 DeepSeek-R1-Zero 输出结果，并由人工注释者通过后处理优化这些结果。

在这项工作中收集了数千条冷启动数据，对 DeepSeek-V3-Base 进行微调，将其作为强化学习的起点。与 DeepSeek-R1-Zero 相比，冷启动数据的优势包括：

- **可读性**：DeepSeek-R1-Zero 的一个关键局限在于其内容往往不适合阅读。回复可能夹杂多种语言，或者缺少用于为用户突出答案的 Markdown 格式。相比之下，在为 DeepSeek-R1 创建冷启动数据时设计了一种可读模式，在每个回复的结尾处添加总结，并过滤掉不便于阅读的回复。在此，将输出格式定义为 |特殊标记|<推理过程>|特殊标记|<总结>，其中推理过程是针对查询的思维链，总结则用于概括推理结果
- **潜力**：通过借助人先验知识精心设计冷启动数据的模式，观察到其性能优于 DeepSeek-R1-Zero。迭代训练对于推理模型而言是一种更优的方式

9.5.2.2、以推理为导向的强化学习

DeepSeek-R1 训练数据的收集方式如下：

在冷启动数据上对 DeepSeek-V3-Base 进行微调后，采用与 DeepSeek-R1-Zero 相同的大规模强化学习训练流程。此阶段着重提升模型的推理能力，尤其是在诸如编码、数学、科学及逻辑推理等推理密集型任务上，这些任务涉及定义明确且有清晰解法的问题。

在训练过程中注意到思维链（CoT）经常出现语言混用的情况，特别是当强化学习提示涉及多种语言时。为缓解语言混用问题，在强化学习训练中引入语言一致性奖励，该奖励按思维链中目标语言词汇的比例来计算。尽管消融实验表明，这种调整会使模型性能略有下降，但该奖励符合人类偏好，使输出更具可读性。最后，通过直接将推理任务的准确率与语言一致性奖励相加，形成最终奖励。然后，对微调后的模型进行强化学习训练，直至其在推理任务上达到收敛。

9.5.2.3、拒绝采样和监督式微调

当面向推理的强化学习收敛后，利用得到的检查点为下一轮收集监督微调（SFT）数据。与最初主要聚焦于推理的冷启动数据不同，此阶段纳入了来自其他领域的的数据，以提升模型在写作、角色扮演及其他通用任务方面的能力。具体而言，按如下方式生成数据并微调模型。

1、推理数据

DeepSeek-R1 精心整理推理提示，并通过对上述强化学习训练得到的检查点进行拒绝采样来生成推理轨迹。在前一阶段，仅纳入了能够使用基于规则的奖励进行评估的数据。然而在本阶段，通过纳入更多数据来扩充数据集，其中一些数据借助生成式奖励模型，将真实值与模型预测结果输入 DeepSeek-V3 进行判断。此外，由于模型输出有时杂乱且难以阅读，过滤掉了语言混杂的思维链、冗长段落以及代码块。对于每个提示，采样多个回复，仅保留正确的回复。总体而言，收集了约 60 万个与推理相关的训练样本。

2、非推理数据

对于非推理数据，如写作、事实性问答、自我认知以及翻译等，采用 DeepSeek-V3 流程，并复用 DeepSeek-V3 的部分监督微调数据集。对于某些非推理任务，在通过提示回答问题之前，调用 DeepSeek-V3 生成一个潜在的思维链。不过，对于诸如“你好”这类简单查询，回复中不提供思维链。最终，总共收集了约 20 万个与推理无关的训练样本。

使用上述精心整理的约 80 万个样本的数据集，对 DeepSeek-V3-Base 进行两个轮次的微调。

9.5.2.4、所有场景下的强化学习

为了让模型更贴合人类偏好，DeepSeek-V3 实施了第二阶段强化学习，旨在提升模型的有用性与无害性，同时优化其推理能力。具体来说，DeepSeek-V3 结合奖励信号和多样化的提示分布来训练模型。

对于推理数据，遵循 DeepSeek-R1-Zero 中概述的方法，利用基于规则的奖励，在数学、代码和逻辑推理领域引导学习过程。

对于通用数据，借助奖励模型，捕捉人类在复杂微妙场景中的偏好。基于 DeepSeek-V3 流程进行构建，并采用类似的偏好对分布和训练提示。

在有用性方面，只关注最终总结，确保评估着重于回复对用户的实用性和相关性，同时尽量减少对底层推理过程的干扰。

在无害性方面，评估模型的整个回复，包括推理过程和总结，以识别并减轻生成过程中可能出现的任何潜在风险、偏差或有害内容。

最终，奖励信号与多样化数据分布的整合，训练出一个在优先确保有用性和无害性的同时，推理能力出色的模型。

9.5.3、蒸馏：赋予小型模型推理能力

为了让更高效的小模型具备类似 DeepSeek-R1 的推理能力，直接使用为 DeepSeek-R1 精心整理的 80 万个样本，对如 Qwen 和 LLaMa 这样的开源模型进行微调。结果表明，这种简单直接的提炼方法显著提升了小模型的推理能力。

对于提炼后的模型仅应用监督微调（SFT），不纳入强化学习（RL）阶段，尽管加入强化学习能大幅提升模型性能。在此的主要目的是展示提炼技术的有效性，将强化学习阶段的探索留给更广泛的研究群体。

参考文献

1. Attention Is All You Need (<https://arxiv.org/pdf/1706.03762>)
2. The Illustrated Transformer (<https://jalammar.github.io/illustrated-transformer/>)
3. Transformer Architecture: The Positional Encoding
(https://kazemnejad.com/blog/transformer_architecture_positional_encoding/)
4. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
(<https://arxiv.org/pdf/1810.04805>)
5. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter
(<https://arxiv.org/pdf/1910.01108>)
6. ALBERT: A LITE BERT FOR SELF-SUPERVISED LEARNING OF LANGUAGE REPRESENTATIONS (<https://arxiv.org/pdf/1909.11942>)
7. RoBERTa: A Robustly Optimized BERT Pretraining Approach
(<https://arxiv.org/pdf/1907.11692>)
8. ELECTRA: PRE-TRAINING TEXT ENCODERS AS DISCRIMINATORS RATHER THAN GENERATORS (<https://arxiv.org/pdf/2003.10555>)
9. SpanBERT: Improving Pre-training by Representing and Predicting Spans
(<https://arxiv.org/pdf/1907.10529>)
10. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension (<https://arxiv.org/pdf/1910.13461>)
11. Improving Language Understanding by Generative Pre-Training
(https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf)
12. Language Models are Unsupervised Multitask Learners (https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf)
13. Release Strategies and the Social Impacts of Language Models
(<https://arxiv.org/pdf/1908.09203>)
14. Language Models are Few-Shot Learners (<https://arxiv.org/pdf/2005.14165>)
15. Generating Long Sequences with Sparse Transformers (<https://arxiv.org/pdf/1904.10509>)
16. A Survey of Transformers (<https://arxiv.org/pdf/2106.04554>)
17. Training language models to follow instructions with human feedback
(<https://arxiv.org/pdf/2203.02155>)
18. GPT-4 Technical Report (<https://arxiv.org/pdf/2303.08774>)
19. Language Models are Unsupervised Multitask Learners (https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf)
20. LLaMA: Open and Efficient Foundation Language Models (<https://arxiv.org/pdf/2302.13971>)
21. LLaMA explained: KV-Cache, Rotary Positional Embedding, RMS Norm, Grouped Query Attention, SwiGLU (<https://www.youtube.com/@umarjamilai>)